

Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

SBOM Lens: A Graph-Based Framework for Software Supply Chain Risk Assessment

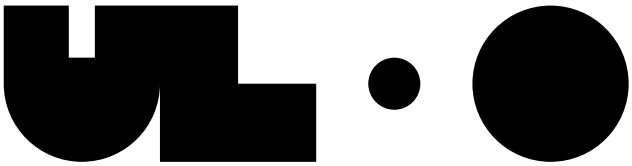
Structural Metrics and Network Topology Analysis of Software Dependency Networks

Igor Ferreira do Nascimento

School of Technology and Management
Department of Computer Engineering
Master in Cybersecurity & Digital Forensics

Leiria, October 2026

This page intentionally left blank.



Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

SBOM Lens: A Graph-Based Framework for Software Supply Chain Risk Assessment

Structural Metrics and Network Topology Analysis of Software Dependency Networks

Igor Ferreira do Nascimento
Student No. 2240253

Supervisor: João da Silva Pereira
Full Professor, University of Leiria and Oeste

School of Technology and Management
Department of Computer Engineering
Master in Cybersecurity & Digital Forensics

Project

Leiria, October 2026

This page intentionally left blank.

SBOM Lens: A Graph-Based Framework for Software Supply Chain Risk Assessment

Copyright © 2026 - Igor Ferreira do Nascimento, School of Technology and Management.

This dissertation is original work, written solely for this purpose, and all the authors whose studies and publications contributed to it have been duly cited. Partial reproduction is allowed with acknowledgment of the author and reference to the degree, academic year, institution — *University of Leiria and Oeste* — and public defense date.



The present work benefited from the use of the *ULO-Thesis* template (derived from the *IPLeiria-Thesis* template developed by José Areia).

This page intentionally left blank.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Professor João da Silva Pereira, for his invaluable guidance, support, and encouragement throughout the course of this project. Also i would like to thank my college professor Daniel Ratton for introducing me to the world of network science, which was fundamental for the development of this project. I am also grateful to my colleagues and friends who provided encouragement and support during the research process. Finally, I would like to thank my family for their support, in particular my grandma and my father-in-law, who always belived in me and encouraged me to pursue my academic goals. This project would not have been possible without the support and encouragement of all these people, and I am deeply grateful for their presence in my life and for been the reason I am where I am.

This page intentionally left blank.

Resumo

A crescente adoção de componentes de software de terceiros aumenta a superfície de ataque das cadeias de fornecimento, tornando difícil para as organizações compreender e mitigar os riscos introduzidos pelas suas dependências. O Software Bill of Materials (SBOM) emerge como um mecanismo de transparência, mas os formatos existentes carecem de ferramentas analíticas que transformem dados de inventário em inteligência de risco acionável.

Esta dissertação apresenta o SBOM Lens, uma framework baseada em grafos para avaliação de risco em cadeias de fornecimento de software. O sistema ingere SBOMs no formato CycloneDX e constrói um grafo de dependências orientado, sobre o qual aplica técnicas de ciência de redes para expor vulnerabilidades estruturais. O trabalho cobre quatro domínios principais: (1) a visualização interativa do grafo SBOM, que permite explorar e editar a topologia das dependências; (2) a gestão de vulnerabilidades, com integração da base de dados OSV, que agrega os GitHub Security Advisories e outras fontes, incluindo o cálculo de scores CVSS v3.1; (3) a avaliação de risco estrutural, que identifica pontos de falha única, dependências circulares, conflitos de versão e concentração de dependências através de métricas de entropia e decomposição em núcleos; e (4) a análise de grafos, que aplica algoritmos de deteção de comunidades, decomposição k-core e análise espetral para estimar a facilidade com que um comprometimento se poderia propagar pelo grafo.

Em 90 aplicações npm reais, as ordenações estruturais e por severidade dos componentes vulneráveis mostraram-se praticamente não relacionadas, e três quartos dos componentes vulneráveis eram dependências apenas transitivas. O SBOM Lens demonstra que a teoria dos grafos e a ciência de redes fornecem, para os riscos de segurança em cadeias de fornecimento de software modernas, um contexto que a severidade, por si só, não oferece, e uma base para os priorizar.

Palavras-Chave: SBOM, Segurança de Cadeias de Fornecimento de Software, Teoria dos Grafos, Análise de Vulnerabilidades, Avaliação de Risco, CycloneDX.

This page intentionally left blank.

Abstract

The widespread adoption of third-party software components continuously expands the attack surface of modern software supply chains, making it difficult for organisations to understand and mitigate the risks introduced by their dependencies. The Software Bill of Materials (SBOM) has emerged as a transparency mechanism, yet existing tooling largely treats SBOMs as flat inventories rather than structured graphs amenable to rigorous analysis.

This dissertation presents SBOM Lens, a graph-based framework for software supply chain risk assessment. The system ingests SBOMs in CycloneDX format and constructs a directed dependency graph, over which network science techniques are applied to expose structural vulnerabilities. The work covers four main domains: (1) interactive SBOM graph visualisation, enabling exploration and editing of the dependency topology; (2) vulnerability management, integrating the OSV database — which aggregates GitHub Security Advisories and other sources — with full CVSS v3.1 base score computation; (3) structural risk assessment, which identifies single points of failure via articulation point and dominance analysis, circular dependencies, version conflicts, and dependency concentration using entropy metrics and k-core decomposition; and (4) graph analysis, applying community detection, k-core decomposition, and spectral graph theory to estimate how readily a compromise could spread through the graph.

On 90 real npm applications, structural and severity rankings of vulnerable components were essentially unrelated, and three-quarters of vulnerable components were transitive-only. SBOM Lens demonstrates that graph theory and network science provide context for security risk in modern software supply chains that severity alone does not, and a basis for prioritising it.

Keywords: SBOM, Software Supply Chain Security, Graph Theory, Vulnerability Analysis, Risk Assessment, CycloneDX.

This page intentionally left blank.

Declaration on the Use of Artificial Intelligence

I acknowledge the use of Claude (https://claude.ai) to refine the academic tone and improve the linguistic accuracy of this work, including aspects of grammar, punctuation, and vocabulary.

This page intentionally left blank.

Contents

Declaration on the Use of Artificial Intelligence **vii**

List of Figures xiv

List of Tables xvi

1 Introduction **1**

1.1 Motivation . . . 1
1.2 Problem Statement . . . 3
1.3 Research Objectives . . . 4
1.4 Contributions . . . 5
1.5 Scope and Limitations . . . 6
1.6 Thesis Structure . . . 7

2 Background **8**

2.1 Software Bill of Materials and CycloneDX . . . 8
2.1.1 What is a Software Bill of Materials? . . . 8
2.1.2 SBOM Format Landscape . . . 9
2.1.3 CycloneDX JSON Structure and SBOM Lens Fields . . . 10
2.2 Software Supply Chain Security . . . 11
2.2.1 The Software Supply Chain . . . 11
2.2.2 Attack Taxonomy . . . 11
2.2.3 Policy and Standards Response . . . 12
2.3 Graph Theory Fundamentals . . . 13
2.3.1 Directed Graphs and Basic Definitions . . . 13
2.3.2 Directed Acyclic Graphs and Cycles . . . 13
2.3.3 Connectivity and Reachability . . . 14
2.3.4 Articulation Points and Bridge Edges . . . 14
2.4 Network Science Concepts . . . 15
2.4.1 Scale-Free Networks and Power-Law Degree Distributions . . . 15
2.4.2 Small-World Networks . . . 16
2.4.3 Centrality Measures . . . 16
2.4.4 Community Detection and Modularity . . . 17
2.4.5 K-Core Decomposition . . . 18

2.4.6 Spectral Analysis and Propagation Modelling 18
2.5 Vulnerability Databases 18
2.5.1 CVE and the National Vulnerability Database 19
2.5.2 Open Source Vulnerabilities (OSV) 19
2.5.3 GitHub Security Advisories 20
2.5.4 CVSS v3.1 Scoring System 20

3 Related Work **22**
3.1 Existing SBOM Analysis Tools 22
3.2 Graph-Based Security Research 23
3.2.1 Dependency Graph Analysis 25
3.2.2 Software Vulnerability Graphs 25
3.2.3 Call Graph and Static Analysis 26
3.2.4 Network Science Applied to Software 26
3.3 Supply Chain Incident Case Studies 27
3.3.1 SolarWinds SUNBURST (December 2020) 27
3.3.2 log4shell (CVE-2021-44228, December 2021) 28
3.4 Gap Analysis 29

4 System Design **31**
4.1 System Architecture 31
4.1.1 Overview and Design Rationale 31
4.1.2 Frontend Layer 32
4.1.3 Backend Layer 34
4.1.4 Communication: REST API 34
4.2 CycloneDX Parsing Pipeline 34
4.2.1 Ingestion and Deduplication 35
4.2.2 Component Extraction 35
4.2.3 Graph Construction from Dependencies 36
4.3 Graph Data Model 36
4.3.1 Node Representation 36
4.3.2 Edge Representation 37
4.3.3 In-Memory Store 37
4.4 SBOM Graph Visualisation 38
4.4.1 Graph Rendering with Cytoscape.js 38
4.4.2 Layout Algorithms 39
4.4.3 Core Interaction Model 39

5 Vulnerability Analysis **42**
5.1 Vulnerability Source 42
5.1.1 Rationale for OSV 43
5.1.2 OSV Integration Pipeline 43
5.2 Deduplication and Storage 44

5.2.1 VulnStore Design . . . 44
5.2.2 Deduplication Logic . . . 45
5.3 CVSS v3.1 Base Score Computation . . . 45
5.3.1 Formula and Metric Groups . . . 46
5.3.2 Worked Example: Log4Shell . . . 46
5.3.3 Custom Client-Side Implementation . . . 47
5.4 Vulnerability Management Workflow . . . 48
5.4.1 Automated Fetch on SBOM Upload . . . 48
5.4.2 Status Lifecycle and REST API . . . 48
5.5 Graph Overlay Visualisation . . . 49
5.5.1 Severity Colour Coding . . . 49
5.5.2 Blast Radius Analysis . . . 51
5.5.3 Dominator Tree Analysis . . . 52

6 Structural Risk and Graph Analysis 54
6.1 Component Scale . . . 55
6.1.1 Articulation Points and Bridges . . . 55
6.1.2 HITS Hub and Authority Scores . . . 56
6.1.3 Spectral Exposure Score . . . 57
6.1.4 Core Number . . . 57
6.1.5 Edge Rankings and Abstraction Candidates . . . 58
6.2 Cluster Scale . . . 58
6.2.1 Louvain Communities . . . 58
6.2.2 Bow-Tie Decomposition . . . 60
6.2.3 Rich-Club Coefficient . . . 60
6.3 Whole-Graph Scale . . . 61
6.3.1 Spectral Radius and Epidemic Threshold . . . 61
6.3.2 Degree Entropy . . . 63
6.3.3 Cycles and Strongly Connected Components . . . 63
6.3.4 Version Conflicts . . . 63
6.3.5 Namespace Concentration . . . 64
6.3.6 Unmaintained Packages . . . 64
6.3.7 Degree Distribution and Power-Law Fit . . . 64
6.3.8 Small-World Coefficient . . . 65
6.4 Summary . . . 66

7 Evaluation 67
7.1 Dataset and Collection Pipeline . . . 67
7.2 Method and Research Questions . . . 69
7.3 RQ1: Dependency Graph Profile . . . 70
7.4 RQ2: Structural Ranking versus Severity . . . 72
7.5 RQ3: Hidden Exposure . . . 73

7.6 Performance 75
7.7 Threats to Validity 76
7.8 Reproducibility 77

8 Conclusion **78**
8.1 Summary of Findings 78
8.2 Contributions 79
8.3 Limitations 80
8.4 Future Work 82

Bibliography 84

This page intentionally left blank.

List of Figures

4.1 SBOM Lens system architecture 32
4.2 The SBOM Lens interface 33
4.3 The tree and force-directed layouts 40
4.4 The Details panel and the severity overlay 40

5.1 The Vulnerabilities page 50
5.2 Blast-radius overlay 52
5.3 Dominated set 53

6.1 Louvain communities 59
6.2 The Risk Assessment page 62

7.1 Distributions of graph size, power-law exponent and small-world coefficient 72
7.2 Agreement between severity and structural rankings 72
7.3 Hidden exposure per SBOM 74
7.4 Analysis time per SBOM 75

This page intentionally left blank.

List of Tables

3.1 Comparison of SBOM analysis tools 24
3.2 log4shell in an illustrative SBOM 28

6.1 Structural measures computed by SBOM Lens 66

7.1 Selection funnel of the evaluation dataset 68
7.2 Profile of the 90 production dependency graphs (RQ1). 70
7.3 Agreement between the severity-band ranking and structural rankings of vulnerable components (RQ2). 72
7.4 Hidden exposure (RQ3). 73
7.5 Analysis time per SBOM in seconds, by endpoint. 75

This page intentionally left blank.

1 Introduction

1.1 Motivation

Modern software is rarely built from scratch. The overwhelming majority of production systems depend on third-party open-source components: the Synopsys 2024 Open Source Security and Risk Analysis report found that 96% of commercial codebases contain open-source code and that the average codebase incorporates over 500 open-source components (Synopsys, 2024). Each of these components introduces its own transitive closure of further dependencies, forming a supply chain whose attack surface is orders of magnitude larger than the first-party code that developers directly author. The software supply chain — the aggregate of all such components, their origins, build pipelines, and trust relationships — has consequently become one of the most consequential and least understood risk surfaces in modern cybersecurity.

The fragility of this supply chain was exposed in dramatic fashion by a succession of high-profile attacks. In December 2020, the SUNBURST campaign targeted SolarWinds, a network management vendor trusted by thousands of organisations including agencies of the United States federal government. Attackers compromised the vendor's build pipeline and inserted malicious code into a signed software update that was subsequently distributed to its customers (Cybersecurity and Infrastructure Security Agency, 2020). The incident demonstrated how a single trusted upstream vendor can become an unwitting vector for cascading compromise across an entire downstream ecosystem. A year later, in December 2021, the disclosure of CVE-2021-44228 in Apache Log4j — universally known as *log4shell* — revealed a critical remote code execution vulnerability in a logging library present, often as a transitive dependency, across the Java ecosystem (National Institute of Standards and Technology, 2021; Wetter et al., 2021). The difficulty organisations faced in determining whether they were exposed illustrates precisely the challenge this thesis addresses: without a structural model of the dependency graph, the presence of a transitive dependency remains invisible until it is exploited. In March 2024, the xz-utils backdoor (CVE-2024-3094, CVSS

10.0) added a further dimension of supply chain risk to the record: a patient, multi-year social engineering campaign against an open-source maintainer resulted in the insertion of a cryptographic backdoor into a core Linux compression utility that, had it reached widespread deployment, would have compromised SSH authentication on affected systems (National Institute of Standards and Technology, 2024). Together, these three incidents establish that supply chain attacks are diverse in method, severe in potential impact, and increasing in sophistication.

Regulators around the world have responded by mandating greater transparency over software composition. In the European Union, the Cyber Resilience Act (CRA), entered into force in December 2024, enforces manufacturers and developers of products with digital elements sold in the EU market to document the components of their software and to address known vulnerabilities throughout the product lifecycle (European Parliament and Council of the European Union, 2024). The CRA explicitly names the SBOM — a machine-readable inventory of all software components and their dependency relationships — as a key instrument of this transparency obligation. In the United States, Executive Order 14028, signed in May 2021, similarly mandated that software supplied to the federal government be accompanied by an SBOM, defining it as "a formal record containing the details and supply chain relationships of various components used in building software" (The White House, 2021). Executive Order 14144, issued in January 2025, reinforced and extended these requirements (The White House, 2025). The SBOM has thus transitioned from a niche software engineering practice to a regulatory expectation on both sides of the Atlantic, and the volume of SBOMs in circulation is set to grow substantially in coming years.

Yet the emergence of the SBOM as a transparency artefact has not been matched by commensurate progress in analytical tooling. Among the open-source tools in the SBOM ecosystem, Dependency-Track, grype and syft treat SBOMs essentially as component inventories, and Sunshine, which visualises the dependency tree, computes nothing over it beyond reachability (OWASP Foundation, n.d.[b]; Anchore, n.d.[a]; Anchore, n.d.[b]; CycloneDX, n.d.). They answer two questions effectively: *what components are present?* and *do any of them have known CVEs?* What they do not do is model the structure of the dependency graph. When a vulnerability is disclosed in a given component, these tools report the affected component largely in isolation. Only Sunshine lists the answer to the question a security analyst immediately needs — *which other components in my system depend on this one, directly or transitively, and are therefore also at risk?* — and it gives that answer as a table, not on the dependency graph. Nor do they identify which components, if compromised, would expose the largest number of upstream dependents — all the components that transitively rely on them and would therefore inherit the risk. The result is that analysts must manually triage long flat lists of CVEs with no structural context to guide prioritisation.

The opportunity for a more rigorous approach lies in the observation that a software dependency graph is, by construction, a directed graph — and graph theory provides precisely the tools needed to answer both questions above. Given a directed depen-

dency graph, the set of all components affected by a vulnerability in component v is the set of ancestors of v in the graph, computable in linear time. Graph-theoretic centrality measures further allow the identification of components that, if compromised, would affect a disproportionately large portion of the dependency tree — components that a security analyst should treat as higher-priority regardless of their individual CVSS score. Articulation points identify components whose removal would disconnect the graph entirely, representing single points of structural failure. These techniques are well-established in graph theory (Cormen et al., 2022; Newman, 2018) but have not been systematically applied to SBOM dependency graphs.

This dissertation presents SBOM Lens, a graph-based framework for software supply chain risk assessment. The system ingests Software Bills of Materials in CycloneDX format and constructs a directed dependency graph, over which graph-theoretic techniques are applied to surface structural vulnerabilities not visible through flat-inventory analysis. The design, implementation, and evaluation of SBOM Lens are described in Chapter 4, Chapter 5, and Chapter 6, respectively, with empirical results presented in Chapter 7.

1.2 Problem Statement

Existing SBOM analysis tooling suffers from a structural limitation that is architectural rather than incidental: it treats the dependency graph as a flat list. When a security analyst queries a tool such as Dependency-Track or grype for the vulnerabilities present in a given SBOM, the result is a table of (component, CVE, severity) tuples ranked by CVSS score, at most combined with exploit likelihood (EPSS) or known exploitation (KEV) — measures of the vulnerability, not of where the component sits. There is no representation of the relationships between components, no indication of how many other components depend on a given vulnerable package, and no means of determining whether a vulnerable component sits at the centre of the dependency tree or at its periphery. This limitation has two concrete consequences for the analyst.

The first consequence is *invisible blast radius*. When a new vulnerability is disclosed in a widely-used library — as occurred with log4shell — the analyst's immediate question is not "does this component have a CVE?" but rather "how much of my system is exposed?" Answering that question requires traversing the dependency graph upstream from the vulnerable component to find all packages that depend on it, directly or transitively. Of the tools surveyed in Chapter 3, only Sunshine performs this traversal, and it reports the result as a list rather than on the graph; with the others, the analyst must do it manually, if they can do it at all.

The second consequence is *undifferentiated prioritisation*. A CVSS score measures the intrinsic severity of a vulnerability in isolation — attack vector, complexity, impact — but says nothing about the structural importance of the affected component within the system. A critical-severity CVE in a component that is a leaf node with no dependents poses a fundamentally different remediation burden than the same CVE in

a component that is a shared transitive dependency of dozens of packages throughout the graph. Without graph-structural context, both appear equally urgent on a flat list, and the analyst has no principled basis for deciding where to act first.

The literature points to this gap. A 2025 landscape study of 84 open-source SBOM tools (Garcia et al., 2025) classifies them into generation, consumption, interoperability, quality assurance and services, and has no category for analysing the dependency structure an SBOM records: the tools that consume SBOMs for vulnerability management query databases such as NVD and report the vulnerable components they find. The survey of Chapter 3 reaches the same conclusion for the most widely used tools. The closest existing work is GUAC (Graph for Understanding Artifact Composition), developed under the OpenSSF umbrella, which constructs provenance graphs across multiple projects and organisations (Open Source Security Foundation, 2024). GUAC addresses supply chain transparency at a cross-project, ecosystem-wide scale, and can search the dependents of a vulnerable package, but it does not compute structural measures to assess the risk profile of a single software system. A detailed comparison of SBOM Lens with GUAC and other related tools is presented in Chapter 3.

SBOM Lens addresses this gap by constructing a directed dependency graph from a CycloneDX SBOM and providing the analyst with two complementary views. The first is *impact traversal*: given any vulnerable component, the system immediately identifies all components that depend on it, at any depth, giving the analyst an accurate and complete picture of the blast radius. The second is *structural risk ranking*: using graph-theoretic measures, the system identifies which components are structurally most likely to propagate risk widely if compromised, allowing the analyst to prioritise monitoring and remediation based not only on CVE severity but on the component's position within the dependency graph. The research question is not whether these capabilities can be built — the system exists — but what they reveal when applied to real-world open-source SBOMs, a question answered empirically in Chapter 7.

1.3 Research Objectives

This work is framed as a design-and-analysis thesis: the system under study, SBOM Lens, has been built, and the objectives concern both its construction and what it reveals when applied empirically. Each objective maps directly to one of the implementation or evaluation chapters, allowing the conclusions drawn in Chapter 8 to be verified against each stated goal.

1. **Graph construction pipeline.** To design and implement a directed dependency graph construction pipeline that ingests CycloneDX JSON SBOMs and produces an annotated graph — with components as nodes and dependency relationships as directed edges — suitable for interactive visualisation and further graph-theoretic analysis. (Chapter 4)
2. **Vulnerability integration.** To integrate vulnerability data from the Open Source

Vulnerabilities database (OSV), which aggregates GitHub Security Advisories and other open-source advisory databases, with full CVSS v3.1 base score computation, and to overlay the results on the dependency graph so that the analyst can immediately see which nodes carry known vulnerabilities and at what severity. (Chapter 5)

3. **Graph-based structural risk analysis.** To apply a suite of graph-theoretic algorithms — including impact traversal to enumerate all components affected by a vulnerability in a given node, centrality and articulation point analysis to identify components with the highest structural propagation potential, and community detection to surface the modular structure of the dependency ecosystem — so as to give the analyst a principled basis for prioritising remediation beyond raw CVSS scores. (Chapter 6)
4. **Empirical evaluation.** To evaluate the framework against a set of real-world open-source project SBOMs and demonstrate, through graph-level and node-level metrics, that the approach surfaces structural risk information that is not available through flat-inventory analysis. (Chapter 7)

1.4 Contributions

This work makes four contributions, each corresponding to a distinct system capability or analytical result.

1. **SBOM Graph Visualisation.** We design and implement an interactive directed dependency graph constructed from CycloneDX JSON SBOMs and rendered using Cytoscape.js. The visualisation supports topology exploration, node inspection, and interactive editing of the dependency structure, providing the foundational data model upon which all subsequent analysis is performed. (Chapter 4)
2. **Vulnerability Analysis.** We implement a vulnerability integration pipeline that queries OSV by Package URL, deduplicates findings per component, and computes full CVSS v3.1 base scores. Vulnerable components are overlaid on the dependency graph and visually distinguished by severity band, giving the analyst an immediate picture of which nodes in the graph carry known vulnerabilities. (Chapter 5)
3. **Graph-Based Structural Risk Analysis.** We apply a suite of graph-theoretic algorithms to the dependency graph to support analyst decision-making beyond raw CVE listings. Impact traversal identifies all components that depend — directly or transitively — on a given vulnerable node, making the blast radius of any vulnerability immediately visible. Centrality analysis and articulation point detection identify the components whose structural position makes them highest-priority targets for monitoring, regardless of whether they currently carry a known CVE. Community detection reveals the modular structure of the dependency ecosystem, supporting higher-level reasoning about isolation and blast containment.

(Chapter 6)

4. **Empirical Evaluation.** We evaluate the framework against a dataset of real-world open-source project SBOMs, collected at scale from public repositories. Through graph-level and node-level metrics we demonstrate that the approach surfaces structural risk information that is not available through flat-inventory analysis, and we discuss the implications for security analyst workflows. (Chapter 7)

1.5 Scope and Limitations

This section defines the explicit boundaries of the work to prevent misreading of the claims made in subsequent chapters.

SBOM format. SBOM Lens supports CycloneDX JSON as its sole input format. SBOMs in SPDX or SWID formats are not supported. This is a deliberate design boundary rather than an oversight: CycloneDX JSON provides the richest representation of component dependency relationships within the OWASP ecosystem, and its structured `dependencies` array maps directly onto a directed graph without the need for format translation. Forward-compatibility with other formats is left as future work.

Evaluation methodology. The evaluation presented in Chapter 7 is metric-based: it reports graph-level and node-level measures computed on real-world open-source project SBOMs. The thesis does not include a formal usability study, a user experiment, or any assessment of whether security analysts find the tool's output actionable in practice. Such a study would require ethical approval and a participant cohort that falls outside the scope of this work.

Features out of scope. Three capabilities that naturally extend the SBOM Lens concept are explicitly excluded from this thesis. SBOM comparison and delta analysis — the comparison of two SBOMs for the same project across time to detect dependency changes — is a distinct analytical problem that would require a temporal data model not present in the current system. Automated restructuring of the dependency graph to reduce structural risk requires a constraint satisfaction or optimisation layer beyond the scope of this work; SBOM Lens's Topology Optimiser offers only heuristic suggestions derived from the spectral exposure score, which are described briefly in Section 6.1.5 and are not evaluated. Finally, SBOM Lens operates on source-level SBOMs generated from package manifests; SBOMs derived from compiled binaries or container images raise format and toolchain questions that are not addressed here.

Novelty claims. This thesis does not claim that the graph-theoretic algorithms applied are themselves novel — articulation point detection, centrality analysis, and community detection are well-established techniques. The contribution lies in their systematic application to SBOM dependency graphs as a coherent risk assessment framework, and in the empirical demonstration of what they reveal about real-world software supply chains.

1.6 Thesis Structure

The remainder of this dissertation is organised as follows.

Chapter 2 establishes the theoretical and technical foundations required to understand SBOM Lens. It covers the concept of a Software Bill of Materials and the CycloneDX format, the software supply chain security landscape, the fundamentals of graph theory and network science, and the vulnerability databases and scoring systems — NVD, OSV, and CVSS v3.1 — that underpin the vulnerability analysis.

Chapter 3 surveys the existing landscape of SBOM analysis tools — including Dependency-Track, grype, syft and Sunshine — alongside graph-based security research and supply chain incident case studies. It identifies the gap that SBOM Lens addresses: the absence of graph-structural risk analysis in current tooling.

Chapter 4 describes the architecture of SBOM Lens. It presents the React and FastAPI system design, the CycloneDX JSON parsing pipeline, the construction of the directed dependency graph, and the interactive SBOM Graph visualisation built with Cytoscape.js.

Chapter 5 presents the vulnerability integration pipeline built on OSV and explains why OSV was chosen over querying NVD and GitHub Security Advisories directly. It details the CVSS v3.1 base score computation model, the vulnerability management workflow, and the graph overlay that visually surfaces vulnerable nodes by severity band.

Chapter 6 presents the structural risk measures at three scales: per component (articulation points and bridges, HITS scores, the spectral exposure score, and core numbers), per cluster (Louvain communities, the bow-tie decomposition, and the rich-club coefficient), and for the whole graph (the spectral radius and epidemic threshold, degree entropy, cycles, version conflicts, namespace concentration, the degree distribution, and the small-world coefficient). For each technique, the chapter explains the theoretical basis and its interpretation in the context of software supply chain risk.

Chapter 7 evaluates the framework against a set of real-world open-source project SBOMs. It reports graph-level and node-level metrics, vulnerability findings, and structural analysis results, and discusses what the analysis reveals about structural risk in software supply chains.

Chapter 8 summarises the contributions of this work, maps them back to the research objectives stated in this chapter, acknowledges the limitations of the current implementation, and proposes directions for future research.

2
Background

Modern software systems depend on hundreds or thousands of third-party components whose internals their developers never inspect. Understanding, securing, and analysing these dependency networks requires a common vocabulary spanning three fields that do not normally overlap in software engineering curricula: supply chain security, graph theory, and vulnerability scoring. This chapter establishes that vocabulary. Every concept defined here reappears by name in Chapter 4, Chapter 5, and Chapter 6 without re-definition. The chapter is organised in dependency order: Section 2.1 defines the data format that SBOM Lens ingests; Section 2.2 frames the threat landscape that motivates the analysis; Section 2.3 and Section 2.4 establish the structural analysis concepts applied to that data; and Section 2.5 defines the scoring and database standards used to annotate the graph.

2.1 Software Bill of Materials and CycloneDX

A Software Bill of Materials is the data foundation of SBOM Lens. This section defines what an SBOM is, surveys the formats in which SBOMs are encoded, and specifies exactly which fields of the CycloneDX format SBOM Lens consumes.

2.1.1 What is a Software Bill of Materials?

A *Software Bill of Materials* (SBOM) is a machine-readable inventory of all software components, their versions, licences, and dependency relationships present in a given software artefact. The analogy to the ingredient label on food packaging is precise: just as a food label provides a standardised, legally mandated declaration of what a product contains, an SBOM provides a standardised, machine-processable declaration of what a software product is made of (National Telecommunications and Information Administration, 2021).

The US National Telecommunications and Information Administration (NTIA) defines seven minimum data fields that every conformant SBOM must contain: Sup-

plier Name, Component Name, Version of the Component, Other Unique Identifiers (such as a Package URL, CPE, or SWID), Dependency Relationship, Author of SBOM Data, and Timestamp (National Telecommunications and Information Administration, 2021). These fields define the baseline for interoperability across tooling and databases. Of the seven, the Dependency Relationship field is the most consequential for SBOM Lens: it is the only field that encodes the graph structure of the software system rather than the properties of individual components.

The regulatory impetus for SBOM adoption has grown substantially since 2021. US Executive Order 14028 (May 2021) mandated SBOMs for software sold to the US federal government; EO 14144 (January 2025) extended and reinforced these requirements (The White House, 2021). The European Cyber Resilience Act (CRA), which entered into force in December 2024, similarly requires software component transparency for products with digital elements sold in the EU market (European Parliament and Council of the European Union, 2024). SBOMs have therefore moved from a voluntary best practice to a regulatory requirement in the two largest software markets in the world.

For SBOM Lens, the SBOM is the sole input artefact. Without the explicit dependency relationship field encoded in a machine-readable format, the graph construction pipeline described in Chapter 4 would not be possible. The choice of CycloneDX as the supported format — examined in Section 2.1.3 — is motivated directly by the richness and explicitness of its dependency graph representation.

2.1.2 SBOM Format Landscape

Three machine-readable SBOM formats exist in practice: CycloneDX (OWASP, originally 2017), SPDX (Linux Foundation, originally 2010, now ISO/IEC 5962), and SWID Tags (ISO/IEC 19770-2). Each was designed with a different primary use case, and that design intent has significant practical consequences for graph-based analysis.

SPDX was created primarily for licence compliance. Its relationship model is broad — it defines approximately twenty-five relationship types including `DESCRIBES`, `CONTAINS`, `DEPENDS_ON`, `GENERATED_FROM`, and `OPTIONAL_DEPENDENCY_OF` — but the dependency relationship is represented implicitly as one type among many, and SPDX 2.3 has no native structure for embedding vulnerability data. SPDX is widely adopted for licence auditing workflows but does not natively support the graph structure required for the analysis SBOM Lens performs.

CycloneDX was designed from the outset for security use cases: vulnerability tracking, dependency analysis, and supply chain integrity verification. Its key structural advantage over SPDX is the explicit separation of components and their dependency relationships into two distinct top-level arrays. A CycloneDX document's `dependencies[]` array makes the dependency graph unambiguous: each entry declares a component (`ref`) and the set of components it directly depends on (`dependsOn`), with no inference required. CycloneDX 1.6 added native support for VEX (Vulnerability Exploitability eXchange) documents via a `vulnerabilities[]` array. The current version, Cy-

cloneDX 1.7, was published in 2025 and is standardised as ECMA-424 (OWASP Foundation, n.d.[a]).

SWID Tags are excluded from consideration for SBOM Lens because they lack a standardised dependency relationship field; they identify installed software but do not encode the graph structure between components.

SBOM Lens supports CycloneDX JSON exclusively. This is an explicit design boundary, not an oversight: CycloneDX provides the most explicit and semantically unambiguous graph representation available in any SBOM format, eliminating the inference step that SPDX dependency reconstruction would require.

2.1.3 CycloneDX JSON Structure and SBOM Lens Fields

A CycloneDX JSON document has a rich schema, but SBOM Lens consumes three top-level constructs. The `metadata.component` object describes the root of the SBOM — the project being analysed, with its name, version, and type. The `components[]` array contains one entry per dependency component, each carrying at minimum a `bom-ref` (a stable local document identifier), a `name`, a `version`, a `purl` (Package URL), and a `type`. The `dependencies[]` array encodes the graph structure: each entry takes the form `{"ref": "bom-ref-A", "dependsOn": ["bom-ref-B", "bom-ref-C"]}`, declaring that the component identified by `bom-ref-A` directly depends on both `bom-ref-B` and `bom-ref-C` (OWASP Foundation, n.d.[a]).

The `bom-ref` field is SBOM Lens's graph node identifier. It is a stable, local reference that uniquely identifies a component within a single CycloneDX document. When `bom-ref` is absent — which occurs in some generator tools — SBOM Lens falls back to the `purl` field, and further to the `name` field as a last resort. This three-step fallback chain is implemented in the graph construction pipeline described in Chapter 4.

The directed dependency graph is constructed directly from these fields. The vertex set V comprises all entries in `components[]` plus the root component from `metadata.component`. The edge set E is derived from `dependencies[]`: for each entry with `ref` = u and `dependsOn` = $[v_1, v_2, \ldots]$, SBOM Lens adds directed edges $u \rightarrow v_1, u \rightarrow v_2, \ldots$. A directed edge from component u to component v means "u depends on v."

The `purl` field serves a second purpose beyond node identification. A Package URL encodes the package ecosystem, namespace, name, and version in a single string — for example, `pkg:npm/lodash@4.17.21` or `pkg:pypi/requests@2.31.0` — enabling cross-database vulnerability matching. OSV uses purl natively; NVD uses CPE but can be cross-referenced via purl. The purl-to-database matching pipeline is described in Chapter 5. The purl specification is standardised as ECMA-427 (Package-URL Project, n.d.).

Fields not consumed by SBOM Lens in the current implementation include the `vulnerabilities[]` array (VEX documents), `services[]`, `compositions[]`, and `externalReferences[]`. Vulnerability data is obtained from OSV rather than from embedded VEX data; this design decision and its trade-offs are discussed in Chapter 8.

2.2 Software Supply Chain Security

Before describing the algorithms SBOM Lens applies, it is necessary to establish what the analysis is defending against. This section defines the software supply chain as a concept, characterises the threat categories it faces, and summarises the regulatory responses that have raised SBOM analysis from an optional practice to a compliance requirement.

2.2.1 The Software Supply Chain

The *software supply chain* encompasses all the components, libraries, tools, and processes — and the people and organisations behind them — involved in developing, building, packaging, and distributing a software product. The supply chain metaphor is precise: each upstream dependency is a supplier whose code is inherited by all downstream consumers. Vulnerabilities, backdoors, and deprecated components in any supplier propagate transitively to every consumer in the dependency graph.

The scale of this dependency is larger than most practitioners appreciate. The Synopsys Open Source Security and Risk Analysis (OSSRA) 2024 report found that 96% of commercial codebases contain open-source components (Synopsys, 2024). More consequentially, direct dependencies — those explicitly declared in a project's manifest — represent only a fraction of the full dependency tree. Transitive dependencies, the dependencies of dependencies recursively, routinely outnumber direct dependencies by an order of magnitude in large projects.

The log4shell vulnerability (CVE-2021-44228, December 2021) illustrates the practical consequence. Apache Log4j was a transitive dependency in millions of Java applications; many of the affected development teams were entirely unaware that their software included it. A flat SBOM inventory can reveal whether Log4j is present; it cannot reveal how deeply embedded it is, how many other components depend on it, or how a vulnerability in it would propagate through the dependency graph. This structural blindspot is the gap that SBOM Lens addresses through the graph analysis methods described in Chapter 6.

2.2.2 Attack Taxonomy

Software supply chain attacks exploit the transitive trust that consumers place in upstream suppliers. Ladisa et al. (2023) provide a systematic taxonomy of open-source supply chain attacks in a peer-reviewed Systematisation of Knowledge (SoK) paper. Four primary categories are relevant to the SBOM analysis context.

Build pipeline compromise describes attacks in which an adversary infiltrates a vendor's CI/CD system or release process and injects malicious code into an otherwise legitimate software artefact. The SolarWinds SUNBURST attack (December 2020, CISA Alert AA20-352A) is the canonical example: malicious code was inserted into a signed software update and distributed to approximately 18,000 organisations, including US

government agencies, via a trusted vendor relationship. The resulting component appears legitimate in any SBOM because it was produced by the authentic vendor.

Compromised maintainer attacks involve an adversary gaining legitimate commit access to an open-source project — typically through social engineering over months or years — and then inserting a backdoor. The xz-utils backdoor (CVE-2024-3094, CVSS 10.0, March 2024) is a recent and technically sophisticated example: a two-year campaign resulted in a backdoor narrowly avoiding widespread compromise of OpenSSH on systemd-based Linux distributions.

Dependency confusion exploits the resolution logic of package managers: an attacker publishes a malicious package on a public registry with the same name as a private internal package, causing the package manager to resolve the public (malicious) version instead of the private (legitimate) one.

Typosquatting relies on registering package names visually similar to widely-used packages — for example, `requests-oauthlib2` alongside `requests-oauthlib` — to capture installations triggered by typographical errors.

Graph-based SBOM analysis complements detection for the latter two categories. K-core decomposition (Chapter 6) reveals how deeply embedded a potentially substituted package is in the dependency network, providing a structural triage signal for incident response.

2.2.3 Policy and Standards Response

The regulatory and standards response to supply chain attacks has accelerated since 2021. US Executive Order 14028 (May 2021) mandated SBOMs for software sold to the US federal government, with Section 10(j) providing the authoritative regulatory definition of SBOM (The White House, 2021). EO 14144 (January 2025) reinforced and extended these requirements. The European Cyber Resilience Act, which entered into force in December 2024, requires manufacturers and developers of products with digital elements sold in the EU to document software components and address known vulnerabilities throughout the product lifecycle (European Parliament and Council of the European Union, 2024).

At the standards level, the NIST Secure Software Development Framework (SP 800-218) describes secure development practices at the process level (Souppaya et al., 2022). Supply-chain Levels for Software Artifacts (SLSA) provides a complementary four-level maturity framework for source integrity, build integrity, and provenance attestation, operationalising trust verification in CI/CD pipelines (Open Source Security Foundation, n.d.[b]). CISA has further published guidance on SBOM minimum elements, sharing mechanisms, and tooling recommendations. These frameworks collectively define the regulatory and operational context in which SBOM Lens operates.

2.3 Graph Theory Fundamentals

The dependency structure encoded in a CycloneDX SBOM is a directed graph. This section establishes the graph-theoretic vocabulary used throughout the thesis. Definitions are given mathematically, with an SBOM interpretation for each concept, and a forward-reference to the chapter where each is applied in SBOM Lens. The primary reference is Cormen et al. (2022).

2.3.1 Directed Graphs and Basic Definitions

A *directed graph* (digraph) $G = (V, E)$ consists of a vertex set V and a set of directed edges $E \subseteq V \times V$ (Cormen et al., 2022). For SBOM Lens, V is the set of software components present in the SBOM and E represents dependency relationships: a directed edge $(u, v) \in E$ means that component u directly depends on component v.

The *in-degree* of a vertex v, written $\deg^-(v)$, is the number of directed edges incoming to v — equivalently, the number of components that directly depend on v. The *out-degree* $\deg^+(v)$ is the number of directed edges outgoing from v — the number of components on which v directly depends. In an SBOM dependency graph, a component with high in-degree is one that many others depend upon; a component with high out-degree has many direct dependencies of its own. Both are indicators of structural importance examined in Chapter 6.

The *adjacency matrix* $\mathbf{A}$ of G is an $|V| \times |V|$ matrix where entry $A_{ij} = 1$ if the directed edge $i \to j$ exists and $A_{ij} = 0$ otherwise. Real SBOM graphs are typically sparse — a graph with thousands of components may have only tens of thousands of edges — so $\mathbf{A}$ is stored in compressed sparse format in practice. The adjacency matrix is the basis for eigenvector centrality analysis described in Section 2.4.3.

A *root node* is a vertex with in-degree zero: no other component in the SBOM depends on it. In SBOM Lens, the root node is the project being analysed, identified by `metadata.component` in the CycloneDX document. The *depth* of a component is the length of the shortest directed path from the root to that component; depth-1 components are direct dependencies, depth-k components are k-hop transitive dependencies.

2.3.2 Directed Acyclic Graphs and Cycles

A *directed acyclic graph* (DAG) is a directed graph that contains no directed cycle (Cormen et al., 2022). Dependency graphs are ideally DAGs: if package A depends on package B which depends on A, neither can be resolved without the other, and standard package managers report this as an error. A DAG admits a *topological ordering* — a linear arrangement of all vertices such that every edge goes from earlier to later — which corresponds to a valid build or installation order.

In practice, SBOMs generated by real tools may contain directed cycles, arising from circular version-pinned dependencies or artefacts of SBOM generation tooling. SBOM Lens models the dependency graph as a general digraph (not an enforced DAG) using

NetworkX's `DiGraph` class. Cycles are identified by computing *strongly connected components* (SCCs): a set of vertices in which every vertex is reachable from every other via directed paths. An SCC of size greater than one indicates a directed cycle. SCCs can be computed in $O(|V| + |E|)$ time with Tarjan's algorithm (Cormen et al., 2022). SBOM Lens reports the directed cycles of the dependency graph and the number of non-trivial SCCs as structural anomalies; this analysis is described in Chapter 6.

A *diamond dependency* occurs when two distinct dependency paths from a common ancestor converge on the same descendant: component A depends on both B and C, and both B and C depend on D. If B requires D@1.0 and C requires D@2.0, the package manager must either fail or install both versions, producing a *version conflict*: the same package present in the graph at two or more versions. SBOM Lens detects version conflicts, the observable consequence of this pattern, as part of the structural risk assessment in Chapter 6.

2.3.3 Connectivity and Reachability

A directed graph is *weakly connected* if the underlying undirected graph — obtained by ignoring edge directions — is connected. It is *strongly connected* if a directed path exists between every ordered pair of vertices. Most SBOM dependency graphs are weakly connected (the components form a single ecosystem) but not strongly connected, because dependency direction is asymmetric: a library does not depend on the applications that use it (Cormen et al., 2022).

The *ancestor set* of a vertex v, written $\mathrm{anc}(v)$, comprises all vertices from which v is reachable via directed paths — all components that transitively depend on v. In vulnerability analysis, the *blast radius* of a vulnerability in v is v together with $\mathrm{anc}(v)$: the vulnerable component and every component that would be affected if it were compromised. Conversely, the *descendant set* $\mathrm{desc}(v)$ comprises all vertices reachable from v — all transitive dependencies of v. SBOM Lens computes these sets using NetworkX's `nx.ancestors` and `nx.descendants` functions. Both are applied in Chapter 5 as a vulnerability impact quantification method.

Reachability is computed via breadth-first search (BFS) and depth-first search (DFS), both running in $O(|V|+|E|)$ (Cormen et al., 2022). BFS from the root visits components in non-decreasing depth order, establishing the dependency layering of the graph. DFS is used for cycle detection and for the articulation point algorithm described in the following subsection.

2.3.4 Articulation Points and Bridge Edges

An *articulation point* (also called a cut vertex) in a connected undirected graph is a vertex whose removal — along with its incident edges — increases the number of connected components (Cormen et al., 2022). In the context of a software dependency graph, an articulation point is a component whose removal would fragment the dependency network into disconnected parts. Such a component is a *single point of failure*:

a vulnerability, deprecation, or licence change affecting it touches both sides of the split.

Detection is performed by a DFS-based algorithm running in $O(|V|+|E|)$ time (Cormen et al., 2022). For each vertex v, the algorithm tracks the discovery time $\mathrm{disc}[v]$ (the DFS visit order) and the lowest discovery time reachable from the subtree rooted at v via back edges, $\mathrm{low}[v]$. A vertex v is an articulation point if any DFS-tree child w of v satisfies $\mathrm{low}[w] \geq \mathrm{disc}[v]$, meaning no back edge in w's subtree can bypass v to reach v's ancestors. Because SBOM dependency graphs are directed, SBOM Lens applies the algorithm to the undirected projection of the graph: `nx.articulation_points(G.to_undirected())`. The set of articulation points is a primary output of the structural risk assessment in Chapter 6.

A *bridge* (or cut edge) is an edge whose removal disconnects the graph — the edge-level analogue of an articulation point. SBOM Lens identifies bridge edges as structural "firebreaks": dependency relationships whose severance partitions the ecosystem. Bridge edges are reported alongside articulation points in Chapter 6.

2.4 Network Science Concepts

Graph theory provides the foundational vocabulary for describing individual graph structures. Network science extends this vocabulary to characterise the global statistical properties of large graphs — properties that emerge from the aggregate behaviour of thousands of nodes and edges and that cannot be read from any single vertex or edge in isolation. This section introduces the network science methods applied in Chapter 6, with mathematical definitions, supply chain interpretations, and forward references. Primary references are Barabási (2016) and Newman (2018).

2.4.1 Scale-Free Networks and Power-Law Degree Distributions

Many real-world networks — the World Wide Web, citation networks, biological protein-interaction networks — exhibit degree distributions that follow a *power law*: $P(k) \sim k^{-\gamma}$ where $\gamma \in [2, 3]$ for most empirical cases (Barabási et al., 1999). Unlike the Poisson distribution characteristic of random Erdős–Rényi graphs, a power-law distribution has no characteristic scale: a small number of nodes ("hubs") have very high degree while the vast majority have low degree. Networks with power-law degree distributions are called *scale-free networks*.

Barabási and Albert (1999) demonstrated that scale-free networks emerge naturally from two mechanisms: network growth (nodes are added over time) and preferential attachment (new nodes connect to existing nodes with probability proportional to their current degree) (Barabási et al., 1999). This "rich get richer" dynamic explains hub emergence. Open-source package ecosystems exhibit both mechanisms: new packages are added continuously, and new packages tend to depend on already-popular libraries.

If the degree distribution of a software dependency graph follows a power law, a small number of packages serve as hubs on which a very large number of others depend. A vulnerability in a hub component has a blast radius far exceeding that of a vulnerability in a leaf node. The degree distributions of the SBOMs evaluated in Chapter 7 are reported empirically; hub components are identified and reported by SBOM Lens as part of the structural risk assessment in Chapter 6.

2.4.2 Small-World Networks

Watts and Strogatz (1998) characterised a class of networks that simultaneously exhibit high *clustering coefficient* C — indicating tightly knit local neighbourhoods — and short *average shortest path length* L comparable to a random graph of the same size (Watts et al., 1998). The small-world coefficient $\sigma = (C/C_{\text{rand}})/(L/L_{\text{rand}})$ quantifies this property: $\sigma \gg 1$ indicates small-world structure.

In software dependency graphs, high clustering can emerge from ecosystem sub-communities: all React-ecosystem packages may share mutual dependencies on a common set of utilities, for example, creating dense local neighbourhoods. Short average path lengths mean that any two components are linked by a short chain of dependency relations, even in a large SBOM, although a vulnerability propagates along such a chain in one direction only, from a dependency to its consumers.

The clustering coefficient and average path length of analysed SBOMs are reported as structural metrics in Chapter 7, providing empirical evidence for or against small-world structure in real dependency graphs.

2.4.3 Centrality Measures

Centrality measures quantify the structural importance of individual nodes in a graph, each from a different perspective on what "importance" means.

Degree centrality is the simplest: a node's degree (or normalised degree) counts its direct connections. In the directed SBOM graph, normalised in-degree centrality identifies components that many others depend on directly; normalised out-degree centrality identifies components with many direct dependencies. Both are computable in $O(|V| + |E|)$ (Barabási, 2016).

Betweenness centrality measures the fraction of shortest paths between all pairs of vertices that pass through a given vertex v:

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

where σ_{st} is the total number of shortest paths from s to t and $\sigma_{st}(v)$ is the number of those paths passing through v. High betweenness identifies relay nodes — bottlenecks through which dependency resolution paths are routed. The Brandes algorithm computes betweenness centrality in $O(|V| \cdot |E|)$ (Barabási, 2016).

The *HITS algorithm* (Hyperlink-Induced Topic Search, Kleinberg 1999) defines two mutually reinforcing per-node scores (Kleinberg, 1999). The *hub score* of a node is proportional to the sum of the authority scores of the nodes it points to; the *authority score* is proportional to the sum of the hub scores of nodes pointing to it. In an SBOM dependency graph, high-authority components are fundamental libraries (widely depended upon by hub components); high-hub components are aggregators that depend on many authoritative libraries. HITS scores are computed iteratively until convergence and applied in Chapter 6.

Eigenvector centrality assigns each node a score proportional to the sum of its neighbours' scores, solved as the leading eigenvector of the adjacency matrix $\mathbf{A}$. It accounts for the quality as well as the quantity of a node's connections: being linked to important components counts more than being linked to peripheral ones. SBOM Lens computes it on the undirected projection of the dependency graph (Section 6.1.3), where depending on a component and being depended on by it count alike.

2.4.4 Community Detection and Modularity

A *community* (or module) is a subgraph that is more densely connected internally than it is to the rest of the graph (Barabási, 2016). In software dependency graphs, communities correspond naturally to ecosystem sub-communities: all packages from the same framework, or all packages serving the same functional role, tend to depend on each other more than on packages from a different ecosystem.

Community quality is quantified by *modularity* Q:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

where $m = |E|$, k_i is the degree of node i, and $\delta(c_i, c_j) = 1$ if and only if nodes i and j belong to the same community. Modularity compares the density of intra-community edges against the expectation under a random null model with the same degree sequence; values above approximately 0.3 are generally considered to indicate meaningful community structure (Barabási, 2016).

The *Louvain algorithm* (Blondel et al., 2008) is a two-phase greedy modularity optimisation (Blondel et al., 2008). In the first phase, each node is assigned to the community of one of its neighbours that maximises the local modularity gain. In the second phase, all nodes in a community are aggregated into a single super-node, and the process repeats on the reduced graph. The algorithm terminates when no further modularity gain is possible. Its practical complexity is $O(n \log n)$, making it tractable on large SBOM graphs. SBOM Lens applies the Louvain algorithm to the undirected projection of the dependency graph and reports the resulting community structure in Chapter 6.

From a security perspective, detected communities reveal which components form tightly coupled subsystems. A vulnerability that enters one high-modularity community tends to spread within it, contained by the sparse inter-community edges. Com-

munities with many inter-community connections are higher-risk propagation bridges.

2.4.5 K-Core Decomposition

The *k-core* of a graph is the maximal induced subgraph in which every vertex has degree at least k within that subgraph (Batagelj et al., 2003). The *coreness* (or core number) of a vertex v is the largest k such that v belongs to the k-core. Coreness is a measure of how deeply embedded a component is in the dependency network.

K-core decomposition is computed by the shell-peeling algorithm of Batagelj and Zaversnik (2003), which runs in $O(|E|)$ time (Batagelj et al., 2003). The algorithm iteratively removes all vertices with degree less than 1 (the 0-shell), then all vertices with degree less than 2 (the 1-shell), and so on, until no vertices remain. The innermost core — the subgraph with maximum coreness $k_{\max}$ — is the architectural backbone of the dependency network: components that are mutually highly interconnected and collectively difficult to remove.

In the supply chain context, high-coreness components are the most entrenched in the dependency network. They are linked, in either direction, to many components that are themselves densely linked. Such a component is hard to replace: replacing it requires finding an alternative that satisfies all components in the inner core simultaneously. SBOM Lens computes coreness on the undirected projection of the dependency graph using `nx.core_number` and reports the maximum coreness and the composition of the innermost core. This analysis is applied in Chapter 6.

2.4.6 Spectral Analysis and Propagation Modelling

Graph-based vulnerability analysis can be extended beyond the structural metrics described above in two ways. The first is *static spectral analysis*: the spectral radius λ_1 of the adjacency matrix governs how readily a contagion spreads over a network, with $1/\lambda_1$ acting as its epidemic threshold (Wang et al., 2003; Pastor-Satorras et al., 2001), and the corresponding leading eigenvector (the eigenvector centrality introduced above) gives each node's share of that spread potential. The second is *propagation simulation*: running a dynamic epidemic model, such as the SIR (Susceptible–Infected–Recovered) framework, over the dependency network through time to estimate the fraction of components a single compromise would eventually reach.

SBOM Lens adopts the first and leaves the second out of scope. The spectral radius, the epidemic threshold, and a per-component spectral exposure score are computed statically from the graph and are presented in Chapter 6; no contagion is simulated. Propagation simulation is identified as a direction for future work in Chapter 8.

2.5 Vulnerability Databases

SBOM Lens overlays vulnerability data on the dependency graph to identify which components carry known security risks and to quantify the severity of those risks. This

section introduces the main vulnerability databases — CVE/NVD, OSV, and GitHub Security Advisories — explains why SBOM Lens sources its data from OSV, and establishes the CVSS v3.1 scoring system used to quantify severity. Definitions here are referenced without re-explanation in Chapter 5.

2.5.1 CVE and the National Vulnerability Database

Common Vulnerabilities and Exposures (CVE) is a programme maintained by MITRE (funded by CISA and the US government) that assigns unique identifiers to publicly disclosed software vulnerabilities (The MITRE Corporation, n.d.). A CVE ID takes the form `CVE-<YEAR>-<SEQUENCE>`, for example `CVE-2021-44228` for the log4shell vulnerability. Each CVE entry provides a standardised description, a list of affected products, references to public advisories and patches, and severity scores. CVE is the canonical cross-database identifier: entries in NVD, OSV, and GitHub Security Advisories all reference CVE IDs when applicable, enabling deduplication across sources.

The NIST National Vulnerability Database (NVD) enriches CVE entries with CVSS base scores (versions 2.0, 3.x, and 4.0), Common Platform Enumeration (CPE) product matching strings, and categorical severity ratings (National Institute of Standards and Technology, n.d.). NVD provides the authoritative CVSS v3.1 base scores for most published CVEs and is the primary reference for severity data in the security industry.

SBOM Lens does not query NVD directly. NVD's primary product identifier is CPE rather than purl, so matching an SBOM component against NVD requires a name-and-version mapping step that introduces imprecision. The CPE format, `cpe:2.3:a:<vendor>:<product>:<ve`
is less precise for open-source packages than purl-based matching: the same package may have multiple CPE strings or none at all. This limitation is discussed in Chapter 5.

2.5.2 Open Source Vulnerabilities (OSV)

The Open Source Vulnerabilities (OSV) database is a distributed vulnerability database designed natively for open-source ecosystems (Open Source Security Foundation, n.d.[a]). Its schema, maintained by the OpenSSF, defines version range bounds using `introduced` and `fixed` fields rather than fixed version lists, enabling precise matching against any version in a given range. OSV aggregates vulnerability data from over fifty upstream sources including RustSec, PyPI advisories, npm advisories, the GitHub Advisory Database, the Debian Security Tracker, and the Alpine Security Database.

Each OSV entry carries an `aliases` field that cross-references CVE IDs and GHSA identifiers, enabling deduplication across sources. OSV's query API (`/v1/query`) accepts a package purl directly, making it the most precise source for open-source component vulnerability matching and the vulnerability source used by SBOM Lens. The integration pipeline is described in Chapter 5.

2.5.3 GitHub Security Advisories

The GitHub Advisory Database is a curated collection of security advisories for open-source packages, using GHSA identifiers in the format `GHSA-xxxx-xxxx-xxxx` (GitHub, n.d.). Its strongest ecosystem coverage is for npm (JavaScript), PyPI (Python), RubyGems, Maven, NuGet (.NET), Go, and Rust. GitHub advisories are published in OSV format and are therefore aggregated by the OSV database.

Because GHSA advisories are aggregated by OSV, querying OSV covers them without a separate GitHub query. SBOM Lens therefore queries OSV alone and relies on it to reach GHSA and the other upstream databases. Findings are stored once per (CVE ID, purl) pair; the deduplication logic is presented in Chapter 5.

2.5.4 CVSS v3.1 Scoring System

The Common Vulnerability Scoring System (CVSS) v3.1 is the standard for quantifying the severity of software vulnerabilities (Forum of Incident Response and Security Teams, 2019). The specification is maintained by FIRST.org (Forum of Incident Response and Security Teams). CVSS produces a numeric base score on the interval $[0.0, 10.0]$ from a set of standardised metrics that characterise the intrinsic properties of the vulnerability, assessed at the time of publication.

The base metric group comprises eight metrics divided into two sub-groups plus a Scope modifier. The four *exploitability metrics* characterise how the vulnerability can be accessed: **Attack Vector** (AV: Network, Adjacent, Local, or Physical), **Attack Complexity** (AC: Low or High), **Privileges Required** (PR: None, Low, or High), and **User Interaction** (UI: None or Required). The **Scope** modifier (S: Unchanged or Changed) indicates whether a successful exploit can affect components beyond the vulnerable component — for example, when a container escape vulnerability affects the host operating system. The three *impact metrics* characterise the effect on the CIA triad of the affected system: **Confidentiality Impact** (C), **Integrity Impact** (I), and **Availability Impact** (A), each rated None, Low, or High (Forum of Incident Response and Security Teams, 2019).

The base score is computed from these eight metrics via the following formula (Forum of Incident Response and Security Teams, 2019). Let $\text{ISS} = 1 - [(1-C)(1-I)(1-A)]$ where C, I, and A are numerical values from the specification's lookup table. The impact sub-score is:

$$\text{Impact} = \begin{cases} 6.42 \times \text{ISS} & \text{if Scope Unchanged} \\ 7.52 \times (\text{ISS} - 0.029) - 3.25 \times (\text{ISS} - 0.02)^{15} & \text{if Scope Changed} \end{cases}$$

The exploitability sub-score is $\text{Exploitability} = 8.22 \times \text{AV} \times \text{AC} \times \text{PR} \times \text{UI}$, where each

metric takes its numerical value from the lookup table. The base score is then

$$\text{BaseScore} = \begin{cases} \min(\text{Impact} + \text{Exploitability},\ 10) & \text{if Scope Unchanged} \\ \min(1.08 \times (\text{Impact} + \text{Exploitability}),\ 10) & \text{if Scope Changed} \end{cases}$$

rounded up to the nearest 0.1. If $\text{Impact} = 0$, the base score is 0.0 regardless of exploitability.

FIRST.org defines five severity bands: None (0.0), Low $(0.1\text{–}3.9)$, Medium $(4.0\text{–}6.9)$, High $(7.0\text{–}8.9)$, and Critical $(9.0\text{–}10.0)$. The log4shell vulnerability (CVE-2021-44228) illustrates the maximum score: its vector string `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:` yields $\text{BaseScore} = 10.0$ because the vulnerability is network-exploitable without authentication, affects scope beyond the vulnerable component, and has maximum confidentiality, integrity, and availability impact. SBOM Lens uses these severity bands to colour-code vulnerable nodes on the SBOM graph — red for Critical, orange for High, yellow for Medium, and green-yellow for Low — as described in Chapter 5.

The Exploit Prediction Scoring System (EPSS, also maintained by FIRST.org) complements CVSS by estimating the probability that a CVE will be exploited in the wild within thirty days, using a machine-learning model trained on observed exploitation data (Forum of Incident Response and Security Teams, n.d.). While CVSS measures the potential severity of a vulnerability if exploited, EPSS measures the likelihood of exploitation. SBOM Lens does not currently integrate EPSS scores; this is noted as a limitation and a direction for future work in Chapter 8.

3
Related Work

Situating SBOM Lens within existing work requires surveying three bodies of literature that do not ordinarily appear side by side: the practical tooling landscape for SBOM analysis, the academic research programme on graph-based software security, and a pair of supply chain incidents that illustrate the structural blindspots the tools leave open. This chapter covers each in turn. Section 3.1 surveys four open-source SBOM tools using a feature matrix. Section 3.2 organises the academic literature into four research clusters and connects each to an SBOM Lens design decision. Section 3.3 re-examines the SolarWinds SUNBURST compromise and the log4shell disclosure through the lens of SBOM graph analysis rather than incident narrative. Section 3.4 synthesises the survey into four named research gaps that SBOM Lens addresses in Chapter 4, Chapter 5, and Chapter 6.

3.1 Existing SBOM Analysis Tools

The SBOM tooling ecosystem has grown rapidly since the US Executive Order 14028 of 2021 accelerated adoption across government-adjacent supply chains. A 2025 landscape study of 84 open-source SBOM tools (Garcia et al., 2025) has no category for analysing the dependency structure an SBOM records. Four open-source tools are surveyed here — three in wide use, and Sunshine, the CycloneDX project's own SBOM viewer: Dependency-Track (OWASP), grype and syft (Anchore), and Sunshine (CycloneDX project). Commercial tools such as FOSSA also exist but are beyond the scope of this survey, which focuses on open-source solutions whose internals can be examined and extended. Closest to the graph-adjacent end of the spectrum is GUAC (Graph for Understanding Artifact Composition), which joined OpenSSF as an incubating project in 2024 (Open Source Security Foundation, 2024); GUAC constructs graphs across multiple SBOMs at portfolio scale and can search them for the dependents of a vulnerable package, but it computes no dominance or structural measures; its scope is complementary to the within-SBOM structural analysis that SBOM Lens addresses.

Table 3.1 compares the four open-source tools along four dimensions: SBOM format support, graph capability, vulnerability integration, and scope/purpose. SBOM Lens appears as the final row, separated by a rule, so the capability gaps are structurally visible.

Dependency-Track is an OWASP flagship project for continuous component analysis across an organisation's portfolio (OWASP Foundation, n.d.[b]). Projects upload SBOMs, and the platform matches every component against several vulnerability intelligence sources, re-evaluates them as new advisories appear, and applies security and licence policies. It is the closest mainstream tool to SBOM Lens in purpose, and it can display the dependency tree recorded in a CycloneDX SBOM. Its analysis, however, remains per component: findings are listed and prioritised by severity and exploit likelihood (EPSS), and the tree is a navigation aid — it can be opened at the path to an affected component — not an object of computation. No dominance or centrality is computed over it.

grype is a vulnerability scanner that takes a container image, a directory or an SBOM and reports the known vulnerabilities of every package it finds, drawing on an aggregated vulnerability database (Anchore, n.d.[a]). It is designed for point-in-time scanning in continuous integration pipelines, and its output is a list of (package, vulnerability, severity) findings. *syft*, from the same vendor, is its complement on the generation side: it produces SBOMs in CycloneDX and SPDX from images, file systems and source trees, and performs no vulnerability analysis of its own (Anchore, n.d.[b]). Together they represent the standard "generate, then scan" workflow, in which the dependency relationships recorded in the SBOM are carried along but never analysed.

Sunshine, from the CycloneDX project, is a browser-based viewer that renders a CycloneDX SBOM's components, dependencies and vulnerabilities as an interactive sunburst chart and tables (CycloneDX, n.d.). It is the only surveyed tool whose primary purpose is to show the dependency structure, and it demonstrates the value of seeing vulnerabilities in their dependency context. Its view, though, is hierarchical: a sunburst draws each dependency path as nested rings, so a component reached along several paths is drawn several times. Sunshine does propagate each vulnerability to every component that depends on it, directly or transitively, and lists those components in a table, but it does so by enumerating dependency paths, and it computes nothing over the structure beyond that reachability.

Across the four tools, the SBOM's dependency relationships are either discarded, displayed, or at most traversed as a tree, to find the path to a component or the components that depend on it. None computes anything over them beyond such reachability. This is the first of the gaps identified in Section 3.4.

3.2 Graph-Based Security Research

A substantial body of academic work applies graph theory and network science to software dependency networks, primarily at the package-ecosystem scale. This section

Table 3.1: *Comparison of SBOM analysis tools across four capability dimensions. SBOM Lens (last row) is the reference implementation evaluated in this thesis. Capabilities as of October 2026 (Dependency-Track 5.2).*

Tool	Format Support	Graph Capability	Vuln Integration	Scope
Dependency-Track	CycloneDX only (SPDX for licence IDs)	Dependency tree view; no graph analysis	NVD, GHSA, OSV, OSS Index, Snyk, Trivy, VulnDB, JVN; EPSS, CISA KEV (5.1+)	Portfolio-wide continuous monitoring
grype	CycloneDX, SPDX	None — flat CVE list	Aggregated DB (NVD, GHSA, OS-distribution feeds), EPSS, CISA KEV	Point-in-time vulnerability scan; CI/CD
syft	CycloneDX (JSON+XML), SPDX	None — SBOM generator only	None (generation only)	SBOM generation from images, filesystems, source
Sunshine	CycloneDX JSON only	Visual/hierarchical only (sunburst chart) — no graph algorithms	EPSS, CISA KEV enrichment; SQL query interface	Interactive SBOM viewer and SQL querying
SBOM Lens	CycloneDX JSON	Directed dependency graph; blast radius, dominated sets, articulation points, HITS, spectral analysis, core numbers, Louvain communities, whole-graph topology	OSV (aggregating GHSA and others); CVSS v3.1 base score; vulnerability–graph overlay	Within-SBOM structural risk assessment

surveys four research clusters — dependency graph analysis, software vulnerability graphs, call graph and static analysis, and network science applied to software — and positions SBOM Lens relative to each. Each cluster closes with a forward reference to the SBOM Lens implementation chapter where the cluster's insights are applied.

3.2.1 Dependency Graph Analysis

Empirical studies of package dependency networks have established that open-source ecosystems form large graphs with highly unequal, heavy-tailed connectivity, in which a small number of packages are transitively depended upon by the majority of the ecosystem. Kikas et al. (2017) analysed the dependency networks of the JavaScript, Ruby and Rust ecosystems over time and found that the number of transitive dependencies of a typical package grows steadily, and that the removal of a single package can break a substantial share of the ecosystem. Decan et al. (2019) compared the evolution of seven packaging ecosystems, including npm, and showed that all of them grow over time with a heavy reliance on transitive dependencies, making each package exposed to changes far beyond the packages it declares. For npm specifically, Zimmermann et al. (2019) found that individual packages could impact large parts of the ecosystem, and that a very small number of maintainer accounts could be used to inject malicious code into the majority of all packages.

These studies share a unit of analysis: the *ecosystem*. Their graphs contain every package of a registry, and their findings are statements about the registry as a whole. They establish why dependency structure matters for security, but they do not tell the owner of a single application which of its components are structurally critical. SBOM Lens transfers the same kind of analysis to the dependency graph of one project, as described by its SBOM (Chapter 6), and Chapter 7 asks whether the ecosystem-scale findings — heavy-tailed degree distributions, concentration on a few packages — also hold at that scale.

3.2.2 Software Vulnerability Graphs

A second cluster of work models vulnerability propagation through dependency graphs, moving beyond per-package CVE counts toward structural propagation metrics. Decan et al. (2018) studied how security vulnerabilities in npm packages affect the packages that depend on them, and how long it takes for fixes to reach those dependents; many dependents remain exposed long after a fix is published, because their version constraints do not admit it. Liu et al. (2022) reconstructed the resolved dependency trees of npm packages and traced how vulnerabilities propagate along them, showing that a large share of vulnerable dependencies are introduced transitively and that the resolution rules of the package manager decide whether a fix can propagate at all. Pashchenko et al. (2018) showed that counting every vulnerable dependency overstates the problem: dependencies used only for development or testing are never shipped,

and excluding them materially changes the picture — an observation that motivates the restriction of the evaluation in Chapter 7 to production dependency graphs.

This body of work confirms that vulnerability exposure is a property of the dependency graph, not of a package in isolation. Like the studies of the previous subsection, however, it measures propagation across an ecosystem, or counts vulnerable dependencies per library across a sample (Pashchenko et al., 2018), rather than analysing the structure of one project's graph. SBOM Lens applies the same idea inside a single project: the blast radius of a vulnerable component and the components it dominates are computed on the project's own graph and shown on it (Chapter 5).

3.2.3 Call Graph and Static Analysis

Call graph reachability analysis determines whether a vulnerable function is actually reachable from application code, filtering CVE noise to the paths along which the vulnerable code can actually be executed. Ponta et al. (2018) combined static and dynamic analysis to determine whether the code affected by a known vulnerability is actually executed by an application, and showed that combining the two finds reachable vulnerable code that either technique alone would miss. Reachability of this kind is the most precise answer to "does this vulnerability affect me?", and it is complementary to SBOM Lens rather than in competition with it.

SBOM Lens deliberately works at the level of components rather than functions. An SBOM records components and their dependency relationships; it contains no code, and many SBOMs are produced and shared without access to the source they describe. Component-level analysis therefore applies wherever an SBOM exists, at the cost of over-approximating exposure: a component in the blast radius of a vulnerability depends on the vulnerable component, but may never call the vulnerable function. The structural measures of Chapter 6 are also meaningful without any vulnerability at all, which function-level reachability is not. Combining the two — using call-graph reachability to prune the vulnerability overlay — is left as future work.

3.2.4 Network Science Applied to Software

The network science literature provides the theoretical toolkit that SBOM Lens applies to individual SBOM dependency graphs. Software has been studied as a complex network for over two decades. Valverde et al. (2002) found scale-free degree distributions in the graphs of classes of object-oriented systems, and Myers (2003) showed that the collaboration graphs of several large software systems are both scale-free and small-world, relating these properties to how software is designed and evolves. Louridas et al. (2008) found power laws across a wide range of software structures, from class dependencies to library usage, concluding that they are the norm rather than the exception. The same concepts — degree distributions, clustering, path lengths, centrality, communities (Section 2.4) — underlie the ecosystem studies above.

Two lessons carry over to SBOM Lens. First, heavy-tailed structure means that a few components carry a disproportionate share of the dependency relationships, so structural measures can single them out. Second, claims about such structure need careful statistics: Clauset et al. (2009) showed that many reported power laws do not withstand rigorous fitting, which is why SBOM Lens fits the degree distribution by maximum likelihood (Section 6.3.7). What the literature has not done, to our knowledge, is apply this toolkit systematically to the dependency graph of a single application, as a routine part of analysing its SBOM; Chapter 6 does so.

3.3 Supply Chain Incident Case Studies

Both cases examined below were introduced in Section 1.1 as evidence of the structural risks motivating SBOM Lens. Rather than re-narrating the incidents, this section frames each through the SBOM graph analysis lens: what information was encoded in the dependency graph, and what graph-theoretic metrics would have made the structural exposure visible before or immediately after disclosure.

3.3.1 SolarWinds SUNBURST (December 2020)

As introduced in Section 1.1, the SolarWinds SUNBURST compromise demonstrated build-pipeline supply chain risk (Cybersecurity and Infrastructure Security Agency, 2020); this subsection examines what graph-based SBOM analysis would have revealed.

The answer is instructive precisely because it is limited. The malicious code was injected into the vendor's own build process and shipped inside a legitimately signed release of the Orion product. No dependency was added, and no known vulnerability existed in any component until the compromise was discovered. An SBOM of Orion would have listed the same component names and versions as an uncompromised build — only file hashes, compared against an independent reference build, could have differed — and no analysis of that SBOM — flat or graph-based — could have detected the attack. Detecting build-pipeline compromise requires provenance and build-integrity controls such as those of SLSA (Open Source Security Foundation, n.d.[b]), not SBOM analysis.

Graph analysis becomes useful after disclosure. Once a compromised component is known, the questions are the ones SBOM Lens answers: which parts of each affected application depend on it, directly or transitively (its blast radius), and which parts would be cut off if it were removed or replaced (its dominated set). For SUNBURST, the compromised artefact was a product deployed across an organisation rather than a library inside one application's dependency graph, so the relevant graph is the organisation's portfolio of software — the scope addressed by GUAC (Open Source Security Foundation, 2024) — rather than the single-SBOM scope of SBOM Lens. The case therefore marks a boundary of this work: SBOM Lens supports impact assessment for components inside an application's dependency graph, not detection of compromised

builds, and not portfolio-wide analysis.

3.3.2 log4shell (CVE-2021-44228, December 2021)

As introduced in Section 1.1, the log4shell vulnerability (National Institute of Standards and Technology, 2021) exposed the hidden scale of transitive dependency risk (Wetter et al., 2021); this subsection quantifies what an SBOM graph would have shown.

The ecosystem-wide scale was measured shortly after disclosure: an analysis of Maven Central found that over 17,000 Java packages — about 4% of the repository — depended on an affected version of `log4j-core`. An earlier count in the same analysis, which also included `log4j-api`, found that the majority of affected artifacts did so only through indirect dependencies, and that for more than 80% of them the vulnerable package sat more than one level deep, in some cases nine (Wetter et al., 2021). For an individual application, the first question after disclosure was whether it contained `log4j-core` at a vulnerable version at all. A flat inventory answers that question. The next questions — which parts of the application reach it, through which chains, and whether it could be removed or replaced without breaking anything else — require the graph.

To make this concrete, Table 3.2 shows what SBOM Lens reports for `log4j-core` 2.14.1 in the illustrative Java SBOM distributed with the tool, a small application with 12 components, which depends on `log4j-core` directly. The blast radius contains the application itself; the dependency path shows how `log4j-core` enters the graph; the dominated set shows that `log4j-api` is reachable only through `log4j-core`, so removing the vulnerable component would also remove its API package; and `log4j-core` is an articulation point of the undirected graph. In this small example the vulnerable component sits one hop from the root, so every measure is small. In the transitive situation that dominated the real incident, the dependency paths would run through intermediate libraries, the blast radius would include every one of them, and the question of whether `log4j-core` can be removed would depend on which of those libraries require it — exactly the information the graph provides and a flat inventory does not. Chapter 7 measures these quantities across real npm applications.

Table 3.2: *What SBOM Lens reports for `log4j-core` 2.14.1 in the illustrative Java SBOM distributed with the tool (13 nodes — the application and 12 components — and 13 dependency edges).*

Measure	Result
Blast radius	`my-spring-app` (the application) and `log4j-core` itself
Dependency path	`my-spring-app` → `log4j-core`
Dominated set	`log4j-api`
Articulation point	Yes
HITS authority	0.167, tied with the application's five other direct dependencies (all other components: 0)
Spectral exposure score	1.15 (maximum in the graph: 2.77, the application itself)
Core number	1 (maximum in the graph: 2)

3.4 Gap Analysis

The survey above reveals four capabilities that, to our knowledge, current tooling and the research literature do not provide; SBOM Lens addresses each of these gaps. The evidence for each absence is grounded in Table 3.1 and the research clusters surveyed in Section 3.2.

Gap 1: no graph analysis in SBOM tooling. The surveyed tools treat an SBOM as an inventory. Dependency relationships are discarded, displayed as a tree, or carried along unchanged, and at most traversed to find the path to, or the dependents of, a component; no surveyed tool computes structural measures over them (Section 3.1). SBOM Lens builds a directed dependency graph from the CycloneDX `dependencies` array and makes it the central data structure of the tool (Chapter 4).

Gap 2: no vulnerability impact in graph terms. When a vulnerability is found, the tools report the affected component and its severity. At most they list the components that depend on it (Sunshine) or open a tree view at the path to it (Dependency-Track); outside the survey, GUAC's patch planning searches the dependents of a vulnerable package. None shows the dependents and the paths together on the dependency graph, and none reports which components could not be reached without it. The research that models vulnerability propagation aggregates over whole ecosystems or samples of projects (Section 3.2.2). SBOM Lens overlays vulnerabilities on the project's graph and computes, for each vulnerable component, its blast radius, the dependency paths that lead to it, and its dominated set (Chapter 5).

Gap 3: no structural prioritisation beyond severity. Findings are ranked by CVSS score, sometimes combined with exploit likelihood (EPSS) and known exploitation (CISA KEV) into a risk score, as grype does by default; all of these describe the vulnerability rather than where the affected component sits. Two components with the same CVE are treated identically whether one is a leaf and the other the only route to half the application, and components without a current CVE are not ranked at all. The nearest exception is the experimental guided remediation of OSV-Scanner, which, for npm and Maven manifests and lockfiles, also weighs dependency depth and the number of vulnerabilities an upgrade fixes; it uses no position in the graph beyond depth. SBOM Lens computes structural measures at the component, cluster and whole-graph scales — including centrality, spectral exposure, core numbers and communities — that rank components by their position in the graph (Chapter 6).

Gap 4: graph analysis at the wrong scale. The research that does analyse dependency graphs works at the scale of whole ecosystems or of the software's internal code structure (Section 3.2.1, Section 3.2.4), and portfolio-scale tools such as GUAC work across many projects. The dependency graph of a single application, as described by its SBOM, has not, to our knowledge, been systematically analysed with these methods. Recent work has begun to study SBOMs as graphs — whether their dependency edges are declared at all (Zięba-Kozarzewski, 2026), and learned prediction of vulnerable components over enriched SBOM graphs (Baird et al., 2026) — but not with the

network-science measures above, and it is not known whether ecosystem-scale properties hold at that scale. SBOM Lens applies the methods at that scale (Chapter 6), and Chapter 7 measures, across real applications, what the structural view reveals that a flat inventory does not.

4
System Design

Chapter 4 describes the design and implementation of SBOM Lens — the system whose absence from existing tooling Section 3.4 identified as the central research gap. The chapter is organised into four sections. Section 4.1 presents the three-layer architecture and the rationale behind its main design choices. Section 4.2 traces the data flow from a CycloneDX JSON file upload through JSON validation, duplicate detection, component extraction, and directed-edge construction, to the in-memory `nx.DiGraph`. Section 4.3 defines the node and edge data model that all algorithms in Chapter 5 and Chapter 6 operate on. Section 4.4 describes the interactive Cytoscape.js visualisation layer — the base rendering, layout options, and core interaction model.

4.1 System Architecture

SBOM Lens is structured as a three-layer system: a React single-page application (frontend), a FastAPI web service (backend), and a REST API that decouples them. Each layer has a single, bounded responsibility — the frontend handles rendering and user interaction, the backend holds all graph computation, and the REST API defines the contract between the two. This section presents the three layers and the design rationale that shaped their boundaries.

4.1.1 Overview and Design Rationale

SBOM Lens follows a three-layer architecture that separates user interaction, graph computation, and data transfer into distinct, independently testable concerns. The frontend (React with Cytoscape.js) is responsible for rendering the SBOM Graph, presenting the vulnerability dashboard, and mediating user input. The backend (FastAPI with NetworkX) owns all structural computation — graph construction, metric calculation, risk assessment, and vulnerability aggregation. A REST API defines the contract between the two layers, allowing either side to evolve without coupling.

The principal motivation for this separation is analytical correctness. Running graph

algorithms in the browser would require reimplementing the backend graph logic in JavaScript, duplicating code that would be harder to test and audit. By keeping computation server-side, the algorithms described in Chapter 5 and Chapter 6 can be verified against known graph fixtures independently of the frontend.

Persistence is deliberately in-memory for this thesis scope. `SBOMStore` holds up to `MAX_SBOMS` SBOM records and their corresponding `nx.DiGraph` objects in process memory; the default limit is fifty, configurable via environment variable. Graph state resets on server restart. This design is intentional: the system is evaluated against a fixed research dataset, and the absence of a database removes an entire class of schema-migration and query-optimisation concerns that would distract from the analytical contribution. Persistent storage is identified as future work.

The architecture also includes modules not evaluated in this thesis — a Topology Optimiser and a Comparison/Delta analysis view — which are shown in Figure 4.1 with a visual distinction to be transparent about the system's full scope while clearly delineating what this thesis covers. All eight registered FastAPI routers are listed for architectural completeness.

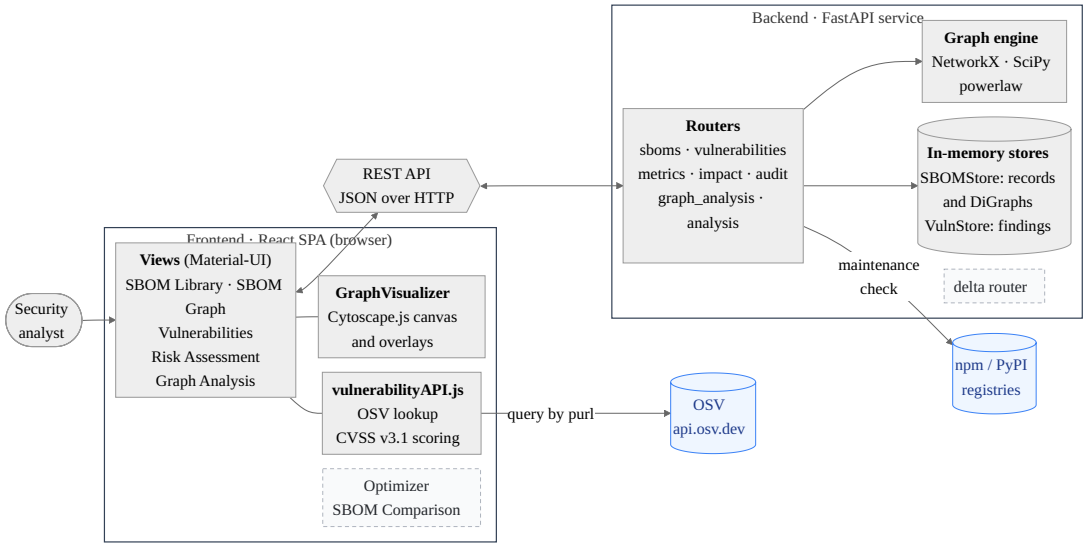


Figure 4.1: *SBOM Lens three-layer architecture. Vulnerabilities are looked up in the browser and stored by the backend; the backend's audit queries the package registries. Modules outside the thesis scope (Topology Optimiser, SBOM Comparison and the delta router) are drawn dashed.*

4.1.2 Frontend Layer

The frontend is a React single-page application styled with the Material-UI (MUI) component library. A persistent side navigation bar connects five pages within the scope of this thesis: the Dashboard (the SBOM library and its vulnerability totals), the SBOM Graph (the central visualisation canvas), Vulnerabilities (the stored vulnerability records and their remediation status), Risk Assessment (the whole-graph audit of Section 6.3), and Graph Analysis (degree and depth distributions). The navigation

also lists an Optimizer, an SBOM Comparison view and a Fix Priorities page, which are outside the scope of this thesis. Figure 4.2 shows the application with the SBOM Graph page open.

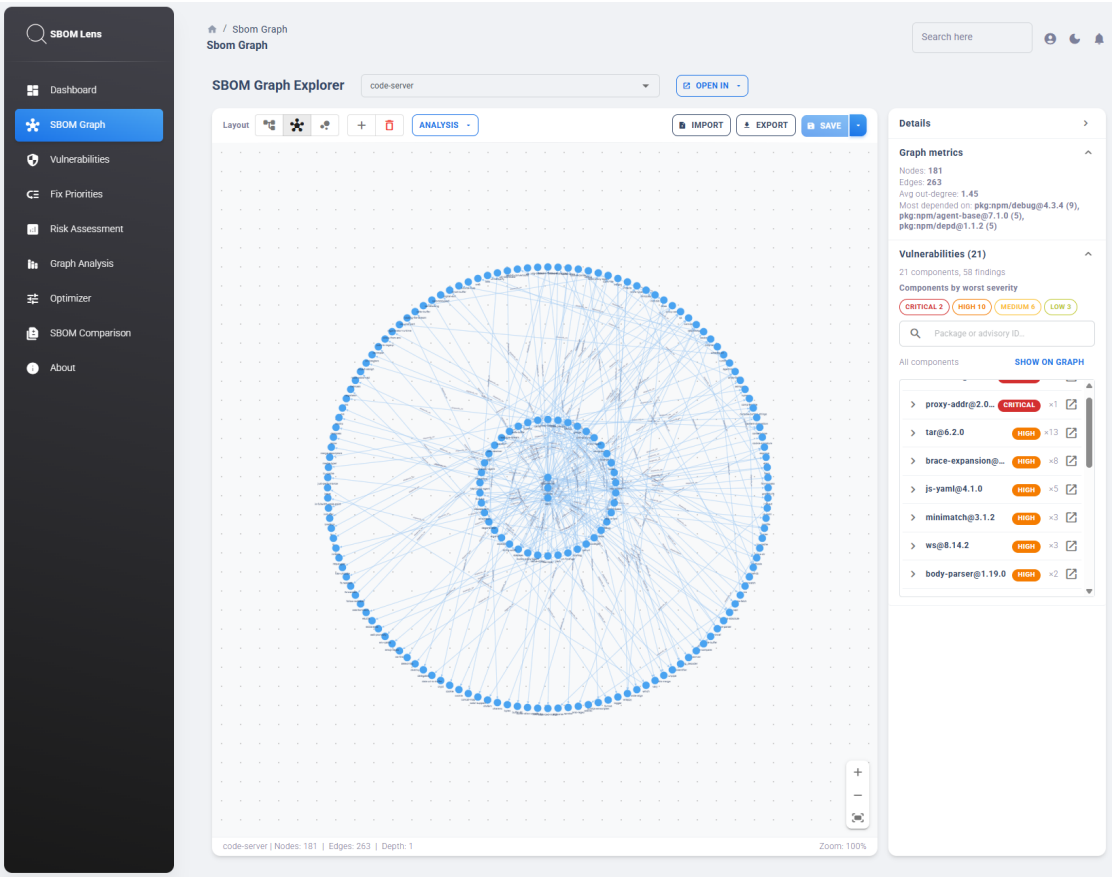


Figure 4.2: *The SBOM Lens interface with the `code-server` SBOM loaded on the SBOM Graph page. The side navigation on the left links the pages of the application; the graph is drawn in the default radial layout, and the Details panel on the right summarises the graph and its vulnerabilities.*

Graph rendering is delegated entirely to Cytoscape.js through a `GraphVisualizer` React component that holds the Cytoscape instance in a `useRef` hook. This architectural choice is deliberate: Cytoscape.js manages the HTML5 canvas directly and responds to user gestures with sub-millisecond latency. Routing all canvas operations through React's virtual-DOM reconciler would introduce unnecessary re-render overhead for every pan, zoom, or node drag event. React manages application state — which SBOM is loaded, which analysis mode is active — while Cytoscape manages the visual representation.

The data layer is controlled by a compile-time constant `DATA_SOURCE` defined in the application configuration file. It switches between `'localStorage'` (an offline convenience mode used during frontend development) and `'api'` (the production path that calls the FastAPI backend). The thesis evaluation uses the `'api'` mode exclusively. The analysis toolbar, which exposes the overlay modes described in Chapter 5 and Chapter 6, is visible only in API mode when an SBOM is loaded.

4.1.3 Backend Layer

The backend is a FastAPI application defined in `main.py` with eight routers registered at startup: `sboms`, `vulnerabilities`, `metrics`, `impact`, `audit`, `graph_analysis`, `analysis` (the analysis report of several SBOMs), and `delta`. Registration order is significant: the `delta` and `graph_analysis` routers are registered before `sboms` to prevent FastAPI's wildcard path parameter `{sbom_id}` from greedily intercepting fixed paths such as `/sboms/compare`.

The graph engine is NetworkX's `DiGraph` class. All structural algorithms operate directly on the stored `DiGraph` object: these include articulation-point detection, k-core decomposition, HITS centrality, Louvain community detection, and the spectral analysis (spectral radius, epidemic threshold and spectral exposure score), all described in Chapter 6. This avoids any serialisation step between storage and computation: the router receives a Python object reference, runs the algorithm, and returns results without intermediate conversion.

Several routers maintain per-SBOM result caches at the module level. Community detection and k-core results are expensive to recompute for large graphs; caching ensures that repeated requests for the same SBOM complete in constant time. All caches are explicitly invalidated when an SBOM is updated or deleted, guaranteeing consistency between the stored graph and any cached analytical results.

4.1.4 Communication: REST API

The REST API is served at `http://localhost:8000` by default, overridable via an environment variable. CORS is configured to permit requests from the React development server. A health endpoint at `GET /health` returns a JSON object confirming the service status and version number; the frontend uses this to detect backend availability before enabling API-mode features.

The endpoints directly relevant to this chapter are: `POST /sboms` (file upload, graph construction, and storage), `GET /sboms/{id}` (retrieve a stored SBOM record), and `GET /sboms/{id}/metrics` (returns graph-level metrics including node count, edge count, average out-degree, and the five most-depended-on components ranked by in-degree). The remaining endpoints — `/vulnerabilities`, `/{id}/impact`, `/{id}/audit`, `/{id}/graph-analysis`, `/{id}/kcore`, `/{id}/communities`, and `/{id}/epidemic` — are described in Chapter 5 and Chapter 6 respectively.

4.2 CycloneDX Parsing Pipeline

Every SBOM in SBOM Lens enters through a single ingest endpoint and exits as a cached `nx.DiGraph` that all downstream routers query directly. The pipeline is designed for a "parse once, query many times" pattern: the graph is built on first ingest and never re-parsed unless the SBOM is explicitly updated. The three subsections trace

each stage: upload and duplicate detection, component extraction and node creation, and dependency iteration and edge construction.

4.2.1 Ingestion and Deduplication

SBOM ingestion begins at the `POST /sboms` endpoint, which accepts a multipart file upload. The raw file bytes are read in full before any graph construction begins, allowing the system to perform two correctness checks upfront.

The first check is JSON validation. The raw bytes are passed to `json.loads()`; if parsing fails, an HTTP 400 response is returned immediately. This guard ensures that no partially constructed graph can enter the store from a malformed file.

The second check is duplicate detection. A SHA-256 hash of the raw file bytes is computed with `hashlib.sha256(contents).hexdigest()` and compared against the `"hash"` field of every stored SBOM record. If a match is found, the endpoint returns `{"isDuplicate": True, "existingSBOM": existing}` without re-storing the file or reconstructing the graph. This "parse once, query many times" design avoids redundant computation and prevents double-counting in metrics when the same SBOM is submitted more than once.

A capacity limit prevents unbounded memory growth: the `MAX_SBOMS` environment variable (default: 10) caps the number of concurrently stored SBOMs. An upload that would exceed the limit is rejected with HTTP 409 Conflict. This limit is appropriate for a research prototype evaluated on a fixed dataset.

4.2.2 Component Extraction

Once the file passes validation and deduplication, the CycloneDX JSON structure is traversed to extract components. The root component — the project or application being described by the SBOM — is extracted from the `metadata.component` field and prepended to the `components[]` list. This placement gives the root a consistent position in the graph and allows the frontend to distinguish it visually as the primary node.

Each entry in the assembled component list is processed into a NetworkX node. The node identifier is resolved through a three-level fallback chain: `bom-ref` (the CycloneDX primary identifier) is used if present; otherwise `purl` (Package URL) is used; otherwise the component `name`. This chain mirrors the CycloneDX field hierarchy defined in Section 2.1 and ensures stable, unique node identifiers across SBOMs produced by different tools with varying levels of metadata completeness.

Node attributes are stored as the NetworkX node's data dictionary. To avoid compatibility issues with NetworkX's attribute handling, only scalar values — strings, integers, floats, and booleans — are retained from the component object. In practice, the persisted attributes are `name`, `version`, `purl`, `type`, and `bom-ref`. Components for which all three identifier fallbacks yield an empty or `None` value are silently skipped, making the parser tolerant of incomplete SBOMs encountered in the wild.

4.2.3 Graph Construction from Dependencies

After all nodes are created, the `dependencies[]` array is iterated to construct directed edges. Each entry in this array follows the CycloneDX dependency schema: a `ref` field identifying the depending component, and a `dependsOn` list identifying the components it depends upon. For each `(ref, target)` pair, a directed edge is added to the graph with `G.add_edge(ref, target)`.

The edge direction encodes a fundamental semantic: an edge from A to B means "A depends on B". This directed convention underpins all downstream traversal algorithms. Blast-radius computation, for example, uses `nx.ancestors(G, node_id)` to find all components that transitively depend on a given node — ancestors in the directed graph are precisely those dependants, consistent with the `ref` → `target` edge direction.

Before adding each edge, a guard verifies that both `ref` and `target` already exist as nodes in the graph. Entries in `dependencies[]` that reference undeclared components are silently skipped. This tolerance is necessary because real-world SBOMs do not always maintain a complete correspondence between the `components[]` and `dependencies[]` sections. Entries in which a component lists itself as its own dependency are also skipped: such a self-loop carries no dependency information, and several graph algorithms, including the core decomposition of Section 6.1.4, are undefined in its presence.

The resulting `nx.DiGraph` is stored in `SBOMStore.graphs[sbom_id]` and never reparsed unless the SBOM is explicitly updated via `PUT /sboms/{id}`. All downstream routers retrieve the graph by calling `sbom_store.get_graph(sbom_id)`, receiving a direct reference to the live `DiGraph` object with zero serialisation overhead.

4.3 Graph Data Model

The directed graph constructed by the parsing pipeline (Section 4.2) encodes the SBOM's component and dependency structure in a form that all downstream algorithms can consume directly. This section defines the node and edge data model — the representation used by all algorithms in Chapter 5 and Chapter 6. The in-memory store that manages graph lifecycle is described last.

4.3.1 Node Representation

Each node in the `DiGraph` represents one software component as declared in the CycloneDX SBOM. The graph contains exactly one node per distinct component identifier, as determined by the `bom-ref`/`purl`/`name` fallback chain described in Section 4.2.2. The node identifier — the key used to reference the node in all NetworkX operations — is a stable string assigned at ingest time and unchanged for the lifetime of the stored SBOM.

Each node carries a data dictionary with the following attributes:

- `name` — human-readable display label for the component.
- `version` — semantic version string as declared in the SBOM.
- `purl` — Package URL, used as the primary lookup key when matching components against vulnerability databases in Chapter 5.
- `type` — CycloneDX component type; one of `library`, `framework`, or `application`.
- `bom-ref` — the original CycloneDX `bom-ref` string, preserved for traceability back to the source file.

The root component (extracted from `metadata.component`) is a regular node in the NetworkX graph. The frontend distinguishes it visually by setting an `isPrimary` flag in the Cytoscape node data object produced by `convertCycloneDXToGraph()`, which triggers a distinct green fill on the canvas.

4.3.2 Edge Representation

Each directed edge represents a direct dependency relationship between two components. An edge from node A to node B encodes the statement: *A directly depends on B*. This follows from the CycloneDX dependency schema, where an entry `(ref: A, dependsOn: [B, ...])` produces the directed edge A → B.

This directed semantics is load-bearing for every traversal algorithm in the system. Blast-radius analysis identifies all components that would be affected if a given component were compromised; these are its *ancestors* in the directed graph — the nodes from which there exists a directed path to the target, consistent with the A → B convention. Dominator-tree construction, depth histograms, and topological ordering all depend on the same directed convention, as described in Chapter 5 and Chapter 6.

No attributes are stored on NetworkX edges. Edges are pure structural connectors: their presence encodes a dependency, and their direction encodes which component depends on which. On the frontend, each Cytoscape edge object carries an identifier formed from the source and target node IDs, a static label `depends_on`, and the CSS class `default-edge` for baseline styling.

4.3.3 In-Memory Store

The `SBOMStore` class manages the lifecycle of all stored SBOMs and their corresponding graphs. It maintains two parallel dictionaries: `self.sboms` maps each SBOM identifier to a full record dictionary (containing the SHA-256 hash, display name, creation timestamp, and the raw CycloneDX component and dependency lists); `self.graphs` maps the same identifier to the corresponding `nx.DiGraph`.

When a router needs to perform graph analysis, it retrieves the stored `DiGraph` via the store's `get_graph` method, which returns a direct Python reference to the live object. There is no serialisation step between storage and computation: the algorithm receives the graph object and operates on it in place. This design minimises latency

for interactive analysis queries, where sub-second response times are necessary for a usable graph exploration interface.

Several routers maintain module-level caches for computationally expensive results — community detection and k-core decomposition are cached after the first request for a given SBOM. These caches are explicitly invalidated whenever an SBOM is updated or deleted, ensuring that stale results are never returned to the frontend.

As noted in Section 4.1.1, the store's state is ephemeral: a server restart clears all SBOMs and graphs. This design is appropriate for the research context, and persistent storage is listed as a limitation and direction for future work in Chapter 8.

4.4 SBOM Graph Visualisation

The SBOM Graph is the primary interface through which the user explores the dependency structure of a loaded SBOM. This section describes the base rendering layer implemented with Cytoscape.js, the three available layout algorithms, and the core interaction model. Analysis overlays — vulnerability highlighting (Chapter 5) and network-science colouring (Chapter 6) — extend the base layer described here.

4.4.1 Graph Rendering with Cytoscape.js

Cytoscape.js is initialised imperatively inside the `GraphVisualizer` React component using the `useRef` and `useEffect` hooks. The Cytoscape instance is stored in a ref rather than component state, so operations on the canvas — panning, zooming, applying a new layout — do not trigger React's reconciliation cycle. This separation keeps the graph canvas responsive under interactive load.

The default visual encoding uses a consistent style throughout the application. Component nodes are rendered as circular shapes (28×28 pixels) filled with Material-UI's info blue, with the component name displayed as a label below the node. The root component — the project whose dependencies are being analysed — is distinguished by a green fill and a 2-pixel border, making it immediately identifiable as the anchor of the dependency tree. Dependency edges are rendered as Bézier curves with triangle arrowheads at the target end, indicating the direction of dependency. Edges carry the static label `depends_on` with auto-rotate enabled so the label follows the curve.

Overlay CSS classes are pre-defined in the Cytoscape stylesheet and applied programmatically when an analysis mode is activated. The pre-defined overlay classes are: `.articulation-point` (red), `.blast-radius` (purple), `.dominator-tree` (cyan), one class per severity band — `.vuln-critical` (red), `.vuln-high` (orange), `.vuln-medium` (yellow), and `.vuln-low` (green-yellow) —, `.diamond-node` (green), and `.hits-active` (node size and colour scaled from HITS score attributes). The analytical meaning of each overlay class is defined in Chapter 5 and Chapter 6; this section establishes the visual layer only.

4.4.2 Layout Algorithms

SBOM Lens exposes three Cytoscape.js layout algorithms, accessible via toolbar buttons, each suited to a different graph exploration objective.

The **Tree Layout** (Cytoscape name: `breadthfirst`, `directed: true`) organises nodes in horizontal levels according to their distance from the root in a breadth-first traversal. This layout makes dependency depth immediately visible: direct dependencies appear on level one, transitive dependencies on level two and beyond. It is the default layout for the Delta and Comparison views, where structural differences between two SBOMs are easier to compare when both graphs share the same hierarchical orientation.

The **Radial Layout** (Cytoscape name: `concentric`) places nodes in concentric rings around a central hub, with the root component at the centre. This is the default layout applied when a new SBOM is first loaded, giving the user an immediate overview of the dependency neighbourhood around the root.

The **Force-Directed Layout** (Cytoscape name: `cose`) simulates a physical spring system to distribute nodes so that connected components cluster together organically without imposing a hierarchical structure. This layout is particularly effective for revealing natural groupings in large, densely connected SBOMs.

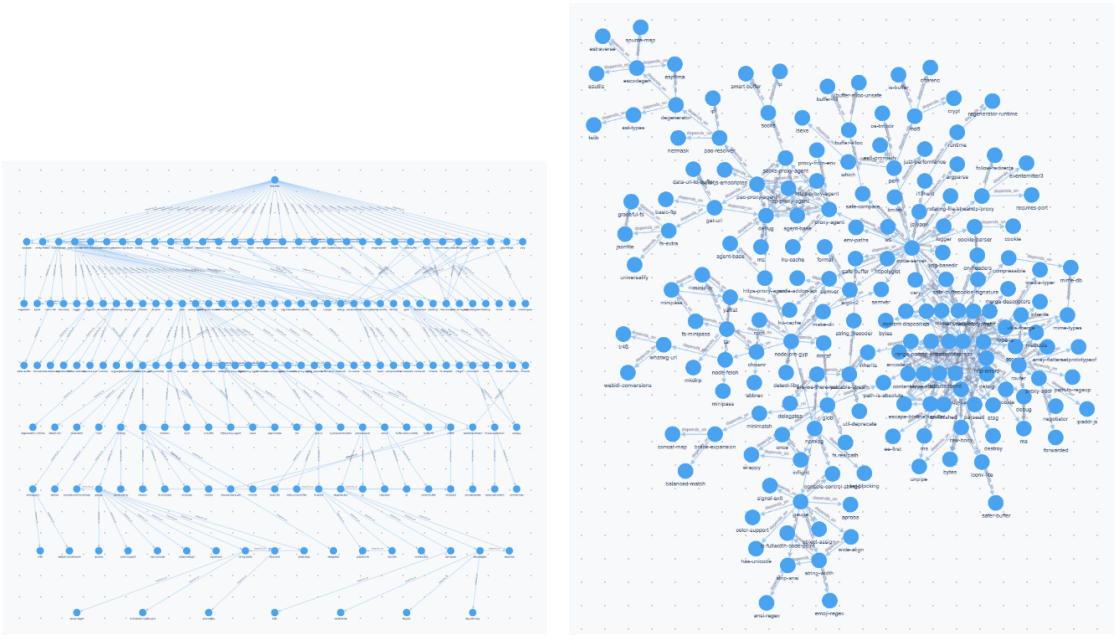
All three layouts share common configuration options: `fit: true` (zoom to fit the graph on screen), a padding of 150 pixels, overlap avoidance, and an animated transition of 600 ms. Layout changes are applied imperatively by re-running the Cytoscape algorithm on the existing node and edge data without issuing a new request to the backend. Figure 4.3 shows the `code-server` SBOM in the tree and force-directed layouts; the radial layout of the same SBOM is the one in Figure 4.2.

4.4.3 Core Interaction Model

The SBOM Graph supports the following core interactions, designed to enable focused exploration of the dependency structure.

Node selection. Tapping a node selects it; a blue border is applied to the highlighted node. Tapping the canvas background deselects all nodes. Double-tapping a vulnerable node opens its records on the Vulnerabilities page.

Details panel. A collapsible panel to the right of the canvas groups the information about the loaded SBOM into expandable sections. The first, *Component*, is open by default and shows the stored attributes of the node last tapped — `name`, `version`, `purl`, `type`, and `bom-ref` — together with the worst severity band and the number of findings of that component, if it has any; tapping the canvas background clears it. These attributes give the analyst the metadata needed to cross-reference the component with external vulnerability databases. *Graph metrics* gives the number of nodes and edges, the average out-degree, and the three most depended-on components. *Vulnerabilities* lists the vulnerable components of the SBOM by worst severity; each expands to its advisories and can be highlighted on the graph. After an impact analysis, a *Dependency*



(a) *Tree* (b) *Force-directed*

Figure 4.3: *The* `code-server` *dependency graph (181 components) in two of the three layouts of SBOM Lens. The tree layout shows dependency depth as levels below the root; the force-directed layout groups densely connected components. The radial layout is shown in Figure 4.2.*

paths section is added (Section 5.5.2). Figure 4.4 shows the panel with a component selected.

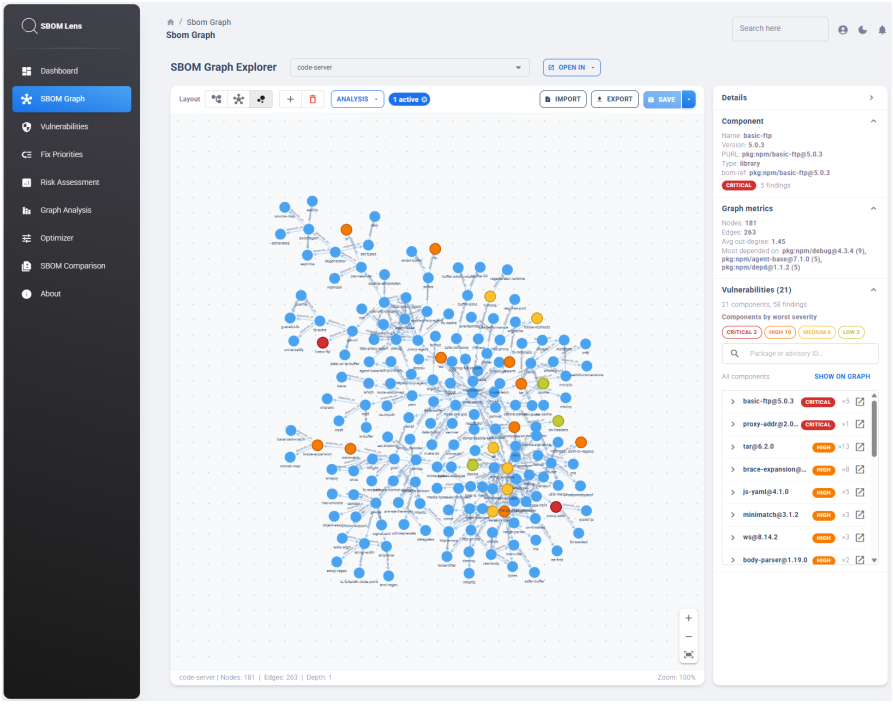


Figure 4.4: *The* `code-server` *SBOM with the severity overlay of Section 5.5.1 active and* `basic-ftp` *selected. Vulnerable components are coloured by their worst severity band: red for* CRITICAL*, orange for* HIGH*, yellow for* MEDIUM*, and green-yellow for* LOW*. The Details panel shows the attributes of* `basic-ftp` *in the Component section, followed by the graph metrics and the vulnerable components of the SBOM.*

Zoom and fit. The current zoom level is tracked in React component state and displayed in the toolbar. A fit-to-view button resets the viewport so the entire graph is visible, which is useful after navigating to a deeply nested subgraph.

SBOM selection. A selector component lists all SBOMs currently stored in `SBOMStore`. Selecting an SBOM converts the stored CycloneDX data to Cytoscape node and edge objects and loads them into the Cytoscape instance.

Impact analysis entry point. Right-clicking a node opens a context menu that includes an impact analysis option, which selects the node for the blast-radius and dominator-tree overlays of the Analysis menu. The analytical meaning of the resulting overlay is described in Chapter 5; the interaction gesture is documented here as part of the interaction model.

Dynamic graph editing features — adding new nodes, repositioning nodes by dragging, and creating edges via the `cytoscape-edgehandles` plugin — are implemented in the application but are outside the scope of this thesis. These are graph construction utilities, not structural risk assessment features, and are not evaluated in this work.

5

Vulnerability Analysis

Chapter 4 established the directed dependency graph that SBOM Lens constructs from a CycloneDX SBOM. Every node in that graph carries a `purl` attribute — a Package URL uniquely identifying the component — as defined in Section 4.3.1. Chapter 5 uses those PURL attributes as the anchor for vulnerability matching: by querying the Open Source Vulnerabilities database (OSV) with the PURLs present in the graph, SBOM Lens enriches the structural model with real-world risk data. OSV aggregates advisories from the GitHub Advisory Database, the PyPA advisory database and many other upstream sources, so a single PURL-native query reaches all of them. The chapter is organised into five sections. Section 5.1 describes why OSV is the vulnerability source and how the integration pipeline works. Section 5.2 describes how vulnerability records are stored in the in-memory `VulnStore` and deduplicated by CVE ID and PURL pair. Section 5.3 explains how CVSS v3.1 base scores are computed on the client side and maps scores to the severity bands used throughout the system. Section 5.4 covers the analyst-facing management workflow: automated fetching on SBOM upload and the three-state status lifecycle exposed through the REST API. Section 5.5 describes how vulnerability data is projected back onto the Cytoscape graph as three distinct overlay types — severity colour coding, blast-radius highlighting, and dominator-tree highlighting — giving analytical meaning to the CSS overlay classes introduced in Section 4.4.1.

5.1 Vulnerability Source

Matching SBOM components against known vulnerabilities requires querying an external database that maintains curated records of published vulnerabilities. This section justifies the choice of OSV as that database and describes the integration pipeline.

5.1.1 Rationale for OSV

No single vulnerability database covers all software ecosystems with equal completeness, and databases differ in how precisely they identify the affected package. NVD, maintained by NIST, identifies products with Common Platform Enumeration (CPE) strings, a scheme designed for commercial software. As noted in Section 2.5.1, matching open-source packages against CPE strings is imprecise: a package may have several CPE strings or none, and name-based matching can return vulnerabilities for unrelated products that share a name. OSV, by contrast, was designed from the outset to accept PURL queries and to express affected versions as ranges, which makes it precise for exactly the identifiers an SBOM carries.

OSV is also an aggregator. It ingests the GitHub Advisory Database (GHSA), the PyPA advisory database, RustSec, the Go vulnerability database, and many other upstream sources, and each entry cross-references CVE and GHSA identifiers in its `aliases` field (Open Source Security Foundation, n.d.[a]). A single OSV query therefore reaches the advisory data that separate GHSA or NVD queries would have provided for open-source packages, without the imprecise name-to-product mapping that NVD requires. Finally, OSV requires no API key, so SBOM Lens produces vulnerability results in any environment without prior registration.

SBOM Lens therefore uses OSV as its sole vulnerability source. The cost of this choice is dependence on OSV's aggregation: an advisory published upstream reaches SBOM Lens only once OSV has ingested it, and vulnerabilities that exist only in NVD, without an open-source advisory, are not reported. Direct integration of further sources is discussed as future work in Chapter 8.

5.1.2 OSV Integration Pipeline

The OSV integration is implemented as a two-phase pipeline in `fetchVulnerabilitiesForSBOM()` inside `vulnerabilityAPI.js`. The first phase issues a single batch request to the `/v1/querybatch` endpoint with an array of PURL queries — one entry per component in the loaded SBOM. This batch design avoids N serial HTTP requests for an SBOM with N components: the entire component list is submitted in one round-trip, and the response returns the matching OSV vulnerability IDs for each PURL position.

The second phase resolves each unique OSV ID to its full advisory record by fetching `/v1/vulns/{id}`. The detail record contains the canonical summary, published and modified dates, a list of CVSS severity objects, and an `aliases` array that may include CVE identifiers. CVE alias resolution is performed by scanning the `aliases` array for an entry beginning with the prefix `"CVE-"`: the first such alias becomes the canonical `cveId` stored in `VulnStore`. If no CVE alias is present, the OSV identifier itself is used as the `cveId` fallback; deduplication logic (Section 5.2.2) still operates correctly on OSV IDs.

Severity is derived in three steps. If the `severity[]` array of the detail record contains an entry whose `type` is `"CVSS_V3"`, its `score` field — a CVSS vector string rather

than a pre-computed number — is passed to `cvssV3BaseScore()` (Section 5.3.3) and the resulting base score is mapped to a severity band. CVSS v4 vectors are not scored, because the v3.1 formula does not apply to them. If no v3 vector is present, the severity label that the originating advisory database attaches to the record (`database_specific.severity`, as published by GHSA) is used, with GHSA's MODERATE mapped to MEDIUM. If neither is available, the record's severity is UNKNOWN; it is never silently treated as LOW.

The pipeline is invoked automatically through the `useVulnerabilityFetcher` React hook whenever an SBOM is uploaded to the library. The hook follows a fire-and-forget pattern: it is asynchronous, no error is ever shown to the analyst, and the graph view is never blocked waiting for vulnerability data to arrive. Records are injected into `VulnStore` sequentially rather than in parallel: submitting all records in parallel would cause duplicate POSTs to race the backend deduplication check before it has processed the first record. When an upload contains pairs already in the store — because another SBOM shares the component, or because the same SBOM is uploaded again after being deleted — the backend rejects each with HTTP 409, and the frontend hook treats 409 as a no-op, so the analyst observes no duplication.

5.2 Deduplication and Storage

The same vulnerability can reach the store more than once: the fetch runs for every SBOM uploaded, and SBOMs in one library often share components; and an analyst can also enter records manually. `VulnStore` is the in-memory store that absorbs these records and ensures that each unique vulnerability-component pair is stored exactly once. This section describes the storage schema and the deduplication logic.

5.2.1 VulnStore Design

`VulnStore` is implemented as a Python class in `backend/store/vuln_store.py`. It maintains a single in-memory dictionary, `self.vulns`, keyed by UUID strings generated at record creation time with `str(uuid.uuid4())`. State is ephemeral: the dictionary is cleared on server restart, consistent with the ephemeral design of `SBOMStore` described in Section 4.3.3 and appropriate for a research prototype evaluated on a fixed dataset.

Each record in `self.vulns` conforms to a consistent schema:

- **id** — UUID string uniquely identifying the record.
- **purl** — Package URL of the affected component, matching the node attribute described in Section 4.3.1.
- **cveId** — CVE identifier or OSV identifier if no CVE alias was resolved.
- **severity** — severity band (CRITICAL, HIGH, MEDIUM, LOW, or UNKNOWN) derived as described in Section 5.1.2.
- **status** — current remediation status; one of `NOT_STARTED`, `IN_PROGRESS`, or `REMEDIATED`.

- **patchAvailable** — boolean, set by the analyst when entering a record manually, indicating that a patched version is known to exist; records from the OSV fetch always carry `false`.
- **notes** — free-text analyst annotation.
- **source** — origin of the record: `"osv"` for the automatic fetch or `"Manual"` for analyst entry.
- **publishedDate** — date the advisory was first published.
- **lastUpdated** — server-side timestamp refreshed on each `PUT` update.
- **timeline** — list of status-transition objects, each carrying a timestamp, the new status value and a note, recording the remediation history of the record.

The `source` field preserves the provenance of each record, allowing an analyst to distinguish vulnerabilities found by the automatic OSV scan from those entered manually.

5.2.2 Deduplication Logic

Deduplication is enforced in the `VulnStore.add()` method before any new record is committed to the store. The dedup key is the pair of `cveId` and `purl`: the method searches the existing records for any entry where both fields match the incoming record.

```
duplicate = next((v for v in self.vulns.values()
    if v.get("cveId") == cve_id and v.get("purl") == purl), None)
```

If a matching record is found, `add()` returns `None` without modifying the store. The REST endpoint wrapping this method responds with HTTP 409 Conflict. The `useVulnerabilityFetcher` hook treats a 409 response as a silent no-op: no error is raised, no duplicate appears in the store, and the analyst sees no change.

OSV IDs that could not be resolved to a CVE alias retain their OSV identifier as the `cveId` field; the deduplication logic operates on that OSV ID equally well, preventing duplicates when the same OSV entry is fetched again for another SBOM. Because the key is the first CVE alias, two distinct advisories that share that alias for the same component are also collapsed into one record, and the advisory stored second is discarded.

The first-written record for a pair is kept: a later record for the same pair, for example a manual entry with a different severity, is rejected rather than merged.

5.3 CVSS v3.1 Base Score Computation

Vulnerability records from the OSV pipeline carry CVSS vector strings rather than precomputed numeric scores. SBOM Lens computes base scores from these vectors entirely on the client side, with no dependency on an external CVSS scoring service. This section describes how the CVSS v3.1 formula is applied, illustrates the computation with the Log4Shell vector, and explains the design of the custom JavaScript implementation.

5.3.1 Formula and Metric Groups

The CVSS v3.1 base score formula and the numerical values for each metric are fully defined in Section 2.5.4; this section does not repeat that derivation, but states explicitly the severity band thresholds that SBOM Lens uses to map numeric scores to visual encodings.

SBOM Lens maps CVSS base scores to four severity bands, plus UNKNOWN for records without any usable severity (Section 5.1.2):

Severity	Score Range
CRITICAL	≥ 9.0
HIGH	7.0–8.9
MEDIUM	4.0–6.9
LOW	< 4.0

These thresholds follow the NVD/NIST severity rating scale. FIRST.org's published five-band scheme adds a "None" band for a base score of exactly 0.0; SBOM Lens collapses this band into LOW to simplify the overlay colour mapping to four distinct visual classes. When the input to `parseCVSSSeverity()` is missing or cannot be scored, the function returns UNKNOWN rather than defaulting to LOW, so that missing severity data is not mistaken for low risk. On the graph, UNKNOWN nodes are drawn with the LOW colour; in the data, and in the evaluation of Chapter 7, they remain a separate band.

The scope-sensitivity of the Privileges Required (PR) metric is handled explicitly: when Scope is Changed (`S:C`), the PR Low coefficient is 0.68 rather than 0.62, and the PR High coefficient is 0.50 rather than 0.27; PR None is 0.85 in both cases. The custom implementation branches on the parsed Scope value when looking up PR coefficients, matching the FIRST.org specification precisely.

5.3.2 Worked Example: Log4Shell

Log4Shell (`CVE-2021-44228`) was introduced as the principal illustrative case study in Section 2.5.4 and analysed from a supply-chain perspective in Section 3.3.2. Chapter 5 applies the same vulnerability to the implementation: given its CVSS vector, the client-side scoring function must produce the expected base score.

The Log4Shell vector is:

```
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
```

Substituting the numerical values specified in Section 2.5.4 gives: $AV = 0.85$, $AC = 0.77$, $PR = 0.85$ (Scope Changed branch), $UI = 0.85$, $C = 0.56$, $I = 0.56$, $A = 0.56$. The Impact Sub-Score (ISS) is:

$$ISS = 1 - (1 - 0.56)(1 - 0.56)(1 - 0.56) = 1 - 0.44^3 = 1 - 0.0852 \approx 0.9148$$

Because Scope is Changed, the Impact term uses the modified formula:

$$\begin{aligned} \mathit{Impact} &= 7.52 \times (\mathit{ISS} - 0.029) - 3.25 \times (\mathit{ISS} - 0.02)^{15} \\ &\approx 7.52 \times 0.8858 - 3.25 \times 0.8948^{15} \\ &\approx 6.661 - 0.614 \approx 6.048 \end{aligned}$$

The Exploitability term is:

$$\mathit{Exploitability} = 8.22 \times 0.85 \times 0.77 \times 0.85 \times 0.85 \approx 3.887$$

The base score is therefore:

$$\begin{aligned} \mathit{Base} &= \min\big(1.08 \times (6.048 + 3.887),\ 10\big) \\ &= \min(1.08 \times 9.935,\ 10) \\ &= \min(10.73,\ 10) = 10.0 \end{aligned}$$

The CVSS roundup rule produces a final score of 10.0, which falls in the CRITICAL band (≥ 9.0). In the graph overlay described in Section 5.5.1, a Log4Shell-affected component node is therefore assigned the `.vuln-critical` CSS class and rendered in red.

5.3.3 Custom Client-Side Implementation

The CVSS scoring logic is implemented entirely in `vulnerabilityAPI.js` with no external CVSS library dependency. This is a deliberate design choice: adding a CVSS computation library to the frontend `package.json` would introduce a runtime dependency whose licensing, maintenance status, and specification compliance would need independent verification. Implementing the formula from the FIRST.org specification in approximately seventy lines of JavaScript keeps the frontend self-contained and makes the scoring logic directly auditable alongside the rest of the source code.

The function `cvssV3BaseScore(vector)` accepts a CVSS vector string, splits it on the `"/"` separator to extract metric abbreviation–value pairs, and looks each pair up in a coefficient table to produce the numeric values used in the formulas above. It branches on the parsed Scope value to select the correct Impact and PR coefficients, computes the ISS, Impact, Exploitability, and Base sub-terms in sequence, and applies the CVSS roundup rule (`Math.ceil(base * 10) / 10`) before returning the final score. This roundup rule is specified by FIRST.org as "the smallest number, specified to one decimal place, that is equal to or higher than its input"; the expression `Math.ceil(x * 10) / 10` implements it exactly.

The entry point for callers is `parseCVSSSeverity(scoreString)`, which accepts either a numeric score string (e.g. `"9.8"`) or a CVSS v3 vector string (as returned by OSV). If the input does not parse as a number and begins with `"CVSS:"`, it is treated as

a vector and passed to `cvssV3BaseScore()`. This single entry point means the rest of the codebase does not need to distinguish numeric scores from vectors.

5.4 Vulnerability Management Workflow

Beyond recording which components carry known CVEs, SBOM Lens provides a lightweight remediation-tracking workflow. Analysts can update the status of each vulnerability record, add notes, and observe how the record's status has changed over time. This section describes the automated fetch that populates the store on SBOM upload and the REST API that exposes the status lifecycle.

5.4.1 Automated Fetch on SBOM Upload

Vulnerability fetching is triggered automatically when an SBOM is uploaded to the library, without requiring any analyst action. The `useVulnerabilityFetcher` React hook receives the `components[]` array from the uploaded SBOM and passes the PURL list to the OSV pipeline described in Section 5.1.2.

The hook follows the fire-and-forget pattern introduced in Section 5.1.2: it is asynchronous, does not block rendering, and never surfaces an error to the analyst: a failed OSV request yields an empty result, and a backend error while storing records is logged with `console.warn` and abandons the remaining records of that SBOM. A network failure therefore never prevents the analyst from viewing the dependency graph, but an empty vulnerability list cannot be told apart from a failed lookup. Records are submitted to the backend's `POST /vulnerabilities` endpoint one at a time rather than in a batch request; this sequential submission prevents a parallel flood of identical records from racing the deduplication check on the server side.

Because records are keyed by CVE ID and PURL rather than by SBOM, an upload that shares components with an SBOM already in the library resubmits pairs that already exist in `VulnStore`; so does an SBOM uploaded again after being deleted, since deleting an SBOM does not delete its vulnerability records. Each such pair receives a 409 Conflict response, which the hook treats as a no-op. Vulnerabilities are fetched once, at upload: selecting an SBOM or reloading the page does not query OSV again, so an advisory published later reaches an SBOM only if it is uploaded again.

5.4.2 Status Lifecycle and REST API

`VulnStore` exposes a full CRUD REST API through the `vulnerabilities` FastAPI router. The endpoints are:

- `GET /vulnerabilities` — returns all records in the store as a JSON array; no SBOM-scoping filter is applied, making the store a global view of all known vulnerabilities across all loaded SBOMs.

- `POST /vulnerabilities` — creates a new record; returns HTTP 201 on success or HTTP 409 if the CVE ID + PURL pair already exists.
- `PUT /vulnerabilities/{id}` — partial update of an existing record: every field sent replaces the stored value; the interface uses it to change `status` and extend `timeline`. The `lastUpdated` timestamp is set by the server.
- `DELETE /vulnerabilities/{id}` — removes a single record.
- `DELETE /vulnerabilities` — clears the entire store; useful for resetting state between evaluation runs without restarting the server.

The `status` field supports a three-state remediation lifecycle. `NOT_STARTED` is the initial state assigned to every newly created record. `IN_PROGRESS` indicates that a remediation effort is under way. `REMEDIATED` is the terminal state, indicating that the vulnerability has been patched or mitigated. The backend stores the field without validating it, so triage states such as a deliberate decision not to fix, or a false positive, could be added to the interface without changing the API; the evaluated version does not offer them. The first `timeline` entry is written by the server when the record is created; each later status change made through the interface appends an entry with a client-side timestamp and sends the extended list back. The server does not enforce this, since `PUT` replaces any field it receives, so the timeline is a remediation history for a single-user prototype rather than a tamper-proof audit trail. Figure 5.1 shows the Vulnerabilities page, where the analyst reviews the stored records and changes their status.

5.5 Graph Overlay Visualisation

The vulnerability data held in `VulnStore` is projected back onto the Cytoscape.js graph through three distinct overlay types. Each overlay assigns one of the CSS classes predefined in Section 4.4.1 — `.vuln-critical`, `.vuln-high`, `.vuln-medium`, `.vuln-low`, `.blast-radius`, and `.dominator-tree` — to the appropriate nodes, giving those classes the analytical meaning that Chapter 4 deferred to this chapter. All three overlays are activated interactively by the analyst: the severity colour-coding overlay from the Analysis menu, and the blast-radius and dominator-tree overlays from the same menu, for the node chosen with *Analyze Impact* in its context menu.

5.5.1 Severity Colour Coding

The severity colour-coding overlay is activated from the Analysis menu once vulnerability records for the loaded SBOM are present in `VulnStore`. When it is active, each graph node is assigned a CSS overlay class determined by the highest-severity CVE that affects the component identified by that node's PURL.

A component may carry multiple CVE records in `VulnStore`; in that case the highest severity wins. A node whose PURL is associated with one CRITICAL and three MEDIUM vulnerabilities is displayed as CRITICAL — the most dangerous exposure takes visual

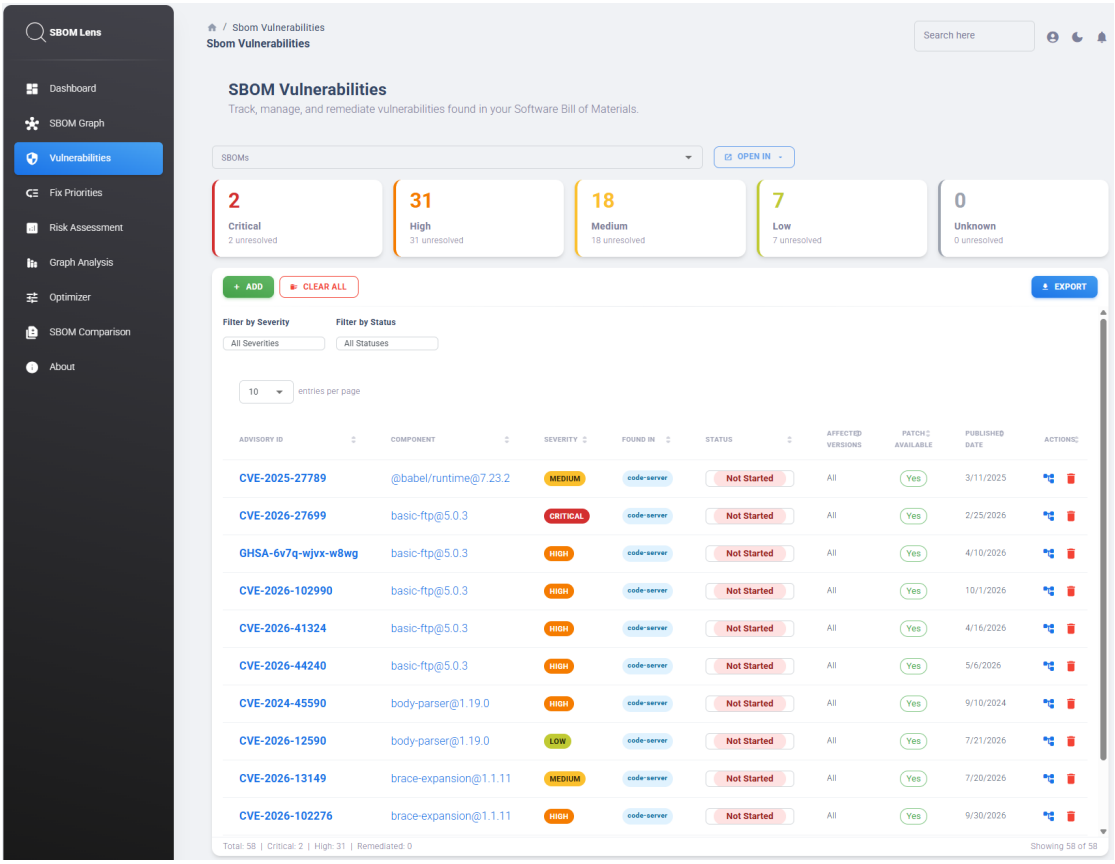


Figure 5.1: *The Vulnerabilities page for the* `code-server` *SBOM: totals by severity band, and the stored vulnerability records with their component, severity band and remediation status.*

precedence. The Cytoscape overlay is applied programmatically: for each node, the vulnerability records whose `purl` field matches the node's PURL are retrieved from the store, the maximum of their severity bands (assigned by `parseCVSSSeverity()` when the records were fetched) is taken, and the node is assigned the matching class (`.vuln-critical`, `.vuln-high`, `.vuln-medium`, or `.vuln-low`). Nodes whose PURL matches no stored vulnerability record receive no severity class and keep their default colour. Figure 4.4 in Chapter 4 shows the overlay applied to the `code-server` SBOM, with `basic-ftp`, its CRITICAL component, selected.

5.5.2 Blast Radius Analysis

The blast-radius overlay is an interactive analysis triggered when the analyst chooses a node with *Analyze Impact* in its context menu and switches the overlay on in the Analysis menu, as described in Section 4.4.3. The query is issued to `GET /{sbom_id}/impact?node_id={id}`, which returns the blast radius, dependency paths, dominated nodes, and the HITS hub and authority scores of the queried node in a single response.

On the backend, the blast radius is computed as `list(nx.ancestors(G, node_id)) + [node_id]`. The semantic interpretation follows directly from the edge direction established in Section 4.3.2: an edge from A to B means A depends on B, so `nx.ancestors(G, B)` returns all nodes from which there exists a directed path to B — the upstream consumers of B whose security posture is degraded if B is compromised; the blast radius is B together with these consumers. This framing is intentional and distinct from a "downstream" interpretation: in the SBOM Lens graph, the upstream consumers are the packages that pull in the vulnerable component, and so the packages that may be affected if the vulnerable node is compromised. Because the analysis is at component level, this over-approximates exposure: a consumer may never execute the vulnerable code (Section 3.2.3).

The API response also includes up to five shortest paths from the root packages (components with in-degree zero) to the vulnerable node, computed with `nx.all_shortest_paths` from each root. These paths provide concrete dependency chains — a sequence of "package X uses package Y which uses the vulnerable package Z" — that an analyst can use to understand exactly how the vulnerable component entered the dependency tree. The interface lists them in the *Dependency paths* section of the Details panel, together with the total number of paths from the root to the node and the number of consumers they pass through; selecting a path highlights it on the graph, and *Highlight all paths* highlights every edge of the blast radius.

On the frontend, blast-radius nodes receive the `.blast-radius` CSS class (purple, as pre-defined in Section 4.4.1). Figure 5.2 shows the overlay for `basic-ftp` 5.0.3, the component with a CRITICAL advisory (CVE-2026-27699) in the `code-server` SBOM.

The impact API response also includes `hitsHub` and `hitsAuthority` scores of the analysed component; scores for every node are returned by `GET /sboms/{id}/global-analysis`. These scores quantify the component's structural centrality in the dependency network

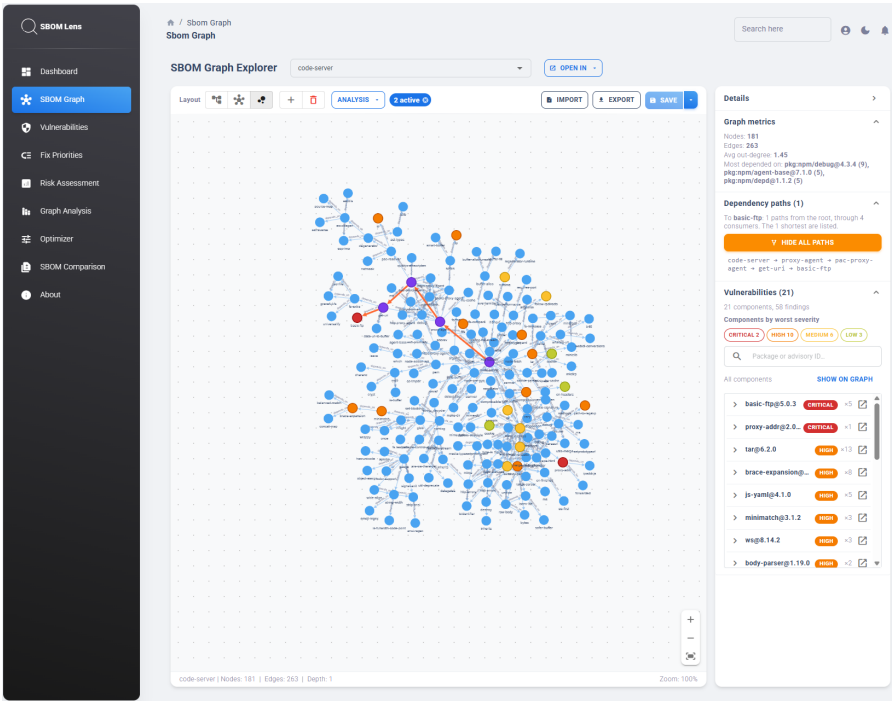


Figure 5.2: *Blast radius of `basic-ftp` in the `code-server` SBOM. Its four consumers — `get-uri`, `pac-proxy-agent`, `proxy-agent` and the application `code-server` — are purple, `basic-ftp` keeps the red of its* CRITICAL *band, and the edges of the single dependency path are highlighted in orange. The Details panel lists the path, `code-server` → `proxy-agent` → `pac-proxy-agent` → `get-uri` → `basic-ftp`.*

and are relevant to prioritising remediation effort; however, the HITS algorithm that produces them is described in detail in Chapter 6 (specifically Section 6.1.2) and is not explained further here.

5.5.3 Dominator Tree Analysis

The dominator-tree overlay is returned in the same `GET /{sbom_id}/impact` response as the blast radius, adding structural risk information without requiring an additional round-trip.

On the backend, dominance is computed from the root of the dependency graph, not from the vulnerable node. `nx.immediate_dominators(G, root)` returns, for every component reachable from the root, its immediate dominator; when the SBOM has several root components, they are first joined under a virtual root so that a single dominator tree exists. The dominated set of the vulnerable node is then its entire subtree in that dominator tree: every component for which each path from the root passes through the vulnerable node. In supply-chain terms, a node with a non-empty dominated set is a structural chokepoint: if it were removed or replaced, all the dominated components would lose their connection to the application with no alternative route.

Dominated nodes are assigned the `.dominator-tree` CSS class on the frontend, displayed in cyan as pre-defined in Section 4.4.1. When both overlays are switched on for the same node, the analyst can read them together: nodes highlighted in purple

are the upstream consumers affected by the vulnerability's propagation, while nodes highlighted in cyan are structurally dependent on the vulnerable node and would be isolated if it were removed. This dual encoding gives both a security-propagation view and an architectural-risk view in a single inspection, providing complementary information that neither overlay conveys alone. In the evaluated version, choosing a new node while both overlays are active refreshes only the blast radius, so the dominator-tree overlay must be switched off and on again to follow the new node. Figure 5.3 shows the dominated set of `express` in the `code-server` SBOM.

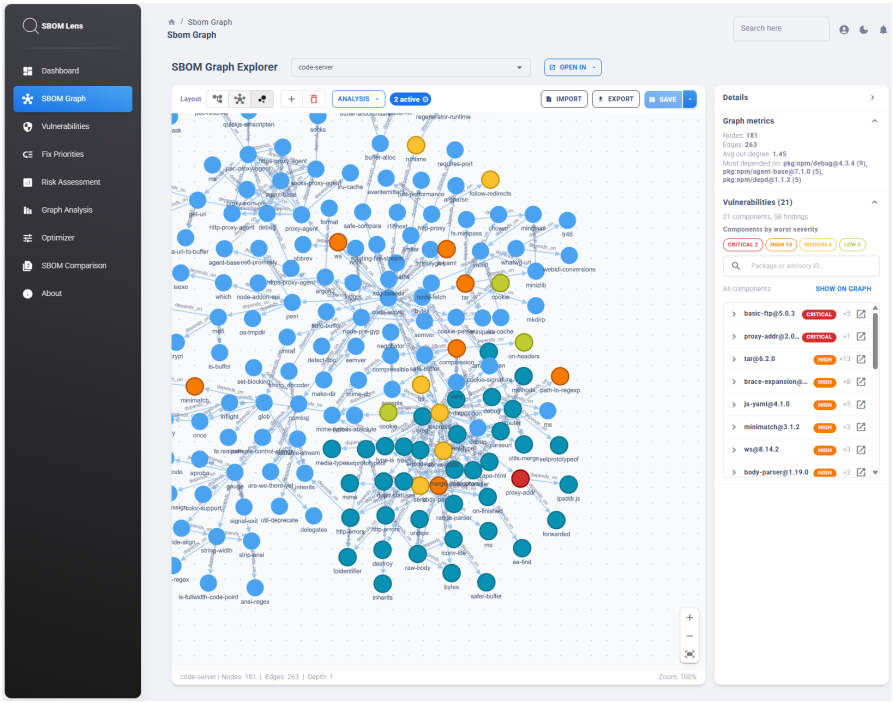


Figure 5.3: *Dominated set of `express` in the `code-server` SBOM (cyan): the components reachable from the root only through `express`, which would lose their connection to the application if it were removed. The severity colours of the vulnerable components are also shown.*

6

Structural Risk and Graph Analysis

Chapter 5 attached known vulnerabilities to the dependency graph and measured, for one vulnerable component at a time, who inherits the vulnerability (its blast radius) and who could not route around it (its dominated set). This chapter turns to risk that does not depend on any particular vulnerability being known. Its central claim is that *where a component sits in the dependency graph carries risk information that flat-inventory analysis cannot express*. A list of components, versions and CVEs says nothing about which component the rest of the application cannot do without, which components form tightly coupled clusters, or how readily a compromise could travel through the graph as a whole. SBOM Lens makes this *structural risk* visible through a set of graph measures, organised in this chapter by the scale at which they describe the graph.

Section 6.1 covers the *component scale*: measures that assign a value to each individual component or dependency edge. Section 6.2 covers the *cluster scale*: measures that describe groups of components — communities, the bow-tie decomposition, and the rich club. Section 6.3 covers the *whole-graph scale*: single numbers or distributions that characterise the dependency graph of an entire project. Section 6.4 summarises every measure with the graph it is computed on and what it adds beyond a flat inventory. The theoretical definitions of the underlying concepts were given in Section 2.3 and Section 2.4; this chapter states how SBOM Lens applies each of them, what exactly it computes, and how the result should be read in supply-chain terms. The values these measures take on real projects are reported in Chapter 7.

Three conventions hold throughout. First, edges point from consumer to dependency (Section 4.3.2): an edge $u \to v$ means that u depends on v, so the in-degree $\deg^-(v)$ counts the direct consumers of v and the out-degree $\deg^+(v)$ counts its direct dependencies. Second, several measures are defined only for undirected graphs, or are uninformative on directed ones; for these, SBOM Lens uses the *undirected projection* G_u of the dependency graph G, in which u and v are adjacent if either depends on the

other. Each section states which of the two graphs a measure uses, because the choice changes what the result means. Third, all measures are computed by the backend on the NetworkX graph (Hagberg et al., 2008) cached at ingestion, with NumPy and SciPy for the eigenpairs and the `powerlaw` package for the degree fit, and are exposed through the REST endpoints listed in Section 6.4; the whole-graph endpoints cache their result per SBOM until the SBOM is edited or deleted, while the impact, global-analysis and metrics endpoints recompute it on every call.

The spectral measures illustrate why the undirected projection is sometimes unavoidable. A dependency graph is acyclic or nearly so (Section 2.3.2), and the adjacency matrix of a directed acyclic graph is nilpotent: every one of its eigenvalues is zero. A spectral radius computed on G itself would be zero for every acyclic SBOM and, in the others, would be set only by their few cycles. On G_u the adjacency matrix is symmetric, its eigenvalues are real, and its largest eigenvalue and the corresponding eigenvector are well defined and non-trivial. The cost is that direction is lost: a measure computed on G_u treats "depends on" and "is depended on by" alike. The chapter returns to this trade-off wherever it matters.

6.1 Component Scale

Component-scale measures answer the question an analyst asks when looking at a single node: how important is this component to the structure of the application, independently of whether it currently has a known vulnerability? SBOM Lens provides five such measures, each reflecting a different notion of importance.

6.1.1 Articulation Points and Bridges

An articulation point is a component whose removal disconnects the graph (Section 2.3.4), and a bridge is a dependency edge whose removal does the same. SBOM Lens computes both on the undirected projection: articulation points with `nx.articulation_points(`G_u`)`, returned by `GET /sboms/{id}/global-analysis`, and bridges with `nx.bridges(`G_u`)`, returned by `GET /sboms/{id}/optimizer`. NetworkX defines neither for directed graphs, and both run in $O(|V| + |E|)$ time.

In supply-chain terms, an articulation point is a component through which two otherwise separate parts of the dependency graph are joined, and a bridge is a single dependency relationship that is the only link between two parts. Such components and edges are natural points of fragility: a compromise, a deprecation or a licence change at an articulation point touches both sides of the split.

The undirected projection, however, means that these are not the same as the *chokepoints* defined in Section 5.5.3. A chokepoint is a component that the application cannot route around *from the root*: its dominated set is non-empty. When every component is reachable from a single root, as in all the evaluated SBOMs, every articulation point is also a chokepoint, since the part of the graph it cuts off from the root can be reached

only through it. The converse fails: a component that dominates a subtree need not disconnect the undirected graph if some unrelated edge happens to link that subtree back to the rest. The two notions answer different questions. Dominance asks which components the application reaches only through a given component; articulation asks which components hold the undirected structure together. SBOM Lens reports both and Chapter 7 compares them on the vulnerable components.

6.1.2 HITS Hub and Authority Scores

The HITS algorithm (Section 2.4.3) assigns each node two mutually reinforcing scores (Kleinberg, 1999). SBOM Lens computes them on the *directed* graph with `nx.hits(G, max_iter=200)`. In the NetworkX version used for the evaluation (3.7), this runs power iteration: starting from uniform hub scores, it alternates $\mathbf{a} \leftarrow \mathbf{A}^\top\mathbf{h}$ and $\mathbf{h} \leftarrow \mathbf{A}\mathbf{a}$, rescaling $\mathbf{h}$ at each step, until $\mathbf{h}$ changes by less than 10^{-8} in total. The limit is the leading singular-vector pair of the adjacency matrix $\mathbf{A}$ — the authority vector is the principal eigenvector of $\mathbf{A}^\top\mathbf{A}$ — and both vectors are normalised to sum to one over the graph. Earlier NetworkX versions computed the same vectors directly, by a sparse singular value decomposition. Scores for all nodes are returned by `GET /sboms/{id}/global-analysis`; the impact endpoint of Section 5.5.2 also returns the two scores of the analysed component. Power iteration converges slowly when the two largest singular values of $\mathbf{A}$ are close; if it has not converged after 200 iterations, SBOM Lens reports every score as zero.

Under consumer-to-dependency edges, a high *authority* score belongs to a component that is depended upon by components which themselves depend on many important libraries: a foundational library at the heart of the dependency graph. A high *hub* score belongs to a component that depends on many such authorities: typically the application itself or an aggregating framework package. Authority is the more useful of the two for risk: it weights a consumer's vote by that consumer's own importance rather than merely counting consumers as in-degree does. On dependency graphs, however, it does not order every library. Two components are linked in $\mathbf{A}^\top\mathbf{A}$ only when they share a direct consumer, and the authority vector is concentrated on the most strongly linked group of such components; in the evaluation, about half of the components of the median graph, and half of its vulnerable components, have authority zero. Authority therefore ranks the components of that group and ties the rest at zero.

Because the scores sum to one over the whole graph, they are shares rather than absolute values, and they shrink as the graph grows; they are therefore used to rank components within one SBOM, not to compare components across SBOMs of different sizes.

6.1.3 Spectral Exposure Score

The *spectral exposure score* of a component is its entry in the leading eigenvector of the adjacency matrix of G_u — its eigenvector centrality (Section 2.4.3). Let λ_1 be the largest eigenvalue of that symmetric matrix and $\mathbf{v}_1$ the corresponding eigenvector. By the Perron–Frobenius theorem the entries of $\mathbf{v}_1$ can be taken non-negative on a connected graph. SBOM Lens reports, for each component i,

$$s_i = \lambda_1 \cdot \frac{v_{1,i}}{\max_j v_{1,j}},$$

through `GET /sboms/{id}/epidemic`. The ranking of components by s_i is exactly their ranking by eigenvector centrality; the scaling places the most exposed component at λ_1, so that the scores share a unit with the spectral radius of Section 6.3.1. For graphs of up to 500 components the eigenpair is computed densely with `numpy.linalg.eigh`; for larger graphs, with the sparse solver `scipy.sparse.linalg.eigsh`, asking for the largest *algebraic* eigenvalue. The distinction matters: a tree is bipartite, so its spectrum is symmetric and $-\lambda_1$ has the same magnitude as λ_1; a solver asked for the eigenvalue of largest magnitude may return either.

The score reflects a component's position with respect to the whole graph rather than its immediate neighbourhood: a component scores highly if it is adjacent to components that themselves score highly. In the dependency graph this identifies the densely interlinked region through which a compromise would most readily travel, which is why the same eigenvector underlies the spread-potential measures of Section 6.3.1. The API field carrying s_i is named `r0` for historical reasons; it is not a basic reproduction number, and SBOM Lens runs no epidemic simulation (Section 2.4.6).

Two caveats apply. The score is undirected, so it does not distinguish a component that many others depend on from one that depends on many others. And if G_u is disconnected, the leading eigenvector is concentrated on the component of the graph with the largest spectral radius, and every node elsewhere scores zero; the score then ranks only the dominant connected part of the graph.

6.1.4 Core Number

The core number of a component is the largest k for which it belongs to the k-core of the graph (Section 2.4.5). SBOM Lens computes it with `nx.core_number(`G_u`)`, the Batagelj–Zaversnik algorithm running in $O(|E|)$ time (Batagelj et al., 2003), and returns every component's core number, the maximum core number $k_{\max}$, and the components of the innermost core through `GET /sboms/{id}/kcore`. Self-referencing dependency entries, which NetworkX's implementation rejects, are dropped when the graph is built (Section 4.2.3).

A high core number marks a component embedded in a densely interconnected region of the dependency graph: it and its neighbours are linked to one another many times over. Such components are hard to replace, because removing or substituting one

affects many others that are themselves tightly linked. As with the other undirected measures, the core number says nothing about direction; a high value means dense interconnection, not depth in the dependency chain.

6.1.5 Edge Rankings and Abstraction Candidates

The optimiser endpoint (`GET /sboms/{id}/optimizer`) applies the spectral exposure score to two practical questions. First, which single dependency edge contributes most to the spread potential of the graph? Removing edge $\{u, v\}$ from a symmetric adjacency matrix lowers λ_1 by approximately $2\, v_{1,u}\, v_{1,v}$ for a unit-norm $\mathbf{v}_1$, a first-order perturbation estimate used in the literature on reducing the spectral radius by link removal (Van Mieghem et al., 2011). SBOM Lens ranks every edge of G_u by this estimate and returns the top twenty, each with its estimated reduction as a percentage of λ_1. Second, which components are candidates for an abstraction layer — an internal wrapper that isolates the rest of the application from a widely used library? SBOM Lens takes every component with at least three direct consumers ($\deg^-(v) \geq 3$) and orders them by spectral exposure score, then by the number of consumers, returning the top twenty. Both outputs are heuristic aids to the analyst rather than measures in their own right: the edge estimate is a linear approximation, not a recomputed eigenvalue, and the candidate list has no threshold on the spectral score. They are included here because they show how the component-scale scores can be turned into remediation suggestions; they are not part of the evaluation in Chapter 7.

6.2 Cluster Scale

Cluster-scale measures describe groups of components. They answer questions that neither a single component nor the whole graph can: which parts of the application form coherent subsystems, whether the graph has a cyclic core, and whether its best-connected components are also connected to one another.

6.2.1 Louvain Communities

SBOM Lens partitions the dependency graph into communities with the Louvain algorithm (Section 2.4.4) (Blondel et al., 2008), calling `louvain_communities(`G_u`, seed=42)` with NetworkX's defaults of unit edge weights and resolution one. Louvain visits nodes in a random order, and the fixed seed makes the partition reproducible for a given graph and library version. `GET /sboms/{id}/communities` returns the number of communities, the community of every component, and the size of every community; the frontend colours the graph by community, as Figure 6.1 shows for the `code-server` SBOM.

In an SBOM, a community is a group of components that depend on one another more than on the rest of the graph: typically a framework and its plugins, or a library

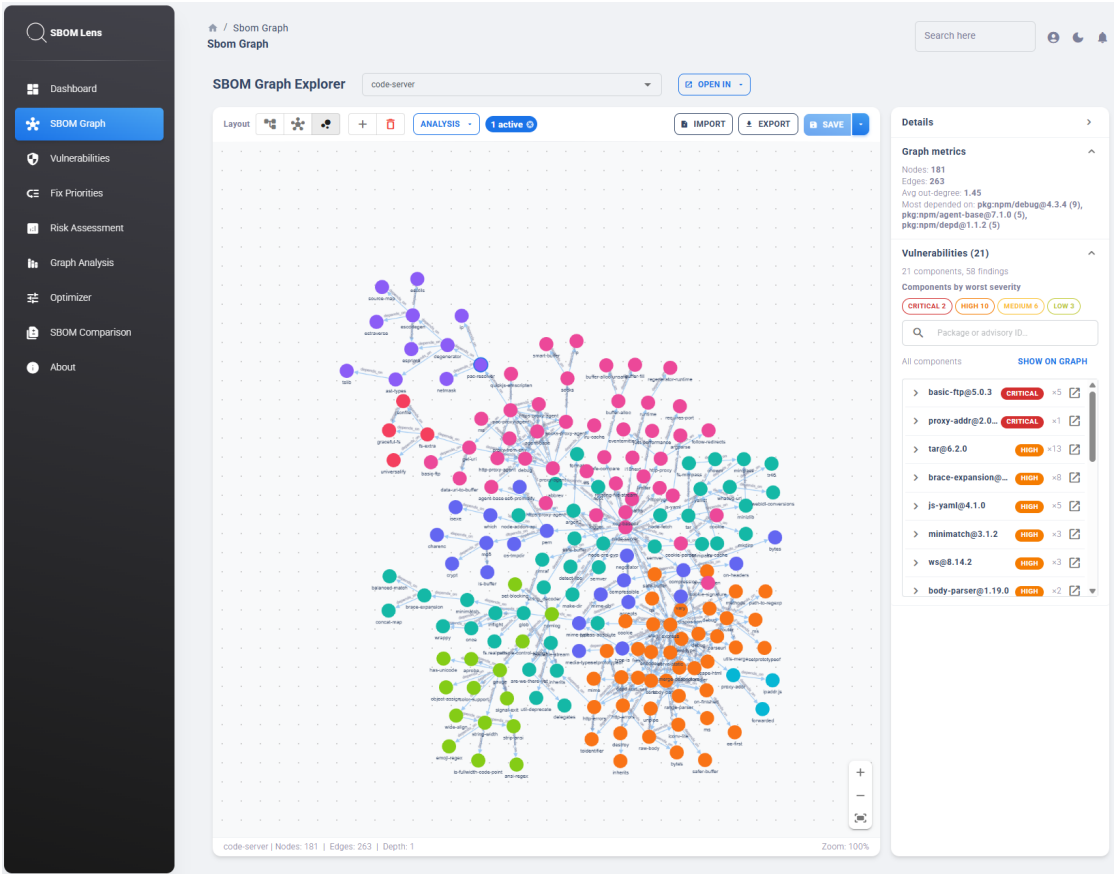


Figure 6.1: *Louvain communities of the* `code-server` *dependency graph, one colour per community.*

together with its private helper packages. For risk, the partition suggests where a vulnerability may stay contained. Because a vulnerability reaches only consumers, what matters is how many components outside the community depend on it: a vulnerable component in a community on which few outside components depend affects mainly that community; a vulnerable component on the boundary between communities links subsystems that would otherwise be isolated. The modularity score of the partition is not reported by the endpoint, and communities are computed on the undirected projection.

6.2.2 Bow-Tie Decomposition

The bow-tie decomposition, introduced for the graph of the Web (Broder et al., 2000), arranges a directed graph around its largest strongly connected component (SCC): the *core*. The IN set contains the nodes that can reach the core, the OUT set the nodes reachable from it, and the remaining nodes — tendrils, tubes and disconnected parts — are grouped together. SBOM Lens computes the SCCs of the directed graph with `nx.strongly_connected_components`, takes the largest as the core, and derives IN and OUT by directed reachability; `GET /sboms/{id}/graph-theory` returns the size and share of each set together with the number of non-trivial SCCs. Under consumer-to-dependency edges, IN holds the consumers of the core and OUT its dependencies.

The decomposition is meaningful only when the graph has a cyclic core. In an acyclic dependency graph every SCC contains a single component, and the "largest" one is an arbitrary node. SBOM Lens therefore reports the IN, OUT and remainder sets only when the core contains more than one component, and otherwise reports just that no cyclic core exists. When a core does exist, it identifies a set of mutually dependent components that must be installed, updated and trusted together, and the relative sizes of IN and OUT show how much of the application sits upstream and downstream of that entanglement.

6.2.3 Rich-Club Coefficient

The rich-club coefficient measures whether the best-connected components of a graph are also connected to one another (Colizza et al., 2006). For each degree k, let N_k be the number of nodes with degree greater than k and E_k the number of edges among them; the coefficient is

$$\phi(k) = \frac{2E_k}{N_k(N_k - 1)},$$

the density of the subgraph formed by those nodes. SBOM Lens computes $\phi(k)$ on G_u with `nx.rich_club_coefficient` and returns the full curve, its maximum, and the smallest k at which $\phi(k)$ exceeds 0.5, through `GET /sboms/{id}/graph-theory`.

A rising curve means that the hubs of the dependency graph form a densely interlinked club: widely used libraries that also depend on one another, so that a compromise in one is close to all of them. The coefficient is reported unnormalised. Its

normalised form divides $\phi(k)$ by the value expected in a random graph with the same degree sequence, and only that ratio shows whether the club is denser than chance (Colizza et al., 2006). The unnormalised curve also rises naturally with k as the high-degree subgraph shrinks, so it is read here as a description of the hubs' interconnection, not as evidence of a rich-club effect. The 0.5 cut-off is a presentation threshold without theoretical basis. On SBOM graphs the maximum and the cut-off are dominated by the last points of the curve: in 39 of the 44 evaluation graphs whose maximum is 1, that value comes from the two highest-degree components alone being adjacent. Both therefore say whether the top two or three hubs are linked, rather than whether a club exists.

6.3 Whole-Graph Scale

Whole-graph measures summarise the dependency graph of a project in a single number or distribution. They serve two purposes: they let an analyst compare the structural risk of different applications, or of different versions of the same one, and they describe the dependency graphs of the evaluation dataset in Chapter 7 (RQ1). In the interface, the measures returned by the audit are shown on the Risk Assessment page (Figure 6.2).

6.3.1 Spectral Radius and Epidemic Threshold

The spectral radius λ_1 of G_u, computed as in Section 6.1.3, summarises in one number how readily a contagion could spread over the graph. For a susceptible–infected–susceptible process on a network, in which each infected node infects each neighbour at rate β and recovers at rate δ, an infection dies out when $\beta/\delta < 1/\lambda_1$ (Wang et al., 2003); the quantity $1/\lambda_1$ is therefore called the *epidemic threshold* of the graph. A larger spectral radius means a lower threshold: the graph can sustain spreading at lower transmission rates. `GET /sboms/{id}/epidemic` returns both λ_1 and $1/\lambda_1$.

SBOM Lens uses these quantities as a static structural index, not as a prediction. Software compromise does not follow a susceptible–infected–susceptible process with homogeneous rates, and no such process is simulated (Section 2.4.6). What the spectral radius does capture is that a graph with densely interconnected hubs offers more routes for any kind of propagation than a sparse, tree-like one of the same size. For a star with n leaves $\lambda_1 = \sqrt{n}$, and for a path it stays below two, so λ_1 grows with hub concentration rather than with size alone. For the reasons given in the introduction to this chapter, it is computed on the undirected projection.

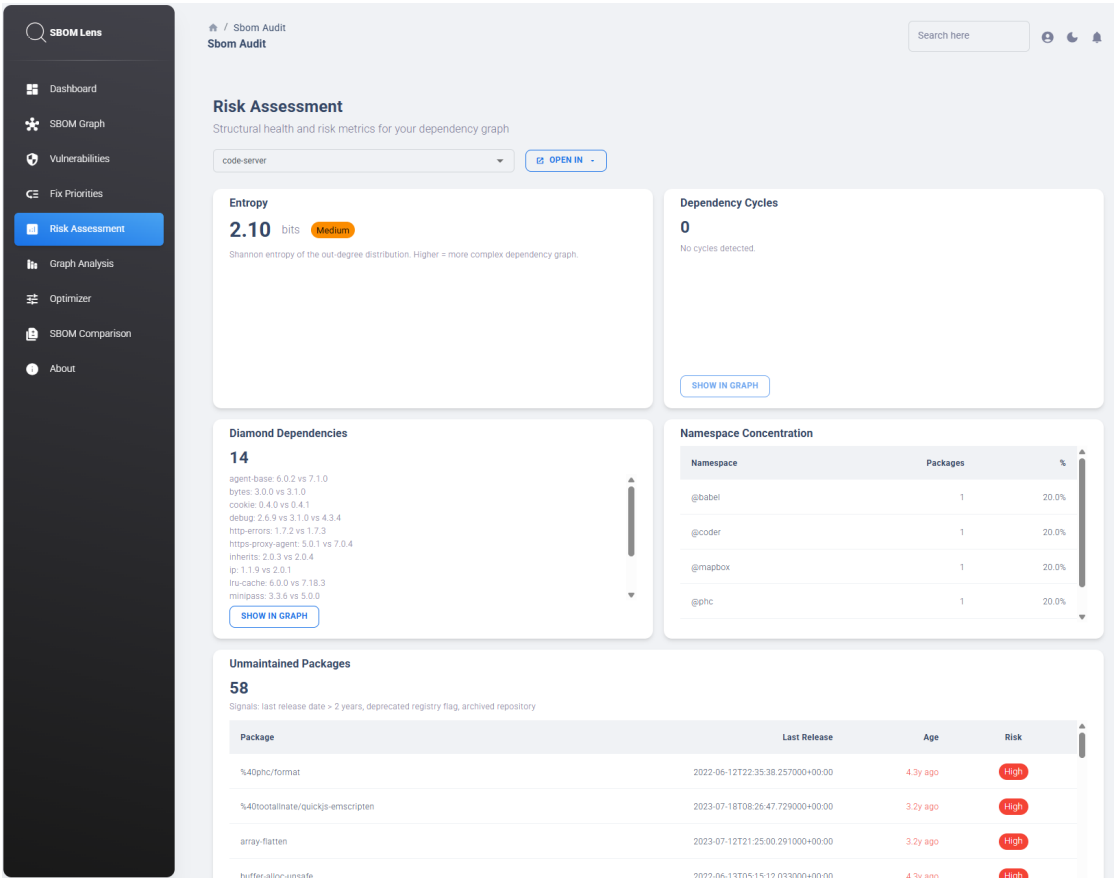


Figure 6.2: *The Risk Assessment page for the* `code-server` *SBOM, showing the whole-graph audit: degree entropy, dependency cycles, version conflicts (labelled* Diamond Dependencies *in the interface), namespace concentration, and unmaintained packages.*

6.3.2 Degree Entropy

SBOM Lens computes the Shannon entropy (Shannon, 1948) of the out-degree distribution,

$$H = -\sum_{k} p(k) \log_2 p(k),$$

where $p(k)$ is the fraction of components with exactly k direct dependencies, including components with none. `GET /sboms/{id}/audit` returns H in bits with a label: *low* below 1.5 bits, *medium* up to 3.0 bits, and *high* above.

Low entropy means that most components have a similar number of direct dependencies — typically a graph dominated by leaf packages with none and a few components with a handful. High entropy means a heterogeneous mix, from leaves to components that each pull in many dependencies. Because the edges point from consumer to dependency, this is the distribution of dependencies per component, not of consumers per component; concentration of consumers on a few popular libraries is captured instead by the in-degree distribution of Section 6.3.7. Two limitations apply: H is bounded by the logarithm of the number of distinct out-degree values, so it tends to grow with graph size, and the label thresholds are heuristic.

6.3.3 Cycles and Strongly Connected Components

A dependency cycle means that no component on it can be installed or updated independently of the others (Section 2.3.2). SBOM Lens enumerates the simple cycles of the directed graph with `nx.simple_cycles`, which implements Johnson's algorithm restricted to the SCCs (Johnson, 1975), stopping after 1,000 cycles; `GET /sboms/{id}/audit` returns the number of cycles found, the components that lie on them, and the first ten cycles. The number of non-trivial SCCs, those with more than one component, is reported with the bow-tie decomposition (Section 6.2.2). In a dependency graph built from a lockfile, cycles are rare and each one is worth an analyst's attention: it either reflects a genuine mutual dependency between packages or an artefact of how the SBOM was generated.

6.3.4 Version Conflicts

A *version conflict* is the same package appearing in the dependency graph at two or more distinct versions. It typically arises from the diamond pattern of Section 2.3.2: two components require incompatible versions of a shared dependency, and the package manager installs both. SBOM Lens groups components by their group and name, as recorded in the SBOM, and reports every package with at least two distinct non-empty versions through `GET /sboms/{id}/audit` (under the key `diamonds`, for historical reasons). The detection examines component identities only; it does not search the edges for the diamond pattern itself.

Each version conflict is a maintenance and security liability. The same package must

be patched in several places, a fix applied to one version leaves the others exposed, and vulnerability scanners report the package once per version. Versions are compared as literal strings, so two spellings of the same version would count as a conflict; SBOMs generated from lockfiles record exact resolved versions, which makes this unlikely.

6.3.5 Namespace Concentration

Namespace concentration measures how much of a dependency graph comes from a single publisher. SBOM Lens derives a namespace for each component from its Package URL for npm (the scope of a scoped package) and PyPI (the leading token of the name), and otherwise from the first two dot-separated segments of the component name, which for Maven, whose component name is the artifact identifier, rarely yields a namespace — and reports the ten largest namespaces with their share of the components that have one, through `GET /sboms/{id}/audit`. A share above 60% is flagged as a concentration risk. A high concentration means that the trustworthiness of one organisation's publishing process, and the security of its accounts, underpins a large part of the application, which is precisely the exposure exploited when a maintainer or a publishing account is compromised (Section 2.2.2). The namespace rules are heuristic, unscoped npm packages have no namespace and are excluded from the denominator, and the 60% threshold is a presentation choice.

6.3.6 Unmaintained Packages

SBOM Lens flags a component as unmaintained when the date it obtains from the package registry is more than two years before the date of the analysis, and returns the flagged components through `GET /sboms/{id}/audit`. For npm packages the date is the registry's `time.modified` field, the last change to the package's registry record, which is not necessarily its last release; for PyPI packages it is the most recent upload of any release. Components whose lookup fails are counted separately, and components from other ecosystems are not checked. The check is a registry heuristic, not a graph measure: it queries the registry once per package at analysis time, so its result for an older release depends on when the analysis is run, and it says nothing about the version actually installed.

6.3.7 Degree Distribution and Power-Law Fit

Whether the in-degree distribution follows a power law (Section 2.4.1) determines whether a few components carry most of the dependency relationships. SBOM Lens fits a discrete power law $p(k) \propto k^{-\alpha}$ for $k \geq x_{\min}$ to the in-degrees of all components with at least one consumer, using the method of Clauset et al. (2009) as implemented in the `powerlaw` package (Alstott et al., 2014). The exponent α is estimated by maximum likelihood for each candidate lower bound $x_{\min}$, and the $x_{\min}$ that minimises the Kolmogorov–Smirnov distance D between the fitted model and the data is retained.

`GET /sboms/{id}/graph-theory` returns α, its standard error σ, $x_{\min}$, D, and the number of components in the fitted tail, together with the five components of highest in-degree and the raw degree histogram. When fewer than three components have consumers, or the in-degrees take fewer than two distinct values, no fit is attempted; a fit that fails, as it can when the in-degrees take only a few distinct values, also returns no exponent. The implementation does not, however, check the package's flag for a fit in which no candidate $x_{\min}$ gives a valid estimate; with `powerlaw` 2.0.0, whose default range bounds α at 3, such a fit is returned with $\alpha = 3$ (Section 7.3).

Maximum likelihood with a fitted lower bound is used because the more common alternative — a least-squares line through the logarithmic histogram — gives biased estimates of α, and a high coefficient of determination on such a plot is not evidence of a power law (Clauset et al., 2009). Even so, a fitted α is a description of the tail, not a test: whether a power law describes the data better than an alternative such as a log-normal requires a separate likelihood-ratio comparison, which SBOM Lens does not perform. The Kolmogorov–Smirnov distance D is reported so that poor fits can be recognised. A small α with a large tail indicates heavy concentration: a few components are depended upon by a large share of the graph, and a vulnerability in one of them has an outsized blast radius.

6.3.8 Small-World Coefficient

The small-world coefficient compares a graph's clustering and path length with those of a random graph of the same size (Watts et al., 1998; Humphries et al., 2008):

$$\sigma = \frac{C/C_{\text{rand}}}{L/L_{\text{rand}}},$$

where C is the average clustering coefficient and L the average shortest path length (Section 2.4.2). SBOM Lens computes it on the largest connected component of G_u, with n nodes and m edges. C is given by `nx.average_clustering`. L is exact for components of up to 500 nodes; for larger ones it is estimated from breadth-first searches from 500 source nodes drawn with a fixed seed, so the estimate is reproducible. The random baseline uses the closed-form Erdős–Rényi approximations $C_{\text{rand}} = p$ and $L_{\text{rand}} = \ln n / \ln(np)$, with $p = 2m/(n(n-1))$. `GET /sboms/{id}/graph-theory` returns σ, C, L, both ratios, and the size and share of the largest component.

A value of σ well above one means that components form tightly clustered neighbourhoods while any two remain a few hops apart: in the undirected projection, every component is close to every other. Closeness in G_u is not exposure: a vulnerability reaches only the transitive consumers of a component, usually a small fraction of the graph. The closed-form baseline ignores the degree sequence of the graph, which a baseline built from degree-preserving randomised graphs would respect; σ is therefore a coarse indicator, reported alongside C and L themselves. Dependency graphs are close to trees, in which $C = 0$ and hence $\sigma = 0$; a non-zero σ already indicates that

some components share dependencies with their own dependencies.

6.4 Summary

Table 6.1 lists every measure described in this chapter together with the graph it is computed on and the endpoint that returns it. Read together with Chapter 5, the measures cover the three scales of the central claim. At the component scale, blast radius and dominated sets describe a specific vulnerability, while articulation points, HITS authority, spectral exposure and core number describe structural importance whether or not a vulnerability is known. At the cluster scale, communities, the bow-tie core and the rich club describe how components group together. At the whole-graph scale, the spectral radius, entropy, cycles, version conflicts, concentration, unmaintained packages and the degree distribution characterise the dependency graph as a whole.

None of these can be computed from a flat inventory. A list of components with their versions and CVEs contains no edges, so it has no degree, no reachability, no dominance, no communities and no spectrum. The only structural information a flat inventory holds is the set of package names and versions, from which version conflicts, namespace concentration and, with a registry lookup, unmaintained packages — but nothing else in the table — could be derived. Whether the structural view actually changes which components an analyst would prioritise is the question Chapter 7 answers empirically, by comparing the ranking of each project's vulnerable components by severity band with their rankings by blast radius and by HITS authority.

Table 6.1: *Structural measures computed by SBOM Lens, by scale. G: directed dependency graph; G_u: its undirected projection.*

Measure	Scale	Graph	Endpoint
Articulation points	Component	G_u	`global-analysis`
Bridges	Component	G_u	`optimizer`
HITS hub and authority	Component	G	`global-analysis`, `impact`
Spectral exposure score	Component	G_u	`epidemic`
Core number	Component	G_u	`kcore`
Edge rankings, abstraction candidates	Component	G_u, G	`optimizer`
Louvain communities	Cluster	G_u	`communities`
Bow-tie decomposition	Cluster	G	`graph-theory`
Rich-club coefficient	Cluster	G_u	`graph-theory`
Spectral radius, epidemic threshold	Whole graph	G_u	`epidemic`
Out-degree entropy	Whole graph	G	`audit`
Cycles	Whole graph	G	`audit`
Version conflicts	Whole graph	components	`audit`
Namespace concentration	Whole graph	components	`audit`
Unmaintained packages	Whole graph	components	`audit`
In-degree power-law fit	Whole graph	G	`graph-theory`
Small-world coefficient	Whole graph	G_u (largest component)	`graph-theory`

7 Evaluation

This chapter evaluates SBOM Lens on the production dependency graphs of 90 real npm applications. It asks three research questions, derived from the fourth research objective (Section 1.3): what these dependency graphs look like (RQ1), whether a structural view of their vulnerable components orders them differently from the severity ranking a flat inventory provides (RQ2), and how much of their vulnerability exposure is hidden from a flat inventory (RQ3). Section 7.1 describes how the dataset was collected, Section 7.2 states the research questions and the measures that answer them, Section 7.3 to Section 7.5 report the results, Section 7.6 reports the analysis time, and Section 7.7 discusses the threats to validity. Every number in this chapter is computed from the published dataset (Section 7.8).

7.1 Dataset and Collection Pipeline

The dataset was built by a deterministic pipeline that selects projects, generates their SBOMs, and analyses them with SBOM Lens exactly as a user would. Its design decisions are recorded in the thesis repository as architecture decision records; this section summarises them.

Candidates. The candidates are JavaScript and TypeScript repositories on GitHub with at least 100 stars that are neither archived nor forks: the roughly 10,000 most-starred of each language returned by the GitHub search API, 21,915 repositories in total. The list was fixed before processing began, so every run visits the candidates in the same order.

Release. For each candidate, the pipeline takes the latest GitHub release, excluding drafts and pre-releases, published on or before 31 March 2024. Analysing a release that is at least two and a half years old ensures that its dependencies have had time to accumulate disclosed vulnerabilities, and fixes the project to a well-defined state: the software as it was shipped at that release.

Filters. At the release tag, a candidate is accepted only if its repository root contains a `package.json` and a committed lockfile, it is not a monorepo — in which "the applica-

tion" and its root are ill-defined — and it declares at least one production dependency. The accepted lockfiles are `package-lock.json`, `npm-shrinkwrap.json`, `yarn.lock` and `pnpm-lock.yaml`; a monorepo is recognised by a `workspaces` field in `package.json`, a `pnpm-workspace.yaml` or a `lerna.json`. A lockfile is required because it pins the entire transitive dependency tree: without one, an SBOM generator would resolve dependencies against today's registry rather than as shipped.

SBOM generation. The repository is cloned at the tag and cdxgen 12.8.5 generates a CycloneDX SBOM from the root lockfile only, ignoring the separate lockfiles that many repositories keep in example or documentation folders. From that SBOM, the pipeline derives the *production dependency graph*: the part of the graph reachable from the root through the dependencies the application declares for production (`dependencies` and `optionalDependencies`), excluding development and test dependencies, which are not shipped (Pashchenko et al., 2018). The graph is kept only if it has between 50 and 1,000 nodes: below 50, the whole-graph measures of Chapter 6 are not meaningful; above 1,000, the application falls outside the scale for which the evaluation was designed.

Vulnerabilities. Vulnerabilities are obtained from OSV by running SBOM Lens's own `vulnerabilityAPI.js` under Node.js, so that the evaluation uses exactly the lookup and severity derivation the tool uses (Chapter 5). The queries were made on 30 September 2026. A project is accepted only if at least one of its components has a CRITICAL vulnerability, so that every project in the dataset contains the most serious case the evaluation is concerned with; all of its findings, whatever their severity, are kept for the analysis.

Analysis. Each accepted SBOM is uploaded to the SBOM Lens backend at a fixed commit, which is then queried through its REST API for every whole-SBOM analysis of Chapter 6 and for the impact analysis (Section 5.5.2) of every vulnerable component.

Table 7.1 shows how the candidates were filtered. The pipeline processed the candidates in order until 90 were accepted, which took 1,281 candidates. The largest losses are repositories without a GitHub release in the time window — typically lists, tutorials or projects created after March 2024 — and monorepos.

Table 7.1: *Selection funnel: candidates rejected at each stage, in the order the filters are applied.*

Stage	Rejected	Reason
Release	490	No release on or before 31 March 2024
Layout	59	No `package.json` at the repository root
Lockfile	135	No committed lockfile at the release tag
Monorepo	232	Monorepo
Production	103	No production dependencies declared
SBOM generation	24	Generator failure (1); root without edges (23, `pnpm`)
Size	115	Fewer than 50 (102) or more than 1,000 (13) nodes
Vulnerable	2	No known vulnerability
Severity	30	Known vulnerabilities, but none CRITICAL
Error	1	Repeated GitHub API failure
Accepted	**90**	

The 90 accepted projects are widely used applications and developer tools — among them excalidraw, uptime-kuma, prettier, Signal Desktop, MetaMask, code-server and CyberChef. Because candidates were processed most-starred first, all have at least 10,968 stars (median 19,059). Fifty are written in TypeScript and forty in JavaScript; 53 use `package-lock.json`, 36 `yarn.lock` and one `pnpm-lock.yaml`. Most analysed releases are recent — 51 from 2024 and 22 from 2023 — but inactive projects contribute older ones, back to 2017. Together, the 90 graphs contain 2,668 vulnerable components with 8,868 findings: 462 CRITICAL, 4,081 HIGH, 3,622 MEDIUM and 703 LOW. No finding fell in the unknown band. These are advisory–component pairs as OSV returns them; `VulnStore`, which keeps one record per CVE ID and component (Section 5.2), would store 8,743, since 125 distinct advisories share a CVE alias and a component with another, without changing any component's worst severity band.

7.2 Method and Research Questions

RQ1: What do the production dependency graphs of real npm applications look like? RQ1 characterises the dataset with the measures of Chapter 6: size, depth, the share of components the application declares directly, out-degree entropy, cycles and strongly connected components, version conflicts, the fitted power-law exponent of the in-degree distribution, the small-world coefficient with its clustering and path length, the presence of a bow-tie core, and the spectral radius with its epidemic threshold. It establishes whether ecosystem-scale properties reported in the literature (Section 3.2.1) also hold for the dependency graph of a single application.

RQ2: Does a structural view order a project's vulnerable components differently from a flat inventory? A flat inventory shows the analyst each vulnerable component with its severity, and the natural way to prioritise is by severity band. RQ2 compares that ordering with two structural ones. For every SBOM, its vulnerable components are ranked three times: by their worst severity band (CRITICAL > HIGH > MEDIUM > LOW); by the size of their blast radius excluding the component itself, i.e. the number of components that depend on them directly or transitively; and by their HITS authority score (Section 6.1.2). Agreement between the severity ranking and each structural ranking is measured with Kendall's τ_b, which accounts for the many ties that four severity bands produce: τ_b (Kendall, 1938; Kendall, 1945) is positive when the structural ranking tends to order the components as the severity bands do and negative when it tends to reverse them (with ties, $|\tau_b|$ cannot reach 1), and $\tau_b \approx 0$ that the two orderings are unrelated. As a second, more practical measure, let k be the number of CRITICAL vulnerable components of an SBOM: the *top-k overlap* is the share of those k components that are also among the k components ranked highest by the structural measure (when components tie at the k-th place, the expected share under random tie-breaking). It answers the question: if the analyst had only the time to look at k components, how many of the ones severity would point to would the structural view also point to?

RQ3: How much vulnerability exposure is hidden from a flat inventory? RQ3

measures three kinds of exposure that a list of components and CVEs does not show. The *transitive-only share* is the share of vulnerable components that the application does not declare directly, identified from the shortest dependency path from the root returned by the impact analysis. A *vulnerable chokepoint* is a vulnerable component whose dominated set is non-empty (Section 5.5.3): part of the application can be reached only through it; RQ3 reports how many vulnerable components are chokepoints and what share of the graph they dominate. The *vulnerability reach* is the share of the graph's non-root nodes that lie in the blast radius of at least one vulnerable component; the root is excluded because it lies in every blast radius by construction.

Every measure yields one value per SBOM. Results are reported as the median, the interquartile range (Q1–Q3) and the range over the 90 SBOMs, so that a handful of unusual projects does not dominate the picture. The evaluation is descriptive: it reports what the measures show across the dataset and makes no claim of statistical significance.

7.3 RQ1: Dependency Graph Profile

Table 7.2 summarises the 90 graphs and Figure 7.1 shows three of the distributions.

Table 7.2: *Profile of the 90 production dependency graphs (RQ1).*

Measure	n	Median	Q1	Q3	Min	Max
Nodes	90	332	199	492	63	995
Edges	90	534	283	943	79	2,424
Maximum depth	90	8	7	9	4	12
Direct dependencies (share of nodes)	90	0.11	0.08	0.18	0.02	0.38
Out-degree entropy (bits)	90	2.29	2.11	2.42	1.68	2.86
Cycles	90	0	0	2	0	16
Non-trivial SCCs	90	0	0	1	0	5
Version conflicts	90	24	11	46	0	137
Power-law exponent α	79	2.58	2.50	2.78	2.09	2.99
$x_{\min}$	79	1	1	2	1	3
Kolmogorov–Smirnov D	79	0.024	0.018	0.037	0.007	0.089
Small-world σ	90	16.75	9.21	24.84	2.18	44.26
Average clustering C	90	0.171	0.144	0.203	0.046	0.284
Average path length L	90	5.01	4.74	5.33	3.09	6.50
Spectral radius λ_1	90	9.24	7.18	11.53	4.68	20.55
Epidemic threshold $1/\lambda_1$	90	0.108	0.087	0.139	0.049	0.214

Size and depth. The median graph has 332 nodes and 534 edges, and its longest shortest path from the root has 8 hops. Only 11% of the nodes of the median graph are direct dependencies; the remaining 89% enter the application transitively. This is the structural fact behind the log4shell experience (Section 3.3.2): most of what an application ships was never chosen by its developers.

Anomalies. Cycles are rare but not absent: the median graph has none, but 30 of the 90 graphs contain at least one non-trivial strongly connected component, and

for those the bow-tie decomposition of Section 6.2.2 is defined. Version conflicts, by contrast, are pervasive, with a median of 24 packages present in more than one version and up to 137. This reflects npm's resolution strategy, which installs a second copy of a package when two consumers require incompatible versions, and means that the same vulnerable package frequently has to be patched in several places.

Degree distribution. A power law could be fitted to the in-degree distribution of 79 graphs. In three, the in-degrees take too few distinct values. In the other eight, no candidate lower bound $x_{\min}$ gives an estimate inside the `powerlaw` package's default range $\alpha \leq 3$, and the package marks the fit as failed; SBOM Lens returns these fits with $\alpha = 3$ (Section 6.3.7), and they are counted here as having no fit. The fitted exponents are clustered around $\alpha = 2.58$ (interquartile range 2.50–2.78) and lie between 2.09 and 2.99, inside the range typical of scale-free networks (Section 2.4.1). The lower bound $x_{\min}$ is 1 in 59 of the 79 graphs, where the fit covers every component with at least one consumer; in the other 20 it is 2 or 3 and the fitted tail holds 11% to 40% of those components. The Kolmogorov–Smirnov distances are small (median $D = 0.024$). As Section 6.3.7 cautions, this is a description of the tail rather than a test against alternative distributions, it is consistent with a few components in each application being depended upon by many others, but does not show that the tail follows a power law rather than another heavy-tailed distribution, as in the ecosystem-wide studies of Section 3.2.1.

Small-world structure. Every graph has a small-world coefficient above 2 (median $\sigma = 16.8$): clustering is far higher than in a random graph of the same size (median $C = 0.17$), while any two components remain about five hops apart (median $L = 5.0$). Given the coarse random baseline (Section 6.3.8), the size of σ is less informative than its consistency: across all 90 applications, components form tightly interlinked neighbourhoods rather than a tree. The bound itself is weak: with edge densities around $p \approx 0.01$, the random-graph clustering $C_{\text{rand}} = p$ is so small that, in the median graph, any clustering above about 0.02 already gives $\sigma > 2$.

Spread potential. The spectral radius ranges from 4.7 to 20.6 (median 9.2), so the epidemic threshold $1/\lambda_1$ ranges from 0.05 to 0.21. Because λ_1 grows with the concentration of edges on hubs (Section 6.3.1), these values are consistent with the heavy-tailed degree distributions above; they also give each application a single index of spread potential that can be compared across versions.

In short, the production dependency graph of a single npm application reproduces, at a scale of hundreds of components, two properties the literature reports for entire ecosystems, a heavy-tailed in-degree distribution and a large majority of transitive dependencies, together with the small-world structure reported for the internal structure of software (Valverde et al., 2002; Myers, 2003). The structural measures of Chapter 6 therefore have structure to work with at the level of one application.

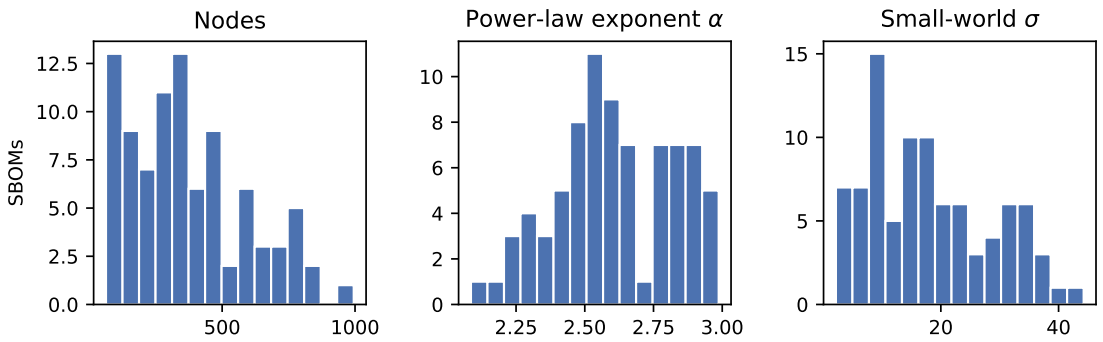


Figure 7.1: *Distributions over the 90 SBOMs of the number of nodes, the fitted power-law exponent α of the in-degree distribution (79 SBOMs), and the small-world coefficient σ.*

7.4 RQ2: Structural Ranking versus Severity

Table 7.3 and Figure 7.2 compare, for each SBOM, the ordering of its vulnerable components by severity band with their ordering by blast-radius size and by HITS authority. The median SBOM has 25 vulnerable components, of which 3 are CRITICAL.

Table 7.3: *Agreement between the severity-band ranking and structural rankings of vulnerable components (RQ2).*

Measure	n	Median	Q1	Q3	Min	Max
Vulnerable components	90	25	17	40	3	138
τ_b: severity band vs blast-radius size	90	0.00	-0.17	0.11	-1.00	0.43
τ_b: severity band vs HITS authority	79	0.00	-0.14	0.14	-0.50	0.46
Top-k overlap: blast-radius size	90	0.00	0.00	0.16	0.00	0.58
Top-k overlap: HITS authority	79	0.00	0.00	0.24	0.00	0.70

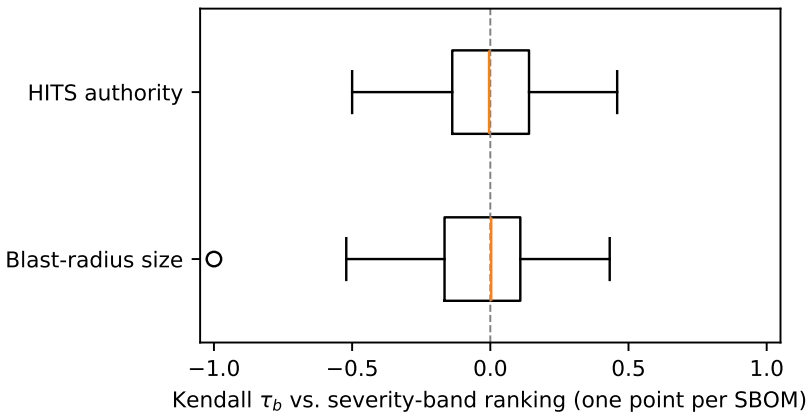


Figure 7.2: *Kendall τ_b between the severity-band ranking of each SBOM's vulnerable components and their ranking by blast-radius size (90 SBOMs) and by HITS authority (79 SBOMs).*

The two views are essentially unrelated. For blast-radius size, the median τ_b is 0.00, and half of the SBOMs lie between -0.17 and 0.11; 64 of the 90 have $|\tau_b| < 0.2$, and 38 have $|\tau_b| < 0.1$. Where $|\tau_b| \geq 0.2$, the association is more often negative than

positive: 17 SBOMs have $\tau_b \leq -0.2$ and 9 have $\tau_b \geq 0.2$. HITS authority gives the same picture, with a median of 0.00 and 13 SBOMs above 0.2 against 14 below -0.2. The practical measure shows what this means for an analyst. In the median SBOM, none of the CRITICAL components is among the equally many components with the largest blast radius, or with the highest authority: the top-k overlap is zero in 61 of the 90 SBOMs for blast radius and in 47 of the 79 rankable SBOMs for authority. With so few CRITICAL components, an unrelated ranking would place on average a share k/N of them in the top k (a mean of 0.13); the observed means are 0.09 for blast radius and 0.12 for authority, so the overlap restates the τ_b result in practical terms rather than adding to it.

For HITS authority, 11 SBOMs could not be ranked. In 8, power iteration had not converged after 200 iterations, because the two largest singular values of their adjacency matrices are close, and SBOM Lens then reports every score as zero (Section 6.1.2). In the other 3, HITS converged, but every vulnerable component has authority zero, because authority is concentrated on a part of the graph that contains none of them. Blast radius, which is computed by a single traversal, is defined for every SBOM.

This result must be read for what it is. A τ_b near zero does not show that the structural ranking is better than the severity ranking, or the reverse; it shows that the two measure different things, and that knowing a component's severity tells an analyst almost nothing about where that component sits in the application. A CRITICAL vulnerability is as likely to be in a package that a single other component uses as in a component through which much of the application is reached, and many of the components whose compromise would affect the largest part of the application carry only HIGH or MEDIUM vulnerabilities. This is the information a flat inventory cannot provide, and it is precisely what the central claim of Chapter 6 asserts.

7.5 RQ3: Hidden Exposure

Table 7.4 and Figure 7.3 report the three measures of exposure that a flat inventory does not show.

Table 7.4: *Hidden exposure (RQ3).*

Measure	n	Median	Q1	Q3	Min	Max
Transitive-only share of vulnerable components	90	0.75	0.65	0.86	0.31	1.00
Vulnerable chokepoints	90	12	8	18	1	48
Vulnerable chokepoints (share of vulnerable components)	90	0.45	0.38	0.55	0.18	0.83
Nodes dominated by vulnerable chokepoints (share)	90	0.214	0.114	0.349	0.015	0.940
Vulnerability reach (share of non-root nodes)	90	0.19	0.15	0.22	0.09	0.35

Transitive-only vulnerable components. In the median SBOM, 75% of the vulnerable components are not declared by the application; they arrive through its dependencies. The share is at least one half in 83 of the 90 SBOMs, and in seven every vulnerable

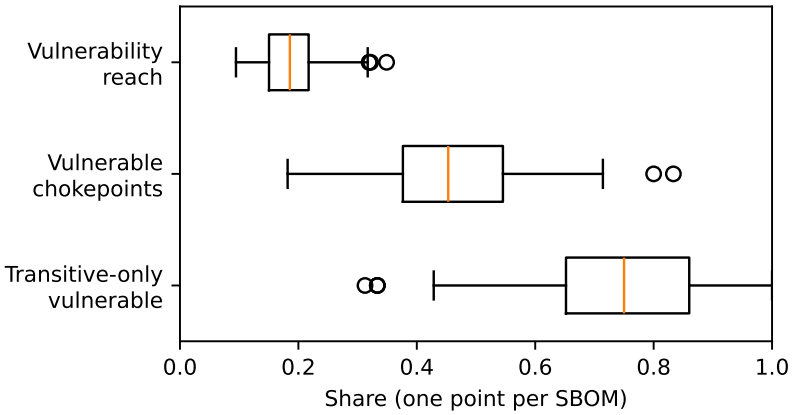


Figure 7.3: *Transitive-only share of vulnerable components, share of vulnerable components that are chokepoints, and vulnerability reach, over the 90 SBOMs.*

component is transitive-only. An analyst who inspected only the application's declared dependencies would miss most of its vulnerable code, and one who sees the vulnerable components in a flat list still has to work out, for each, how it entered the application — the dependency path SBOM Lens shows directly.

Vulnerable chokepoints. Every one of the 90 SBOMs contains at least one vulnerable chokepoint, with a median of 12. All 986 vulnerable articulation points are chokepoints, but 200 of the 1,186 vulnerable chokepoints are not articulation points, so the undirected measure would miss a sixth of them (Section 6.1.1). Almost half of all vulnerable components (median 45%) are chokepoints: parts of the application are reachable only through them, so those parts arrive in the application only through them. Together, the vulnerable chokepoints of the median SBOM dominate 21% of its graph, and in one project 94%. Removing or replacing such a component also removes the parts it dominates, which are its own dependencies; a flat inventory gives no indication of which components these are.

Vulnerability reach. In the median SBOM, 19% of the non-root components lie in the blast radius of at least one known vulnerability, ranging from 9% to 35%. Reach is a measure of how much of the application inherits some known vulnerability. It includes the vulnerable components themselves (a median 8%); counting only components that depend on at least one vulnerable component gives a median of 12% (4% to 29%).

Taken together, the three measures show that the exposure an SBOM describes is mostly structural. Most vulnerable components are hidden behind other dependencies, nearly half of them are the only route to some of the application's dependencies, and in the median application a fifth of the components is vulnerable or depends on a vulnerable component, and an eighth depends on one.

7.6 Performance

Table 7.5 reports the time each analysis took per SBOM, measured by the collection pipeline on a single workstation, and Figure 7.4 relates the total time to graph size.

Table 7.5: *Analysis time per SBOM in seconds, by endpoint.*

Measure	n	Median	Q1	Q3	Min	Max
audit	90	4.720	1.948	7.649	0.492	16.387
communities	90	0.018	0.010	0.028	0.004	0.057
epidemic	90	0.022	0.012	0.099	0.003	2.910
global analysis	90	0.015	0.010	0.019	0.004	0.042
graph analysis	90	0.002	0.001	0.002	0.001	0.004
graph theory	90	0.067	0.036	0.130	0.007	0.855
impact (all vulnerable nodes)	90	0.151	0.091	0.290	0.013	0.990
kcore	90	0.005	0.004	0.007	0.002	0.030
metrics	90	0.002	0.002	0.003	0.001	0.010
optimizer	90	0.030	0.018	0.120	0.003	2.064
Total per SBOM	90	5.28	2.15	9.55	0.54	17.18

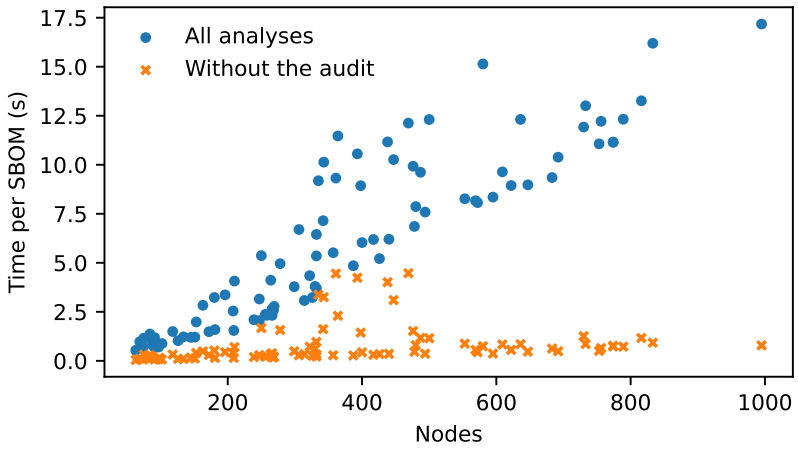


Figure 7.4: *Total analysis time per SBOM against its number of nodes, with and without the audit, whose time is dominated by queries to the npm registry.*

A complete analysis of one SBOM took a median of 5.3 seconds and at most 17.2 seconds. Nearly all of it is the audit, whose unmaintained-package check (Section 6.3.6) queries the npm registry once per package, so the audit's time grows with the number of packages and with network latency. Every other analysis had a median between 2 and 150 milliseconds, including the impact analysis of all vulnerable components of an SBOM. The slowest single runs were of the two spectral analyses, at most 2.9 seconds; they occurred on graphs of 250 to 476 nodes, while on the graphs above 500 nodes these analyses took under 0.07 seconds, so the peaks are not an effect of size. At the scale of a single application, the structural analysis of Chapter 6 is therefore interactive.

7.7 Threats to Validity

Construct validity. The severity-band ranking stands in for how an analyst would prioritise from a flat inventory; tools that rank by numeric CVSS score, or combine severity with exploit likelihood, would produce finer orderings, whose agreement with the structural rankings was not measured. Both structural measures describe exposure at the level of components: a component in the blast radius of a vulnerable component depends on it, but may never execute the vulnerable code (Section 3.2.3), so blast radius and reach over-approximate actual exposure. HITS authority could not rank the vulnerable components of 11 of the 90 graphs (Section 7.4); the RQ2 results for authority, including the top-k overlap, rest on the remaining 79.

Internal validity. The dependency graphs are those cdxgen generates, and generators are imperfect. For the 53 projects with an npm lockfile, the production graph was compared with the lockfile's own marking of production packages: in the median graph, 99.9% of the nodes are marked as production in the lockfile, and 23 graphs match it exactly, but four graphs fall below 90% agreement, the lowest at 75%, because cdxgen occasionally links a production package to a different installed copy of a dependency than npm resolves. In the other direction, 44 of the 53 graphs contain every package the lockfile marks as production, but two miss more than a tenth of them (14% and 20%). In seven SBOMs, the generated root also lacks one to five of the production dependencies declared in `package.json`, so their subtrees are absent and the size, reach and vulnerability counts of those graphs are lower bounds. These graphs were kept as generated, because the evaluation concerns SBOMs as real tools produce them; the graphs of the 37 projects with `yarn` or `pnpm` lockfiles could not be checked in this way. Vulnerability data reflects OSV's state on the query date; advisories published later, or recorded only outside OSV, are not included. Conversely, most of the advisories that are included post-date the release analysed: 8,232 of the 8,868 findings (93%), and every CRITICAL finding in 60 of the 90 applications, were published after it — an upper bound, since an advisory can be published later than the original disclosure. The evaluation therefore describes where vulnerabilities known today sit in each release's graph, not what an analyst could have known at the release.

External validity. The dataset is deliberately narrow: popular npm applications (at least 10,968 stars), with a committed lockfile, not organised as monorepos, with 50 to 1,000 production components, and with at least one CRITICAL known vulnerability, analysed at releases from 2017 to 2024. Each filter shapes the result. The generator shaped it too: cdxgen produced no dependency edges from the root for 23 projects with a `pnpm` lockfile, so only one `pnpm` project remains, and the findings describe `npm` and `yarn` installations. npm's nested installation of duplicate versions inflates version conflicts relative to ecosystems such as Maven, which install a single version; very large applications and monorepos are excluded; and projects without critical vulnerabilities may have different structures. The findings describe vulnerable, medium-sized npm applications, and their replication for other ecosystems is left as future work (Section 8.4).

Conclusion validity. The evaluation is descriptive and reports distributions rather than hypothesis tests. Per-SBOM rank correlations rest on few components when an SBOM has few vulnerable components — as few as three — which widens the spread of τ_b, visible in the extreme values of Figure 7.2; the medians and interquartile ranges, which the conclusions rely on, are robust to such cases.

7.8 Reproducibility

The collection pipeline, the list of candidates in processing order, the rejected candidates with their reasons, the 90 SBOMs, every SBOM Lens response, and the analysis script that produces the tables and figures of this chapter are published in the `sbom-lens-eval` repository. Each run records the versions of the tools it used: SBOM Lens at commit `dfc8e23`; its analysis code and `vulnerabilityAPI.js` are unchanged in commit `28113e7`, the version described in Chapters 4 to 6, which adds only interface features and an endpoint the evaluation does not use; cdxgen 12.8.5 and Node.js 20.20.2. The backend ran with Python 3.14.4, NetworkX 3.7, SciPy 1.18.1, NumPy 2.5.3 and powerlaw 2.0.0; the run record does not include these library versions, and the NetworkX version affects the HITS results (Section 6.1.2). Rerunning the analysis script on the published data regenerates Table 7.2 to Table 7.5 and every figure of this chapter; the selection funnel and the remaining counts in the text are computed from the same data; rerunning the collection reproduces the dataset up to changes in the vulnerability databases and in the GitHub repositories themselves.

8 Conclusion

This thesis set out to show that the dependency relationships recorded in a Software Bill of Materials are an analytical resource, not just a record. The SBOM tools surveyed in Section 3.1 treat an SBOM as an inventory, at most tracing the paths to a component or listing its dependents; SBOM Lens builds a directed dependency graph and computes dominance and structural measures over it. This chapter summarises what the evaluation found (Section 8.1), revisits the research objectives of Section 1.3 against the contributions made (Section 8.2), states the limitations of the work (Section 8.3), and identifies directions for future work (Section 8.4).

8.1 Summary of Findings

The evaluation of Chapter 7 applied SBOM Lens to the production dependency graphs of 90 popular npm applications, each with at least one critical known vulnerability, and answered three research questions.

RQ1: the dependency graph of a single application has the heavy-tailed structure found in whole ecosystems. The median graph has 332 components, of which only 11% are declared by the application itself. A power law could be fitted to the in-degree distribution of 79 of the 90 graphs, with exponents between 2.09 and 2.99 (median 2.58), and every graph is small-world against a weak random baseline: components form tightly clustered neighbourhoods while remaining about five undirected hops apart. Version conflicts are pervasive (median 24 per application), and 30 applications contain dependency cycles. The structural measures of Chapter 6 therefore have structure to work with at the scale of one application.

RQ2: severity says almost nothing about where a vulnerable component sits. Ranking each application's vulnerable components by severity band and by structural position gives essentially unrelated orderings: the median Kendall τ_b is 0.00 both for blast-radius size and for HITS authority (in the 79 graphs where it is defined), so that in most applications none of the critical components is among the equally many components with the largest blast radius. A critical vulnerability is as likely to affect a

package with a single consumer as a component through which much of the application is reached.

RQ3: most vulnerability exposure is structural, and a flat inventory does not show how it arises. In the median application, 75% of the vulnerable components are not declared by the application; every application has vulnerable chokepoints — vulnerable components that parts of the application can be reached only through — and in the median application they are 45% of the vulnerable components and dominate 21% of the graph; and 19% of the application's components are vulnerable or depend on a vulnerable component (12% depend on one).

Together, these results support the central claim of the thesis: the position of a component in the dependency graph carries risk information that a flat inventory of components and vulnerabilities does not, and SBOM Lens makes that information available at interactive speed: on graphs of up to a thousand components, every analysis except the audit took milliseconds in the median case and under three seconds at most; the audit, which also queries the npm registry once per package, took up to 16 seconds.

8.2 Contributions

The four research objectives of Section 1.3 map one-to-one to the contributions of this work; each was met, with the qualification stated for RO3.

Graph construction pipeline (RO1). SBOM Lens ingests CycloneDX JSON SBOMs and constructs a directed dependency graph in which every component is a node and every declared dependency a consumer-to-dependency edge (Chapter 4). The graph is built once at ingestion, cached, and shared by every analysis, and it is rendered interactively with Cytoscape.js, where the analyst can explore, edit and export it. Edits are written back to the backend, which rebuilds the graph and invalidates every cached analysis, so the structural results always describe the current graph.

Vulnerability integration (RO2). Every component with a Package URL is matched against OSV, which aggregates the GitHub Advisory Database and other open-source advisory sources (Chapter 5). Severity is derived from the CVSS v3 vector by a client-side implementation of the v3.1 specification, falling back, when no such vector is present, to the advisory's own severity label, and otherwise recorded as unknown rather than silently treated as low. Vulnerable components are coloured on the graph by their highest severity band, and for any vulnerable component the analyst obtains its blast radius, up to five shortest dependency paths through which it enters the application, and its dominated set — the components that could not be reached without it.

Graph-based structural risk analysis (RO3). SBOM Lens computes structural measures at three scales (Chapter 6): per component (articulation points and bridges, HITS scores, a spectral exposure score, core numbers), per cluster (Louvain communities, the bow-tie decomposition, the rich-club coefficient), and for the whole graph (the spectral radius and epidemic threshold, degree entropy, cycles, version conflicts,

namespace concentration, the in-degree distribution and the small-world coefficient). For each measure, the thesis states precisely what is computed, on which graph, and how the result should be read — including where a measure is only a coarse indicator. The evaluation shows that these measures carry information that severity does not; it does not show that prioritising by them leads to better remediation, which would need exploitation data or analyst outcomes.

Empirical evaluation (RO4). The evaluation of Chapter 7 applies SBOM Lens to the production dependency graphs of npm applications, collected reproducibly from public repositories at fixed release tags and restricted to applications with at least one critical known vulnerability. The collection pipeline, the SBOMs, every analysis result and the analysis script are published in the `sbom-lens-eval` repository, from which the tables and figures of Chapter 7 can be regenerated. On 90 applications, the structural and severity rankings of vulnerable components were found to be essentially unrelated (median $\tau_b = 0.00$), and three-quarters of the vulnerable components to be transitive-only.

Beyond the objectives, the work contributes a clarification of the vocabulary of structural SBOM analysis: it distinguishes the blast radius of a component from its dominated set, chokepoints from articulation points, version conflicts from the diamond pattern that causes them, and static spectral analysis from propagation simulation. Where the interface uses a name that suggests more than is computed — the `r0` field, the *Diamond Dependencies* label — the thesis states what is actually computed.

8.3 Limitations

Vulnerability data. SBOM Lens relies on OSV as its only vulnerability source. OSV aggregates the main open-source advisory databases, but an advisory reaches SBOM Lens only once OSV has ingested it, and vulnerabilities recorded only in NVD, without an open-source advisory, are not reported. Severity is computed from CVSS v3 vectors only; records that carry only a CVSS 4.0 vector fall back to the advisory's own label or to the unknown band. Exploit likelihood (EPSS) and known-exploitation catalogues are not integrated, so severity describes potential impact rather than the probability of exploitation.

Embedded vulnerability data (VEX). SBOM Lens ignores the `vulnerabilities` array of a CycloneDX document (Section 2.1.3), which may carry Vulnerability Exploitability eXchange statements from the software's producer. Querying OSV directly gives every analysis the same vulnerability data, current at upload, regardless of what the producer chose to embed, and avoids trusting producer-supplied assessments. The cost is that a producer's statement that a vulnerability is not exploitable in their product — often the most useful information for prioritisation — is not taken into account, so SBOM Lens may report vulnerabilities that the producer has already assessed as irrelevant.

Component-level analysis. All analysis is at the level of components. A com-

ponent in the blast radius of a vulnerability depends on the vulnerable component, but may never execute the vulnerable code (Section 3.2.3); exposure is therefore over-approximated.

Directed and undirected measures. Several measures — articulation points, bridges, core numbers, communities, the rich-club coefficient, the small-world coefficient and all spectral measures — are computed on the undirected projection of the dependency graph, either because they are defined only for undirected graphs or because, as for the spectral radius, they vanish on acyclic directed graphs (Chapter 6). These measures describe how densely components are interconnected, not the direction in which dependencies run, and they must be read accordingly.

Heuristic thresholds and baselines. Some outputs rest on conventions rather than theory: the labels of the degree entropy, the 60% namespace-concentration flag, the 0.5 rich-club threshold, and the fan-in threshold for abstraction candidates. The small-world coefficient uses a closed-form random baseline that ignores the degree sequence, the rich-club coefficient is not normalised against a null model, and the power-law fit is not compared against alternative distributions. These outputs are reported as descriptive indicators, not as statistical tests.

Numerical edge cases. When HITS power iteration does not converge, SBOM Lens reports every score as zero, and when no valid power-law fit exists it returns the package's bound $\alpha = 3$; neither case is flagged. In the evaluation each occurred in 8 graphs, and the HITS results, which rest on 79 of the 90 graphs, depend on the NetworkX version (Section 6.1.2).

Evaluation sample. The evaluation covers popular npm applications only, not organised as monorepos, at release tags on or before 31 March 2024, with a committed lockfile, between 50 and 1,000 components in their production dependency graph, and at least one critical known vulnerability. Each of these filters shapes the sample, and so does the SBOM generator, which produced no usable graph for all but one `pnpm` project and an incomplete one, whose counts are lower bounds, for seven applications. The results describe vulnerable, medium-sized npm applications and may not transfer to other ecosystems, where duplicate versions are rarer, or to very large applications. Vulnerability data reflects OSV on 30 September 2026, so it includes advisories published after each release, which an analyst could not have seen at the time: 93% of the findings, and every critical finding in 60 of the 90 applications (Section 7.7).

Implementation. SBOM Lens is a research prototype. It keeps all state in memory, so SBOMs and vulnerability records are lost when the backend restarts; it has no authentication and is intended as a single-user local tool; it accepts CycloneDX JSON only; and it queries OSV only when an SBOM is uploaded, so vulnerability records are not refreshed and a component added by editing is not looked up.

8.4 Future Work

Propagation simulation. SBOM Lens analyses the dependency graph statically. A natural extension is to simulate how a compromise spreads over time, with an epidemic model such as SIR run on the dependency graph (Section 2.4.6). The spectral radius already computed by SBOM Lens would then serve as a check on the simulation, and the simulated reach could be compared with the static vulnerability reach reported in Chapter 7.

Richer vulnerability data. Integrating EPSS scores and known-exploitation catalogues would let structural measures be combined with the likelihood of exploitation, not only with severity. Direct integration of NVD, despite the imprecision of CPE matching (Section 5.1.1), and of the GitHub Advisory Database by version range would reduce dependence on OSV's aggregation, and a CVSS 4.0 calculator would remove the unknown band for records that carry only a version-4 vector. Consuming VEX statements from the SBOM would let the producer's exploitability assessments suppress irrelevant findings.

Reachability. Combining the component-level graph with call-graph reachability (Section 3.2.3) would prune the vulnerability overlay to vulnerabilities whose code the application can actually execute, where source code is available.

Broader evaluation. Replicating the evaluation for PyPI and Maven applications would test whether the findings hold in ecosystems with different dependency resolution rules. A user study with security analysts would test the question the metric-based evaluation cannot: whether the structural view changes, and improves, the decisions analysts actually make.

Platform. Persistent storage, support for SPDX input, and a version history for each application, extending the pairwise SBOM comparison view outside the scope of this thesis (Section 4.1.1), would turn the prototype into a tool for continuous use, in which the structural risk of an application could be tracked as its dependencies evolve.

This page intentionally left blank.

Bibliography

Alstott, Jeff, Ed Bullmore, and Dietmar Plenz (2014). "powerlaw: A Python Package for Analysis of Heavy-Tailed Distributions". In: *PLoS ONE* 9.1, e85777. DOI: 10.1371/journal.pone.0085777.

Anchore (n.d.[a]). *Grype: A Vulnerability Scanner for Container Images and Filesystems*. URL: https://github.com/anchore/grype (visited on 2026-10-09).

Anchore (n.d.[b]). *Syft: CLI Tool and Library for Generating a Software Bill of Materials*. URL: https://github.com/anchore/syft (visited on 2026-10-09).

Baird, Laura and Armin Moin (2026). *Cascaded Vulnerability Attacks in Software Supply Chains*. arXiv: 2601.20158.

Barabási, Albert-László (2016). *Network Science*. Cambridge University Press.

Barabási, Albert-László and Réka Albert (1999). "Emergence of Scaling in Random Networks". In: *Science* 286.5439, pp. 509–512. DOI: 10.1126/science.286.5439.509.

Batagelj, Vladimir and Matjaz Zaversnik (2003). *An $O(m)$ Algorithm for Cores Decomposition of Networks*. arXiv: cs/0310049.

Blondel, Vincent D. et al. (2008). "Fast Unfolding of Communities in Large Networks". In: *Journal of Statistical Mechanics: Theory and Experiment* 2008.10, P10008. DOI: 10.1088/1742-5468/2008/10/P10008.

Broder, Andrei et al. (2000). "Graph Structure in the Web". In: *Computer Networks* 33.1–6, pp. 309–320. DOI: 10.1016/S1389-1286(00)00083-9.

Clauset, Aaron, Cosma Rohilla Shalizi, and M. E. J. Newman (2009). "Power-Law Distributions in Empirical Data". In: *SIAM Review* 51.4, pp. 661–703. DOI: 10.1137/070710111.

Colizza, Vittoria et al. (2006). "Detecting Rich-Club Ordering in Complex Networks". In: *Nature Physics* 2, pp. 110–115. DOI: 10.1038/nphys209.

Cormen, Thomas H. et al. (2022). *Introduction to Algorithms*. 4th ed. MIT Press.

Cybersecurity and Infrastructure Security Agency (Dec. 2020). *Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations*. Alert AA20-352A. CISA. URL: https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a.

CycloneDX (n.d.). *Sunshine: Actionable CycloneDX Visualization*. https://github.com/CycloneDX/Sunshine. Accessed 2026-09-30.

Decan, Alexandre, Tom Mens, and Eleni Constantinou (2018). "On the Impact of Security Vulnerabilities in the npm Package Dependency Network". In: *Proceedings of the 15th International Conference on Mining Software Repositories (MSR)*, pp. 181–191. DOI: 10.1145/3196398.3196401.

Decan, Alexandre, Tom Mens, and Philippe Grosjean (2019). "An Empirical Comparison of Dependency Network Evolution in Seven Software Packaging Ecosystems". In: *Empirical Software Engineering* 24.1, pp. 381–416. DOI: 10.1007/s10664-017-9589-y.

European Parliament and Council of the European Union (2024). *Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)*. Official Journal of the European Union, L series.

Forum of Incident Response and Security Teams (2019). *Common Vulnerability Scoring System v3.1: Specification Document*. URL: https://www.first.org/cvss/v3.1/specification-document.

Forum of Incident Response and Security Teams (n.d.). *Exploit Prediction Scoring System (EPSS)*. URL: https://www.first.org/epss/ (visited on 2026-10-09).

Garcia, Derek et al. (2025). "A Landscape Study of Open-Source Tools for Software Bill of Materials (SBOM) and Supply Chain Security". In: *2025 IEEE/ACM 3rd International Workshop on Software Vulnerability Management (SVM)*. Ottawa, ON, Canada: IEEE, pp. 37–45. DOI: 10.1109/SVM66695.2025.00010.

GitHub (n.d.). *GitHub Advisory Database*. URL: https://github.com/advisories (visited on 2026-10-09).

Hagberg, Aric A., Daniel A. Schult, and Pieter J. Swart (2008). "Exploring Network Structure, Dynamics, and Function using NetworkX". In: *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pp. 11–15.

Humphries, Mark D. and Kevin Gurney (2008). "Network 'Small-World-Ness': A Quantitative Method for Determining Canonical Network Equivalence". In: *PLoS ONE* 3.4, e0002051. DOI: 10.1371/journal.pone.0002051.

Johnson, Donald B. (1975). "Finding All the Elementary Circuits of a Directed Graph". In: *SIAM Journal on Computing* 4.1, pp. 77–84. DOI: 10.1137/0204007.

Kendall, Maurice G. (1938). "A New Measure of Rank Correlation". In: *Biometrika* 30.1/2, pp. 81–93. DOI: 10.2307/2332226.

Kendall, Maurice G. (1945). "The Treatment of Ties in Ranking Problems". In: *Biometrika* 33.3, pp. 239–251. DOI: 10.1093/biomet/33.3.239.

Kikas, Riivo et al. (2017). "Structure and Evolution of Package Dependency Networks". In: *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*, pp. 102–112. DOI: 10.1109/MSR.2017.55.

Kleinberg, Jon M. (1999). "Authoritative Sources in a Hyperlinked Environment". In: *Journal of the ACM* 46.5, pp. 604–632. DOI: 10.1145/324133.324140.

Ladisa, Piergiorgio et al. (2023). "SoK: Taxonomy of Attacks on Open-Source Software Supply Chains". In: *2023 IEEE Symposium on Security and Privacy (SP)*, pp. 1509–1526. DOI: 10.1109/SP46215.2023.10179304.

Liu, Chengwei et al. (2022). "Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem". In: *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, pp. 672–684. DOI: 10.1145/3510003.3510142.

Louridas, Panagiotis, Diomidis Spinellis, and Vasileios Vlachos (2008). "Power Laws in Software". In: *ACM Transactions on Software Engineering and Methodology* 18.1, 2:1–2:26. DOI: 10.1145/1391984.1391986.

Myers, Christopher R. (2003). "Software Systems as Complex Networks: Structure, Function, and Evolvability of Software Collaboration Graphs". In: *Physical Review E* 68.4, p. 046116. DOI: 10.1103/PhysRevE.68.046116.

National Institute of Standards and Technology (2021). *CVE-2021-44228 Detail*. URL: https://nvd.nist.gov/vuln/detail/CVE-2021-44228.

National Institute of Standards and Technology (2024). *CVE-2024-3094 Detail*. URL: https://nvd.nist.gov/vuln/detail/CVE-2024-3094.

National Institute of Standards and Technology (n.d.). *National Vulnerability Database*. URL: https://nvd.nist.gov (visited on 2026-10-09).

National Telecommunications and Information Administration (July 2021). *The Minimum Elements For a Software Bill of Materials (SBOM)*. Report. U.S. Department of Commerce.

Newman, Mark (2018). *Networks*. 2nd ed. Oxford University Press.

Open Source Security Foundation (2024). *GUAC: Graph for Understanding Artifact Composition*. URL: https://guac.sh.

Open Source Security Foundation (n.d.[a]). *OSV: Open Source Vulnerabilities*. URL: https://osv.dev (visited on 2026-10-09).

Open Source Security Foundation (n.d.[b]). *Supply-chain Levels for Software Artifacts (SLSA)*. URL: https://slsa.dev (visited on 2026-10-09).

OWASP Foundation (n.d.[a]). *CycloneDX Bill of Materials Standard (ECMA-424)*. URL: https://cyclonedx.org/specification/overview/ (visited on 2026-10-09).

OWASP Foundation (n.d.[b]). *OWASP Dependency-Track*. URL: https://dependencytrack.org (visited on 2026-10-09).

Package-URL Project (n.d.). *Package-URL (purl) Specification (ECMA-427)*. URL: https://github.com/package-url/purl-spec (visited on 2026-10-09).

Pashchenko, Ivan et al. (2018). "Vulnerable Open Source Dependencies: Counting Those That Matter". In: *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 42:1–42:10. DOI: 10.1145/3239235.3268920.

Pastor-Satorras, Romualdo and Alessandro Vespignani (2001). "Epidemic Spreading in Scale-Free Networks". In: *Physical Review Letters* 86.14, pp. 3200–3203. DOI: 10.1103/PhysRevLett.86.3200.

Ponta, Serena Elisa, Henrik Plate, and Antonino Sabetta (2018). "Beyond Metadata: Code-Centric and Usage-Based Analysis of Known Vulnerabilities in Open-Source Software". In: *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 449–460. DOI: 10.1109/ICSME.2018.00054.

Shannon, Claude E. (1948). "A Mathematical Theory of Communication". In: *Bell System Technical Journal* 27.3, pp. 379–423. DOI: 10.1002/j.1538-7305.1948.tb01338.x.

Souppaya, Murugiah, Karen Scarfone, and Donna Dodson (2022). *Secure Software Development Framework (SSDF) Version 1.1*. NIST Special Publication 800-218. National Institute of Standards and Technology. DOI: 10.6028/NIST.SP.800-218.

Synopsys (2024). *2024 Open Source Security and Risk Analysis Report*. Technical Report. Synopsys, Inc.

The MITRE Corporation (n.d.). *Common Vulnerabilities and Exposures (CVE)*. URL: https://www.cve.org (visited on 2026-10-09).

The White House (May 2021). *Executive Order 14028: Improving the Nation's Cybersecurity*. 86 Fed. Reg. 26633.

The White House (Jan. 2025). *Executive Order 14144: Strengthening and Promoting Innovation in the Nation's Cybersecurity*.

Valverde, Sergi, Ramon Ferrer Cancho, and Ricard V. Solé (2002). "Scale-Free Networks from Optimal Design". In: *Europhysics Letters* 60.4, pp. 512–517. DOI: 10.1209/epl/i2002-00248-2.

Van Mieghem, Piet et al. (2011). "Decreasing the Spectral Radius of a Graph by Link Removals". In: *Physical Review E* 84.1, p. 016101. DOI: 10.1103/PhysRevE.84.016101.

Wang, Yang et al. (2003). "Epidemic Spreading in Real Networks: An Eigenvalue Viewpoint". In: *22nd International Symposium on Reliable Distributed Systems (SRDS 2003)*, pp. 25–34. DOI: 10.1109/RELDIS.2003.1238052.

Watts, Duncan J. and Steven H. Strogatz (1998). "Collective Dynamics of 'Small-World' Networks". In: *Nature* 393, pp. 440–442. DOI: 10.1038/30918.

Wetter, James and Nicky Ringland (2021). *Understanding the Impact of Apache Log4j Vulnerability*. Google Security Blog, https://security.googleblog.com/2021/12/understanding-impact-of-apache-log4j.html. Accessed 2026-09-30.

Zięba-Kozarzewski, Artur (2026). *No Edges, No Verdict: A Large-Scale Empirical Study of Declared Dependency Graphs in 78K SBOMs in the Wild*. arXiv: 2607.22140.

Zimmermann, Markus et al. (2019). "Small World with High Risks: A Study of Security Threats in the npm Ecosystem". In: *28th USENIX Security Symposium (USENIX Security 19)*, pp. 995–1010.

This page intentionally left blank.